

API Reference

Jupiter System for PC API Reference



Document Version 2006.1.17
Jupiter Build 69

Legal Notices

The Jupiter™ Development System is copyright 1998-2006 by Touchdown Entertainment, Inc., Kirkland, Washington, U.S.A. All rights reserved.

ImageMagick is copyright 2000 by ImageMagick Studio, a non-profit organization dedicated to making software imaging solutions freely available.

The FreeType Project is copyright 1996-2000 by David Turner, Robert Wilhelm, and Werner Lemberg. All rights reserved except as specified in the *Credits* appendix.

API Reference

__ClassDefiner Class Reference

Public Member Functions

[__ClassDefiner](#) ([ClassDef](#) *pDef)

Data Fields

[ClassDef](#) * [m_pClass](#)
[__ClassDefiner](#) * [m_pNext](#)

Constructor & Destructor Documentation

[__ClassDefiner::__ClassDefiner](#)([ClassDef](#) * pDef) [inline]

Field Documentation

[ClassDef](#)* [__ClassDefiner::m_pClass](#)

[__ClassDefiner](#)* [__ClassDefiner::m_pNext](#)

The documentation for this class was generated from the following file:

- [Itserverobj.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

`__cpp_classlist_auto_free` Class Reference

Public Member Functions

[~__cpp_classlist_auto_free](#) ()

Constructor & Destructor Documentation

`__cpp_classlist_auto_free::~__cpp_classlist_auto_free( ) [inline]`

The documentation for this class was generated from the following file:

- [ltserverobj.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

`_LTGUID` Struct Reference

Data Fields

[uint32 a](#)
[uint16 b](#)
[uint16 c](#)
[uint8 d](#) [8]

Detailed Description

Globally unique identifier.

Field Documentation

[uint32 _LTGUID::a](#)

[uint16 _LTGUID::b](#)

[uint16 _LTGUID::c](#)

[uint8 _LTGUID::d](#)[8]

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTGUID_64 Struct Reference

Data Fields

[uint64 a](#)
[uint64 b](#)

Field Documentation

[uint64 LTGUID_64::a](#)

[uint64 LTGUID_64::b](#)

The documentation for this struct was generated from the following file:

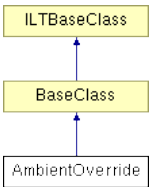
- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

AmbientOverride Class Reference

Inheritance diagram for AmbientOverride:



Detailed Description

Ambient Override class.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

AnimTimeRef Class Reference

Public Member Functions

```

    AnimTimeRef()
    void Init(Model *pModel, uint32 iAnim1=0, uint32 iFrame1=0, uint32 iAnim2=0, uint32 iFrame2=0, float percent=0.0f)
    void Clear()
    bool IsValid()
    ModelAnim * GetAnim1()
    ModelAnim * GetAnim2()
    bool SetKeyframePosition(uint32 iAnim, uint32 iFrame1, uint32 iFrame2, float fPercent)

```

Data Fields

```

    Model * m\_pModel
    FrameLocator m\_Prev
    FrameLocator m\_Cur
    float m\_Percent
    uint32 m\_iWeightSet

```

Detailed Description

References a specific time in an animation.

Constructor & Destructor Documentation

```

AnimTimeRef::AnimTimeRef( ) [inline]

```

Member Function Documentation

```

void AnimTimeRef::Clear( ) [inline]

```

```

ModelAnim* AnimTimeRef::GetAnim1( )

```

```

ModelAnim* AnimTimeRef::GetAnim2( )

```

```

void AnimTimeRef::Init( Model * pModel,
    uint32 iAnim1 = 0,
    uint32 iFrame1 = 0,
    uint32 iAnim2 = 0,
    uint32 iFrame2 = 0,
    float percent = 0.0f
) [inline]

```

```

bool AnimTimeRef::IsValid( )

```

```

bool AnimTimeRef::SetKeyframePosition( uint32 iAnim,
    uint32 iFrame1,
    uint32 iFrame2,
    float fPercent
)

```

Set it between the specified keyframes on the specified animation.

Field Documentation

[FrameLocator](#) [AnimTimeRef::m_Cur](#)

[uint32](#) [AnimTimeRef::m_iWeightSet](#)

If multiple [LTAnimTracker](#) objects are being used together, this tells how to blend each one. This is ignored for the first weight set.

float [AnimTimeRef::m_Percent](#)

Percentage between the two frames. Must be between 0 and 1.

Model* [AnimTimeRef::m_pModel](#)

[FrameLocator](#) [AnimTimeRef::m_Prev](#)

This tells exactly where we want the vertices set up.

The documentation for this class was generated from the following file:

- [ltanimtracker.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ArgList Struct Reference

Data Fields

char ** [argv](#)
int [argc](#)

Field Documentation

int [ArgList::argc](#)

char** [ArgList::argv](#)

The documentation for this struct was generated from the following file:

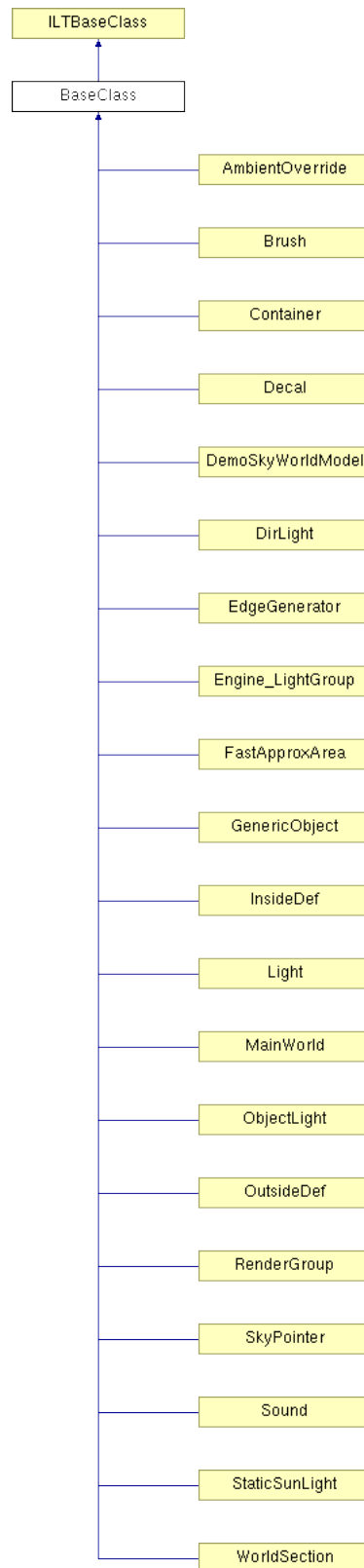
- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

BaseClass Class Reference

Inheritance diagram for BaseClass:



Public Member Functions

[BaseClass](#) ([uint8](#) nType=OT_NORMAL)
 virtual [~BaseClass](#) ()
 virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Static Public Member Functions

static [ILTServer](#) * [GetServerDE](#) ()

Detailed Description

Defines basic object functionality for all objects in a game.

All server-side objects in your game must derive from **BaseClass**.

Each object must have an object type (defined by the **OT_*** defines). Use the **SetType** function to set an object's type.

This class provides basic engine-to-object and object-to-object event notification functionality (using the [ILTBaseClass](#) notification interface, **EngineMessageFn** (obsolete) and **ObjectMessageFn**).

Generic behavior across different objects can be implemented by using aggregates. An object will forward event notification messages to all its associated aggregates. For example, a damage aggregate can be used to control damage logic for all objects that can be damaged. Use the **AddAggregate** and **RemoveAggregate** functions to implement aggregate functionality.

You can implement any custom behavior required for your derived object.

See the Object chapter of the LithTech Programming Guide for more information.

Constructor & Destructor Documentation

```
BaseClass::BaseClass( uint8 nType = OT_NORMAL ) [inline]
```

```
virtual BaseClass::~BaseClass( ) [virtual]
```

Member Function Documentation

```
virtual uint32 BaseClass::EngineMessageFn( uint32 messageID,  
                                          void * pData,  
                                          float  lData  
                                          ) [virtual]
```

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [ILTBaseClass](#).

Reimplemented in [MainWorld](#), [Container](#), [Sound](#), [StaticSunLight](#), [DemoSkyWorldModel](#), [SkyPointer](#), and [GenericObject](#).

```
static ILTServer* BaseClass::GetServerDE( ) [inline, static]
```

Returns:

A pointer to the engine's [ILTServer](#) instance.

Returns a pointer to the engine's [ILTServer](#) instance.

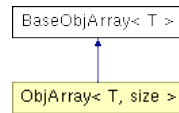
Used For: Object.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

BaseObjArray< T > Class Template Reference

Inheritance diagram for BaseObjArray< T >:



Public Member Functions

[BaseObjArray](#) ()
 bool [AddObject](#) (const T &obj)
 T [GetObject](#) ([uint32](#) index)
 void [Reset](#) ()
[uint32](#) [NumObjects](#) ()

Protected Attributes

T * [m_pArray](#)
[uint32](#) [m_MaxListSize](#)
[uint32](#) [m_nElements](#)

```

template<class T>
class BaseObjArray< T >
  
```

Constructor & Destructor Documentation

```

template<class T>
BaseObjArray< T >::BaseObjArray( ) [inline]
  
```

Member Function Documentation

```

template<class T>
bool BaseObjArray< T >::AddObject( const T & obj ) [inline]
  
```

```

template<class T>
T BaseObjArray< T >::GetObject( uint32 index ) [inline]
  
```

```

template<class T>
uint32 BaseObjArray< T >::NumObjects( ) [inline]
  
```

```

template<class T>
void BaseObjArray< T >::Reset( ) [inline]
  
```

Field Documentation

```

template<class T>
uint32 BaseObjArray< T >::m_MaxListSize [protected]
  
```

```

template<class T>
uint32 BaseObjArray< T >::m_nElements [protected]

    Number of valid objects in m_pList.
  
```

```

template<class T>
T* BaseObjArray< T >::m_pArray [protected]
  
```

The documentation for this class was generated from the following file:

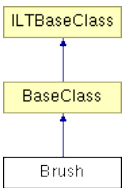
- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Brush Class Reference

Inheritance diagram for Brush:



Detailed Description

Brush class.

The documentation for this class was generated from the following file:

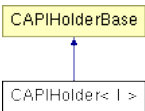
- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CAPiHolder< I > Class Template Reference

Inheritance diagram for CAPiHolder< I >:



Public Member Functions

[CAPiHolder](#) (const char *api_string_name, I *&holder_var, [int32](#) version)
[~CAPiHolder](#) ()
virtual void [APiRemoved](#) ()
virtual void [APiFound](#) ([IBase](#) *api)
virtual [IBase](#) * [Interface](#) ()

Data Fields

I *& [p](#)

template<class I>
class CAPiHolder< I >

Constructor & Destructor Documentation

```
template<class I>
CAPiHolder< I >::CAPiHolder( const char * api_string_name,
                             I *&          holder_var,
                             int32         version
                             )
```

```
template<class I>
CAPIHolder< I >::~~CAPIHolder( )
```

Member Function Documentation

```
template<class I>
void CAPIHolder< I >::APIFound( IBase * api ) [virtual]
    Implements CAPIHolderBase.
```

```
template<class I>
void CAPIHolder< I >::APIRemoved( ) [virtual]
    Implements CAPIHolderBase.
```

```
template<class I>
IBase * CAPIHolder< I >::Interface( ) [virtual]
    Implements CAPIHolderBase.
```

Field Documentation

```
template<class I>
I*& CAPIHolder< I >::p
```

The documentation for this class was generated from the following file:

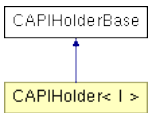
- [Itmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CAPIHolderBase Class Reference

Inheritance diagram for CAPIHolderBase:



Public Member Functions

```
CAPIHolderBase (const char *api_string_name, int32 version)
virtual ~CAPIHolderBase ()
virtual void APIRemoved ()=0
virtual void APIFound (IBase *api)=0
virtual IBase * Interface ()=0
__inline const char * InterfaceName ()
__inline int32 Version ()
```

Protected Attributes

```
const char * api\_name
int32 version
```

Constructor & Destructor Documentation

```
CAPIHolderBase::CAPIHolderBase( const char * api\_string\_name,
int32 version
)
```

virtual CAPIHolderBase::~CAPIHolderBase() [virtual]

Member Function Documentation

virtual void CAPIHolderBase::APIFound([IBase](#) * *api*) [pure virtual]
Implemented in [CAPIHolder<I>](#).

virtual void CAPIHolderBase::APIRemoved() [pure virtual]
Implemented in [CAPIHolder<I>](#).

virtual [IBase](#)* CAPIHolderBase::Interface() [pure virtual]
Implemented in [CAPIHolder<I>](#).

const char * CAPIHolderBase::InterfaceName()

[int32](#) CAPIHolderBase::Version()

Field Documentation

const char* [CAPIHolderBase::api_name](#) [protected]

[int32](#) [CAPIHolderBase::version](#) [protected]

The documentation for this class was generated from the following file:

- [Itmodule.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CAPIInstanceDefines< I > Class Template Reference

Public Member Functions

[CAPIInstanceDefines](#) (I *[implementation](#), const char *[implemented_class_name](#), [int32](#) interface_version)
[~CAPIInstanceDefines](#) ()

Data Fields

I * [implementation](#)
const char * [implemented_class_name](#)

template<class I>
class CAPIInstanceDefines< I >

Constructor & Destructor Documentation

template<class I>
[CAPIInstanceDefines](#)< I >::CAPIInstanceDefines(I * *implementation*,
const char * *implemented_class_name*,
[int32](#) *interface_version*
)

```
template<class I>
C-APIInstanceDefines< I >::~~C-APIInstanceDefines( )
```

Field Documentation

```
template<class I>
I* C-APIInstanceDefines< I >::implementation
```

```
template<class I>
const char* C-APIInstanceDefines< I >::implemented_class_name
```

The documentation for this class was generated from the following file:

- [ltmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CCallStackTracker Class Reference

Public Member Functions

```
CCallStackTracker(SFunctionInfo &function_info)
~CCallStackTracker()
void Progress()
```

Data Fields

[SFunctionInfo](#) & [info](#)

Constructor & Destructor Documentation

```
CCallStackTracker::CCallStackTracker(SFunctionInfo &function_info) [inline]
```

```
CCallStackTracker::~CCallStackTracker() [inline]
```

Member Function Documentation

```
void CCallStackTracker::Progress() [inline]
```

Field Documentation

[SFunctionInfo](#)& [CCallStackTracker::info](#)

The documentation for this class was generated from the following file:

- [lterror.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CChooserRegister Class Reference

Public Member Functions

[CChooserRegister](#) ([CInterfaceChooser](#) &[chooser_var](#), void(*hook)())
[~CChooserRegister](#) ()

Protected Attributes

[CInterfaceChooser](#) & [chooser_var](#)

Constructor & Destructor Documentation

CChooserRegister::CChooserRegister([CInterfaceChooser](#) & *chooser_var*,
void(*)() *hook*
) [inline]

CChooserRegister::~CChooserRegister() [inline]

Field Documentation

[CInterfaceChooser](#) & [CChooserRegister::chooser_var](#) [protected]

The documentation for this class was generated from the following file:

- [ltmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CConsolePrintData Class Reference

Data Fields

[LTRGB](#) [m_Color](#)
int [m_nFilterLevel](#)
const char * [m_pMessage](#)

Field Documentation

[LTRGB](#) [CConsolePrintData::m_Color](#)

int [CConsolePrintData::m_nFilterLevel](#)

const char* [CConsolePrintData::m_pMessage](#)

The documentation for this class was generated from the following file:

- [iclientshell.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CFontInfo Class Reference

Public Types

```
enum {  
    kStyle\_Bold = 0x01,  
    kStyle\_Italic = 0x02  
}  
enum { knMaxTypefaceLen = 32 }
```

Public Member Functions

```
CFontInfo ()  
CFontInfo (const wchar_t *pszTypeface, uint32 nHeight, uint8 lfCharSet=1, uint32 nStyle=0)  
bool IsStyleSet (uint32 nStyle) const  
void SetStyle (uint32 nStyle)  
void ClearStyle (uint32 nStyle)
```

Data Fields

```
wchar_t m\_szTypeface [knMaxTypefaceLen]  
uint32 m\_nHeight  
uint32 m\_nStyle  
uint8 m\_lfCharSet
```

Member Enumeration Documentation

anonymous enum

Enumeration values:

[kStyle_Bold](#)
[kStyle_Italic](#)

anonymous enum

Enumeration values:

[knMaxTypefaceLen](#)

Constructor & Destructor Documentation

[CFontInfo::CFontInfo](#)() [[inline](#)]

```
CFontInfo::CFontInfo( const wchar_t * pszTypeface,  
                     uint32 nHeight,  
                     uint8 lfCharSet = 1,  
                     uint32 nStyle = 0  
) [inline]
```

Member Function Documentation

void [CFontInfo::ClearStyle](#)([uint32](#) *nStyle*) [[inline](#)]

bool [CFontInfo::IsStyleSet](#)([uint32](#) *nStyle*) const [[inline](#)]

void [CFontInfo::SetStyle](#)([uint32](#) *nStyle*) [[inline](#)]

Field Documentation

[uint8 CFontInfo::m_lfCharSet](#)

[uint32 CFontInfo::m_nHeight](#)

[uint32 CFontInfo::m_nStyle](#)

wchar_t [CFontInfo::m_szTypeface](#)[knMaxTypefaceLen]

The documentation for this class was generated from the following file:

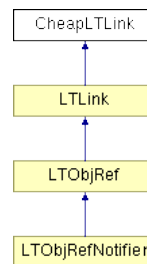
- [ilttexturestring.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CheapLTLINK Class Reference

Inheritance diagram for CheapLTLINK:



Public Member Functions

```

void Init ()
void Term ()
void Remove ()
void TieOff ()
void AddAfter (CheapLTLINK *pLink)
void AddBefore (CheapLTLINK *pLink)
LTLINK * operator\[\] (unsigned long i)
LTLINK * AsDLink ()
LTLINK * AsLTLINK ()
bool IsTiedOff ()
  
```

Data Fields

```

LTLINK * m\_pPrev
LTLINK * m\_pNext
  
```

Detailed Description

Use [LTLINK](#) whenever you need **m_pData**, but you can use this if you have a list head.

Member Function Documentation

```
void CheapLTLINK::AddAfter( CheapLTLINK * pLink ) [inline]
```

```
void CheapLTLINK::AddBefore( CheapLTLINK * pLink ) [inline]
```

[LTLINK](#)* CheapLTLink::AsDLink() [inline]

[LTLINK](#)* CheapLTLink::AsLTLink() [inline]

void CheapLTLink::Init() [inline]

bool CheapLTLink::IsTiedOff() [inline]

[LTLINK](#) * CheapLTLink::operator[](unsigned long *i*) [inline]
Make it easy to traverse the list in either direction.

void CheapLTLink::Remove() [inline]

void CheapLTLink::Term() [inline]

void CheapLTLink::TieOff() [inline]

Field Documentation

[LTLINK](#) * [CheapLTLink::m_pNext](#)

[LTLINK](#)* [CheapLTLink::m_pPrev](#)

The documentation for this class was generated from the following file:

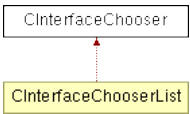
- [ltlink.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CInterfaceChooser Class Reference

Inheritance diagram for CInterfaceChooser:



Public Member Functions

void [Init](#) (const char *[name](#), [int32](#) [version](#))
virtual void [Add](#) ([IBase](#) *api, [CInterfaceNameMgr](#) *mgr)=0
virtual void [Remove](#) ([IBase](#) *api, [CInterfaceNameMgr](#) *mgr)=0
virtual [EChooserRun](#) [Run](#) ([CInterfaceNameMgr](#) *mgr)=0
const char * [InterfaceName](#) ()
[int32](#) [InterfaceVersion](#) ()

Protected Attributes

const char * [name](#)
[int32](#) [version](#)

Member Function Documentation

virtual void CInterfaceChooser::Add([IBase](#) * *api*,
 [CInterfaceNameMgr](#) * *mgr*
) [pure virtual]

Implemented in [CInterfaceChooserList](#).

void CInterfaceChooser::Init(const char * *name*,
 [int32](#) *version*
)

const char * CInterfaceChooser::InterfaceName() [inline]

[int32](#) CInterfaceChooser::InterfaceVersion() [inline]

virtual void CInterfaceChooser::Remove([IBase](#) * *api*,
 [CInterfaceNameMgr](#) * *mgr*
) [pure virtual]

Implemented in [CInterfaceChooserList](#).

virtual [EChooserRun](#) CInterfaceChooser::Run([CInterfaceNameMgr](#) * *mgr*) [pure virtual]

Implemented in [CInterfaceChooserList](#).

Field Documentation

const char* [CInterfaceChooser::name](#) [protected]

[int32](#) [CInterfaceChooser::version](#) [protected]

The documentation for this class was generated from the following file:

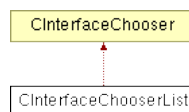
- [Itmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CInterfaceChooserList Class Reference

Inheritance diagram for CInterfaceChooserList:



Public Member Functions

void [Init](#) (const char *[name](#), [int32](#) [version](#), [uint32](#) [num_choices](#), const char **[choices](#), const char *[def_name](#))
[EChooserRun](#) [SetImplementation](#) ([uint32](#) [index](#))
[EChooserRun](#) [SetImplementation](#) (const char *[name](#))
virtual void [Add](#) ([IBase](#) **api*, [CInterfaceNameMgr](#) **mgr*)
virtual void [Remove](#) ([IBase](#) **api*, [CInterfaceNameMgr](#) **mgr*)
virtual [EChooserRun](#) [Run](#) ([CInterfaceNameMgr](#) **mgr*)

Protected Attributes

const char ** [choices](#)
[uint32](#) [num_choices](#)

char [def_name](#) [64]

Member Function Documentation

virtual void CInterfaceChooserList::Add([IBase](#) * *api*,
[CInterfaceNameMgr](#) * *mgr*
) [virtual]

Implements [CInterfaceChooser](#).

void CInterfaceChooserList::Init(const char * *name*,
[int32](#) *version*,
[uint32](#) *num_choices*,
const char ** *choices*,
const char * *def_name*
)

virtual void CInterfaceChooserList::Remove([IBase](#) * *api*,
[CInterfaceNameMgr](#) * *mgr*
) [virtual]

Implements [CInterfaceChooser](#).

virtual [EChooserRun](#) CInterfaceChooserList::Run([CInterfaceNameMgr](#) * *mgr*) [virtual]
Implements [CInterfaceChooser](#).

[EChooserRun](#) CInterfaceChooserList::SetImplementation(const char * *name*)

[EChooserRun](#) CInterfaceChooserList::SetImplementation([uint32](#) *index*)

Field Documentation

const char** [CInterfaceChooserList::choices](#) [protected]

char [CInterfaceChooserList::def_name](#)[64] [protected]

[uint32](#) [CInterfaceChooserList::num_choices](#) [protected]

The documentation for this class was generated from the following file:

- [Itmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CInterfaceDatabase Class Reference

Public Member Functions

[CInterfaceDatabase](#) ()
[~CInterfaceDatabase](#) ()
__inline [CPointerTracked](#)<
[CInterfaceDatabase](#) > * & [PointerTrackedList](#) ()
virtual void [TransferFrom](#) ([CInterfaceDatabase](#) *other)
virtual void [Destroy](#) ()

Static Public Member Functions

```

static void AddAPI (IBase *api, const char *implemented_class_name, int32 interface_version)
static void RemoveAPI (IBase *api, const char *implemented_class_name)
static void AddHolder (CAPIHolderBase *api_holder)
static void RemoveHolder (CAPIHolderBase *api_holder)
static void AddChooser (CInterfaceChooser *chooser)
static void RemoveChooser (CInterfaceChooser *chooser)
static EChooserRun RunChooser (CInterfaceChooser *chooser)
static void DatabaseItemCountInc ()
static void DatabaseItemCountDec ()

```

Protected Attributes

[CPointerTrackedTarget](#)< [CInterfaceDatabase](#) > [_CInterfaceDatabaseour_tracked_pointers](#)

Private Member Functions

```

void VersionError (const char *interface_name)
virtual void Add (IBase *api, const char *implementation_class_name, int32 interface_version)
virtual void Remove (IBase *api, const char *implementation_class_name)
virtual void Add (CAPIHolderBase *holder)
virtual void Remove (CAPIHolderBase *holder)
virtual void Add (CInterfaceChooser *chooser)
virtual void Remove (CInterfaceChooser *chooser)
virtual EChooserRun Run (CInterfaceChooser *chooser)
virtual bool IsEmpty ()

```

Private Attributes

```

database_array< CInterfaceNameMgr,
const char * > * interfaces
bool version\_check\_failed
char * version\_check\_interface\_name

```

Constructor & Destructor Documentation

[CInterfaceDatabase](#)::[CInterfaceDatabase](#)()

[CInterfaceDatabase](#)::~[CInterfaceDatabase](#)()

Member Function Documentation

virtual void [CInterfaceDatabase](#)::[Add](#)([CInterfaceChooser](#) * *chooser*) [private, virtual]

virtual void [CInterfaceDatabase](#)::[Add](#)([CAPIHolderBase](#) * *holder*) [private, virtual]

```

virtual void CInterfaceDatabase::Add( IBase *    api,
const char * implementation_class_name,
int32      interface_version
) [private, virtual]

```

```

static void CInterfaceDatabase::AddAPI( IBase *    api,
const char * implemented_class_name,
int32      interface_version
) [static]

```

static void [CInterfaceDatabase](#)::[AddChooser](#)([CInterfaceChooser](#) * *chooser*) [static]

static void [CInterfaceDatabase](#)::[AddHolder](#)([CAPIHolderBase](#) * *api_holder*) [static]

static void [CInterfaceDatabase](#)::[DatabaseItemCountDec](#)() [static]

static void [CInterfaceDatabase](#)::[DatabaseItemCountInc](#)() [static]

```
virtual void CInterfaceDatabase::Destroy( ) [virtual]
```

```
virtual bool CInterfaceDatabase::IsEmpty( ) [private, virtual]
```

```
__inline CPointerTracked<CInterfaceDatabase>* & CInterfaceDatabase::PointerTrackedList( ) [inline]
```

```
virtual void CInterfaceDatabase::Remove( CInterfaceChooser * chooser ) [private, virtual]
```

```
virtual void CInterfaceDatabase::Remove( CAPIHolderBase * holder ) [private, virtual]
```

```
virtual void CInterfaceDatabase::Remove( IBase * api,  
const char * implementation class name  
 ) [private, virtual]
```

```
static void CInterfaceDatabase::RemoveAPI( IBase * api,  
const char * implemented class name  
 ) [static]
```

```
static void CInterfaceDatabase::RemoveChooser( CInterfaceChooser * chooser ) [static]
```

```
static void CInterfaceDatabase::RemoveHolder( CAPIHolderBase * api_holder ) [static]
```

```
virtual EChooserRun CInterfaceDatabase::Run( CInterfaceChooser * chooser ) [private, virtual]
```

```
static EChooserRun CInterfaceDatabase::RunChooser( CInterfaceChooser * chooser ) [static]
```

```
virtual void CInterfaceDatabase::TransferFrom( CInterfaceDatabase * other ) [virtual]
```

```
void CInterfaceDatabase::VersionError( const char * interface_name ) [private]
```

Field Documentation

```
CPointerTrackedTarget<CInterfaceDatabase> CInterfaceDatabase::CInterfaceDatabaseour\_tracked\_pointers [protected]
```

```
database_array<CInterfaceNameMgr, const char *>* CInterfaceDatabase::interfaces [private]
```

```
bool CInterfaceDatabase::version\_check\_failed [private]
```

```
char* CInterfaceDatabase::version\_check\_interface\_name [private]
```

The documentation for this class was generated from the following file:

- [Itmodule.h](#)

API Reference

CInterfaceNameMgr Class Reference

Public Member Functions

```

        CInterfaceNameMgr (const char *name, int32 version)
virtual ~CInterfaceNameMgr ()
void Add (IBase *api)
void Add (CAPIHolderBase *holder)
void Add (CInterfaceChooser *chooser)
void Remove (IBase *api)
void Remove (CAPIHolderBase *holder)
void Remove (CInterfaceChooser *chooser)
EChooserRun Run (CInterfaceChooser *chooser)
const char * InterfaceName ()
int32 InterfaceVersion ()
bool IsEmpty ()
void TransferFrom (CInterfaceNameMgr *other)
bool UseImplementation (const char *name)
uint32 NumImplementations ()
const char * ImplementationName (uint32 index)

```

Protected Member Functions

```

void ConnectHolders (IBase *api)
void ConnectHolder (CAPIHolderBase *holder)
void DisconnectHolders ()

```

Protected Attributes

```

        char * name
        int32 version
        database_array< IBase, const
        char * > * interfaces
        database_list< CAPIHolderBase > * holders
        IBase * current\_interface
        database_list< CInterfaceChooser > * choosers

```

Constructor & Destructor Documentation

```

CInterfaceNameMgr::CInterfaceNameMgr( const char * name,
        int32 version
        )

```

```

virtual CInterfaceNameMgr::~CInterfaceNameMgr( ) [virtual]

```

Member Function Documentation

```

void CInterfaceNameMgr::Add( CInterfaceChooser * chooser )

```

```

void CInterfaceNameMgr::Add( CAPIHolderBase * holder )

```

```

void CInterfaceNameMgr::Add( IBase * api )

```

```

void CInterfaceNameMgr::ConnectHolder( CAPIHolderBase * holder ) [protected]

```

```

void CInterfaceNameMgr::ConnectHolders( IBase * api ) [protected]

```

```

void CInterfaceNameMgr::DisconnectHolders( ) [protected]

```

```
const char* CInterfaceNameMgr::ImplementationName( uint32 index )
```

```
const char * CInterfaceNameMgr::InterfaceName( ) [inline]
```

```
int32 CInterfaceNameMgr::InterfaceVersion( ) [inline]
```

```
bool CInterfaceNameMgr::IsEmpty( )
```

```
uint32 CInterfaceNameMgr::NumImplementations( )
```

```
void CInterfaceNameMgr::Remove( CInterfaceChooser * chooser )
```

```
void CInterfaceNameMgr::Remove( CAPIHolderBase * holder )
```

```
void CInterfaceNameMgr::Remove( IBase * api )
```

```
EChooserRun CInterfaceNameMgr::Run( CInterfaceChooser * chooser )
```

```
void CInterfaceNameMgr::TransferFrom( CInterfaceNameMgr * other )
```

```
bool CInterfaceNameMgr::UseImplementation( const char * name )
```

Field Documentation

```
database_list<CInterfaceChooser>* CInterfaceNameMgr::choosers [protected]
```

```
IBase* CInterfaceNameMgr::current\_interface [protected]
```

```
database_list<CAPIHolderBase>* CInterfaceNameMgr::holders [protected]
```

```
database_array<IBase, const char *>* CInterfaceNameMgr::interfaces [protected]
```

```
char* CInterfaceNameMgr::name [protected]
```

```
int32 CInterfaceNameMgr::version [protected]
```

The documentation for this class was generated from the following file:

- [Itmodule.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

ClassDef Struct Reference

Public Member Functions

CProjectClass * [DEditGetClassHook](#) ()
void [DEditSetClassHook](#) (CProjectClass *pClass)

Data Fields

char * [m_ClassName](#)
[ClassDef](#) * [m_ParentClass](#)
uint32 [m_ClassFlags](#)
[ConstructObjectFn](#) [m_ConstructFn](#)
[DestructObjectFn](#) [m_DestructFn](#)
[CreateObjectPluginFn](#) [m_PluginFn](#)
short [m_nProps](#)
[PropDef](#) * [m_Props](#)
long [m_ClassObjectSize](#)
void * [m_pInternal](#) [2]

Member Function Documentation

CProjectClass* ClassDef::DEditGetClassHook() [inline]

void ClassDef::DEditSetClassHook(CProjectClass * *pClass*) [inline]

Field Documentation

[uint32 ClassDef::m_ClassFlags](#)

A combination of the **CF_** flags above.

char* [ClassDef::m_ClassName](#)

long [ClassDef::m_ClassObjectSize](#)

How big an object of this class is (set automatically).

[ConstructObjectFn ClassDef::m_ConstructFn](#)

[DestructObjectFn ClassDef::m_DestructFn](#)

short [ClassDef::m_nProps](#)

[ClassDef](#)* [ClassDef::m_ParentClass](#)

void* [ClassDef::m_pInternal](#) [2]

Don't touch!

[CreateObjectPluginFn ClassDef::m_PluginFn](#)

[PropDef](#)* [ClassDef::m_Props](#)

The documentation for this struct was generated from the following file:

- [Itserverobj.h](#)
-

API Reference

ClassPropInfo Class Reference

Data Fields

char [m_PropName](#) [128]
short [m_PropType](#)

Detailed Description

Used by [ILTServer::GetClassProp](#).

Field Documentation

char [ClassPropInfo::m_PropName](#)[128]

short [ClassPropInfo::m_PropType](#)

The documentation for this class was generated from the following file:

- [iltserver.h](#)
-

API Reference

ClientInfo Struct Reference

Data Fields

char [m_sName](#) [64]
[uint32 m_ClientID](#)
float [m_Ping](#)

Field Documentation

[uint32 ClientInfo::m_ClientID](#)

float [ClientInfo::m_Ping](#)

char [ClientInfo::m_sName](#)[64]

The documentation for this struct was generated from the following file:

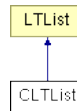
- [server_interface.h](#)
-

[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CLTList Class Reference

Inheritance diagram for CLTList:



Public Member Functions

```

    CLTList()
    LTLINK* Begin()
    LTLINK* End()
    void AddAfter(LTLINK &after, LTLINK &link, void *pObj)
    void AddAfter(LTLINK &after, LTLINK &link)
    void AddBefore(LTLINK &before, LTLINK &link, void *pObj)
    void AddBefore(LTLINK &before, LTLINK &link)
    void AddHead(LTLINK &link, void *pObj)
    void AddHead(LTLINK &link)
    void AddTail(LTLINK &link, void *pObj)
    void AddTail(LTLINK &link)
    uint32 GetCount() const
    void Remove(LTLINK &link)
    void Reset()
  
```

Data Structures

class [Iterator](#)

Constructor & Destructor Documentation

CLTList::CLTList() [inline]

Member Function Documentation

```

void CLTList::AddAfter(LTLINK &after,
                     LTLINK &link
                     ) [inline]
  
```

```

void CLTList::AddAfter(LTLINK &after,
                     LTLINK &link,
                     void * pObj
                     ) [inline]
  
```

```

void CLTList::AddBefore(LTLINK &before,
                      LTLINK &link
                      ) [inline]
  
```

```

void CLTList::AddBefore(LTLINK &before,
                      LTLINK &link,
                      void * pObj
                      ) [inline]
  
```

```

void CLTList::AddHead(LTLINK &link ) [inline]
  
```

```

void CLTList::AddHead(LTLINK &link,
                    void * pObj
                    ) [inline]
  
```

```
void CLTList::AddTail( LTLINK & link ) [inline]
```

```
void CLTList::AddTail( LTLINK & link,  
                      void *   pObj  
                      ) [inline]
```

```
LTLINK* CLTList::Begin( ) [inline]
```

```
LTLINK* CLTList::End( ) [inline]
```

```
uint32 CLTList::GetCount( ) const [inline]
```

```
void CLTList::Remove( LTLINK & link ) [inline]
```

```
void CLTList::Reset( ) [inline]
```

The documentation for this class was generated from the following file:

- [ltlink.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

[CLTList::Iterator](#)

CLTList::Iterator Class Reference

Public Member Functions

```
Iterator ()  
Iterator (LTLINK *pLink)  
Iterator (LTLINK &link)  
LTLINK * operator-> ()  
operator LTLINK * ()  
operator LTLINK & ()  
Iterator & operator++ ()  
Iterator operator++ (int)  
Iterator & operator-- ()  
Iterator operator-- (int)  
Iterator & operator= (LTLINK *pLink)  
bool operator== (LTLINK *pLink) const  
bool operator!= (LTLINK *pLink) const
```

Private Attributes

```
LTLINK * m_pCur
```

Constructor & Destructor Documentation

```
CLTList::Iterator::Iterator( ) [inline]
```

```
CLTList::Iterator::Iterator( LTLINK * pLink ) [inline]
```

CLTList::Iterator::Iterator([LTLink](#) & *link*) [inline]

Member Function Documentation

CLTList::Iterator::operator [LTLink](#) &() [inline]

CLTList::Iterator::operator [LTLink](#) *() [inline]

bool CLTList::Iterator::operator!=([LTLink](#) * *pLink*) const [inline]

[Iterator](#) CLTList::Iterator::operator++(int) [inline]

[Iterator](#)& CLTList::Iterator::operator++() [inline]

[Iterator](#) CLTList::Iterator::operator--(int) [inline]

[Iterator](#)& CLTList::Iterator::operator--() [inline]

[LTLink](#)* CLTList::Iterator::operator->() [inline]

[Iterator](#)& CLTList::Iterator::operator=([LTLink](#) * *pLink*) [inline]

bool CLTList::Iterator::operator==([LTLink](#) * *pLink*) const [inline]

Field Documentation

[LTLink](#)* [CLTList::Iterator::m_pCur](#) [private]

The documentation for this class was generated from the following file:

- [ltlink.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CLTReference< T > Class Template Reference

Public Member Functions

[CLTReference](#) ()
[CLTReference](#) (T *pPtr)
[~CLTReference](#) ()
[CLTReference](#) & [operator=](#) (const [CLTReference](#) &cOther)
[CLTReference](#) & [operator=](#) (T *pPtr)
 bool [operator=](#) (const T *pPtr) const
 bool [operator!=](#) (const T *pPtr) const
 T & [operator*](#) () const

```
T * operator-> () const
operator T \* () const
```

Private Member Functions

```
void \_assign (T *pPtr)
```

Private Attributes

```
T * m\_pPtr
```

```
template<class T>
class CLTReference< T >
```

Constructor & Destructor Documentation

```
template<class T>
CLTReference< T >::CLTReference( ) [inline]
```

```
template<class T>
CLTReference< T >::CLTReference( T * pPtr ) [inline]
```

```
template<class T>
CLTReference< T >::~~CLTReference( ) [inline]
```

Member Function Documentation

```
template<class T>
void CLTReference< T >::_assign( T * pPtr ) [inline, private]
```

```
template<class T>
T& CLTReference< T >::operator *( ) const [inline]
```

```
template<class T>
CLTReference< T >::operator T *( ) const [inline]
```

```
template<class T>
bool CLTReference< T >::operator!=( const T * pPtr ) const [inline]
```

```
template<class T>
T* CLTReference< T >::operator->( ) const [inline]
```

```
template<class T>
CLTReference& CLTReference< T >::operator=( T * pPtr ) [inline]
```

```
template<class T>
CLTReference& CLTReference< T >::operator=( const CLTReference< T > & cOther ) [inline]
```

```
template<class T>
bool CLTReference< T >::operator==( const T * pPtr ) const [inline]
```

Field Documentation

```
template<class T>
T* CLTReference< T >::m_pPtr [private]
```

The documentation for this class was generated from the following file:

- [iltrefcount.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CMemBlockAllocator Class Reference

Public Member Functions

[CMemBlockAllocator](#) ([uint8](#) *pDataBlock, [uint32](#) nBlockSize)
[uint32](#) [GetBlockSize](#) () const
[uint32](#) [GetAllocationOffset](#) () const
[uint8](#) * [Allocate](#) ([uint32](#) nSize)
template<typename T>
T * [AllocateObjects](#) ([uint32](#) nNumObjects)
template<typename T>
T * [AllocateObject](#) ()

Private Attributes

[uint8](#) * [m_pDataBlock](#)
[uint32](#) [m_nBlockSize](#)
[uint32](#) [m_nAllocOffset](#)

Constructor & Destructor Documentation

CMemBlockAllocator::CMemBlockAllocator([uint8](#) * *pDataBlock*,
 [uint32](#) *nBlockSize*
) [inline]

Member Function Documentation

[uint8](#)* CMemBlockAllocator::Allocate([uint32](#) *nSize*) [inline]

template<typename T>
T* CMemBlockAllocator::AllocateObject() [inline]

template<typename T>
T* CMemBlockAllocator::AllocateObjects([uint32](#) *nNumObjects*) [inline]

[uint32](#) CMemBlockAllocator::GetAllocationOffset() const [inline]

[uint32](#) CMemBlockAllocator::GetBlockSize() const [inline]

Field Documentation

[uint32](#) [CMemBlockAllocator::m_nAllocOffset](#) [private]

[uint32](#) [CMemBlockAllocator::m_nBlockSize](#) [private]

```
uint8* CMemBlockAllocator::m_pDataBlock [private]
```

The documentation for this class was generated from the following file:

- [memblockallocator.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CMemBlockArray< ArrayType, IntSizeType > Class Template Reference

Public Member Functions

```

    CMemBlockArray ()
    ~CMemBlockArray ()
    IntSizeType GetSize () const
    ArrayType * GetArray ()
    const ArrayType * GetArray () const
    void Free ()
    void Allocate (IntSizeType nNumItems, CMemBlockAllocator &Allocator)
    const ArrayType & operator[] (uint32 nIndex) const
    ArrayType & operator[] (uint32 nIndex)

```

Private Member Functions

```

    CMemBlockArray (const CMemBlockArray &)
    CMemBlockArray & operator= (const CMemBlockArray &)

```

Private Attributes

```

    ArrayType * m_pArray
    IntSizeType m_nSize

```

```

template<typename ArrayType, typename IntSizeType = uint32>
class CMemBlockArray< ArrayType, IntSizeType >

```

Constructor & Destructor Documentation

```

template<typename ArrayType, typename IntSizeType = uint32>
CMemBlockArray< ArrayType, IntSizeType >::CMemBlockArray( ) [inline]

```

```

template<typename ArrayType, typename IntSizeType = uint32>
CMemBlockArray< ArrayType, IntSizeType >::~~CMemBlockArray( ) [inline]

```

```

template<typename ArrayType, typename IntSizeType = uint32>
CMemBlockArray< ArrayType, IntSizeType >::CMemBlockArray( const CMemBlockArray< ArrayType, IntSizeType > & ) [private]

```

Member Function Documentation

```

template<typename ArrayType, typename IntSizeType = uint32>
void CMemBlockArray< ArrayType, IntSizeType >::Allocate( IntSizeType nNumItems,
    CMemBlockAllocator & Allocator
) [inline]

```

```

template<typename ArrayType, typename IntSizeType = uint32>
void CMemBlockArray< ArrayType, IntSizeType >::Free ( ) [inline]

```

```
template<typename ArrayType, typename IntSizeType = uint32>
const ArrayType* CMemBlockArray< ArrayType, IntSizeType >::GetArray( ) const [inline]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
ArrayType* CMemBlockArray< ArrayType, IntSizeType >::GetArray( ) [inline]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
IntSizeType CMemBlockArray< ArrayType, IntSizeType >::GetSize( ) const [inline]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
CMemBlockArray& CMemBlockArray< ArrayType, IntSizeType >::operator=( const CMemBlockArray< ArrayType, IntSizeType > & ) [private]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
ArrayType& CMemBlockArray< ArrayType, IntSizeType >::operator[]( uint32 nIndex ) [inline]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
const ArrayType& CMemBlockArray< ArrayType, IntSizeType >::operator[]( uint32 nIndex ) const [inline]
```

Field Documentation

```
template<typename ArrayType, typename IntSizeType = uint32>
IntSizeType CMemBlockArray< ArrayType, IntSizeType >::m_nSize [private]
```

```
template<typename ArrayType, typename IntSizeType = uint32>
ArrayType* CMemBlockArray< ArrayType, IntSizeType >::m_pArray [private]
```

The documentation for this class was generated from the following file:

- [memblockallocator.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CollisionInfo Struct Reference

Data Fields

[LTPlane](#) [m_Plane](#)
[HOBJECT](#) [m_hObject](#)
[HPOLY](#) [m_hPoly](#)
[LTVector](#) [m_vStopVel](#)

Detailed Description

Used to determine information in GetStandingOn or GetLastCollision.

Field Documentation

[HOBJECT](#) [CollisionInfo::m_hObject](#)

The object collided with.

[HPOLY CollisionInfo::m_hPoly](#)

If the m_hObject is the world, then m_hPoly contains a handle to the polygon the object collided with, otherwise it is equal to LTNUL.

[LTPlane CollisionInfo::m_Plane](#)

The blocking plane of whatever the object collided with. If both object's are non-solid, then this will have a normal vector of mag 0.

[LTVector CollisionInfo::m_vStopVel](#)

Stopping velocity. Engine will automatically apply this velocity to stop object from continuing to collide with blocker.

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CompRot Class Reference

Public Member Functions

bool [operator==](#) ([CompRot](#) &testRot)

Data Fields

char [m_Bytes](#) [6]

Detailed Description

Class to store compressed rotation. Includes an overridden equality operator.

See also:

CompressRotation()

UncompressRotation().

Member Function Documentation

bool [CompRot::operator==\(\[CompRot\]\(#\) & testRot \)](#) [inline]

Field Documentation

char [CompRot::m_Bytes](#)[6]

The documentation for this class was generated from the following file:

- [iltcommon.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CompVector Struct Reference

Data Fields

float [fA](#)
[uint32](#) [dwB](#)
[uint32](#) [dwC](#)
[uint8](#) [order](#)

Detailed Description

Class to store compressed vector.

See also:

[CompressRotation\(\)](#)

[UncompressRotation\(\)](#)

Note: Only the lower 16 bits of **dwB** and **dwC** are packed by engine networking code.

Field Documentation

[uint32](#) [CompVector::dwB](#)

[uint32](#) [CompVector::dwC](#)

float [CompVector::fA](#)

[uint8](#) [CompVector::order](#)

The documentation for this struct was generated from the following file:

- [iltcommon.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CompWorldPos Class Reference

Public Member Functions

bool [operator==](#) ([CompWorldPos](#) &testPos)

Static Public Member Functions

static [uint32](#) [DataSize](#) ()

Data Fields

[uint16](#) [m_Pos](#) [3]
char [m_Extra](#)

Detailed Description

Class to store compressed position vector.

Member Function Documentation

static [uint32](#) [CompWorldPos::DataSize](#)() [[inline](#), [static](#)]

```
bool CompWorldPos::operator==( CompWorldPos & testPos ) [inline]
```

Field Documentation

char [CompWorldPos::m_Extra](#)

[uint16 CompWorldPos::m_Pos](#)[3]

The documentation for this class was generated from the following file:

- [iltcommon.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ConParse Class Reference

Public Member Functions

[ConParse](#) ()
[ConParse](#) (const char *pBuffer)
void [Init](#) (const char *pBuffer)
bool [Parse](#) ()
bool [ParseFind](#) (char *pLookFor, bool bCaseSensitive, [uint32](#) minTokens=1)

Data Fields

char * [m_Args](#) [64]
int [m_nArgs](#)

Private Attributes

const char * [m_pCommandPos](#)
char [m_ArgBuffer](#) [64*80]

Detailed Description

The new console parsing thing.

Constructor & Destructor Documentation

[ConParse::ConParse](#)() [inline]

[ConParse::ConParse](#)(const char * *pBuffer*) [inline]

Member Function Documentation

void [ConParse::Init](#)(const char * *pBuffer*) [inline]
Sets it up to parse the specified buffer.

bool [ConParse::Parse](#)()

Parse the next set of tokens.

Returns:

true if it parsed anything.

```
bool ConParse::ParseFind( char * pLookFor,
                        bool  bCaseSensitive,
                        uint32 minTokens = 1
                      )
```

Parses until it finds tokens with pLookFor as the first one. skipCount says how many to skip. You can use this just like Parse like this:

```
while(ParseFind("AmbientLight", false)) { ... }
```

Field Documentation

char [ConParse::m_ArgBuffer](#)[64 *80] [private]

char* [ConParse::m_Args](#)[64]

The parsed arguments.

int [ConParse::m_nArgs](#)

const char* [ConParse::m_pCommandPos](#) [private]

The documentation for this class was generated from the following file:

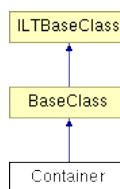
- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Container Class Reference

Inheritance diagram for Container:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

[uint32](#) [m_ContainerCode](#)

Detailed Description

Container.

Member Function Documentation

```
virtual uint32 Container::EngineMessageFn( uint32 messageID,
                                         void * pData,
                                         float lData
                                       ) [virtual]
```

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

[uint32 Container::m_ContainerCode](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

ContainerPhysics Struct Reference

Data Fields

[LTVector m_Acceleration](#)
[LTVector m_Velocity](#)
[uint32 m_Flags](#)
[HOBJECT m_hObject](#)

Detailed Description

Used for containers to apply physics to an object. You can change velocity, acceleration, and flags without changing the actual values of the object.

Field Documentation

[LTVector ContainerPhysics::m_Acceleration](#)

[uint32 ContainerPhysics::m_Flags](#)

[HOBJECT ContainerPhysics::m_hObject](#)

[LTVector ContainerPhysics::m_Velocity](#)

The documentation for this struct was generated from the following file:

- [Itserverobj.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CPointerTracked< C > Class Template Reference

Public Member Functions

```

__inline CPointerTracked()
__inline CPointerTracked(C *obj)
__inline ~CPointerTracked()

__inline CPointerTracked<
    C > & operator=(C *obj)
__inline CPointerTracked<
    C > & operator=(CPointerTracked< C > &other)
__inline C * operator->()
__inline operator C \*()
__inline bool operator==(C *obj)
__inline bool operator!=(C *obj)
__inline operator bool()
__inline bool IsNull()

```

Protected Member Functions

```

void Set(C *obj)
void Unset()

```

Protected Attributes

```

C * pointer
CPointerTracked< C > * prev
CPointerTracked< C > * next

```

```

template<class C>
class CPointerTracked< C >

```

Constructor & Destructor Documentation

```

template<class C>
__inline CPointerTracked< C >::CPointerTracked()

```

```

template<class C>
__inline CPointerTracked< C >::CPointerTracked(C * obj)

```

```

template<class C>
CPointerTracked< C >::~~CPointerTracked()

```

Member Function Documentation

```

template<class C>
__inline bool CPointerTracked< C >::IsNull()

```

```

template<class C>
__inline CPointerTracked< C >::operator bool()

```

```

template<class C>

```

```

__inline CPointerTracked< C >::operator C *( )

template<class C>
__inline bool CPointerTracked< C >::operator!=( C * obj )

template<class C>
__inline C* CPointerTracked< C >::operator->( )

template<class C>
__inline CPointerTracked<C>& CPointerTracked< C >::operator=( CPointerTracked< C > & other )

template<class C>
__inline CPointerTracked<C>& CPointerTracked< C >::operator=( C * obj )

template<class C>
__inline bool CPointerTracked< C >::operator==( C * obj )

template<class C>
void CPointerTracked< C >::Set( C * obj ) [protected]

template<class C>
void CPointerTracked< C >::Unset( ) [protected]

```

Field Documentation

```

template<class C>
CPointerTracked<C> * CPointerTracked< C >::next [protected]

template<class C>
C* CPointerTracked< C >::pointer [protected]

template<class C>
CPointerTracked<C>* CPointerTracked< C >::prev [protected]

```

The documentation for this class was generated from the following file:

- [Itmodule.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CPointerTrackedTarget< C > Class Template Reference

Public Member Functions

```

__inline CPointerTrackedTarget ()
__inline ~CPointerTrackedTarget ()
    operator CPointerTracked ()
void RedirectPointers (C *new_value)

```

Protected Attributes

[CPointerTracked](#)< C > * [tracked_pointers](#)

```
template<class C>
class CPointerTrackedTarget< C >
```

Constructor & Destructor Documentation

```
template<class C>
__inline CPointerTrackedTarget< C >::CPointerTrackedTarget( )
```

```
template<class C>
__inline CPointerTrackedTarget< C >::~CPointerTrackedTarget( )
```

Member Function Documentation

```
template<class C>
CPointerTrackedTarget< C >::operator CPointerTracked( ) [inline]
```

```
template<class C>
void CPointerTrackedTarget< C >::RedirectPointers( C * new_value )
```

Field Documentation

```
template<class C>
CPointerTracked<C>* CPointerTrackedTarget< C >::tracked_pointers [protected]
```

The documentation for this class was generated from the following file:

- [Itmodule.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CRenderStyle Class Reference

Public Member Functions

```

CRenderStyle ()
virtual ~CRenderStyle ()
virtual bool SetLightingMaterial (LightingMaterial &LightMaterial)=0
virtual bool GetLightingMaterial (LightingMaterial *pLightMaterial)=0
virtual bool AddRenderPass (RenderPassOp &Renderpass)=0
virtual bool RemoveRenderPass (uint32 iPass)=0
virtual bool SetRenderPass (uint32 iPass, RenderPassOp &RenderPass)=0
virtual bool GetRenderPass (uint32 iPass, RenderPassOp *pRenderPass)=0
virtual uint32 GetRenderPassCount ()=0
virtual bool SetDirect3D\_Options (RSD3DOptions &Options)
virtual bool GetDirect3D\_Options (RSD3DOptions *pOptions)
virtual bool SetRenderPass\_D3DOptions (uint32 iPass, RSD3DRenderPass *pD3DRenderPass)
virtual bool GetRenderPass\_D3DOptions (uint32 iPass, RSD3DRenderPass *pD3DRenderPass)
virtual bool Load\_LTBDData (ILTStream *pFileStream)=0
const char * GetFilename () const
virtual bool Compile ()=0
virtual void SetDefaults ()=0
virtual bool CopyRenderStyle (CRenderStyle *pSrcRenderStyle)=0
virtual bool IsSupportedOnDevice ()=0

```

```
virtual uint32 GetRefCount ()
virtual uint32 IncRefCount ()
virtual uint32 DecRefCount ()
    void SetFilename (const char *new_name)
```

Protected Attributes

[uint32 m_iRefCount](#)

Private Attributes

char * [m_pFilename](#)

Detailed Description

Manages settings for a render style.

Constructor & Destructor Documentation

```
CRenderStyle::CRenderStyle( ) [inline]
```

```
virtual CRenderStyle::~CRenderStyle( ) [inline, virtual]
```

Member Function Documentation

```
virtual bool CRenderStyle::AddRenderPass( RenderPassOp & Renderpass ) [pure virtual]
```

Parameters:

Renderpass The address of the [RenderPassOp](#) structure to add to this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Adds a [RenderPassOp](#) structure to this render style. A **CRenderStyle** instance may have any number of [RenderPassOp](#) structures applied to it. The **CRenderStyle** instance keeps a 0-based list of these structures.

See also:

[CRenderStyle::RemoveRenderPass\(\)](#)

```
virtual bool CRenderStyle::Compile( ) [pure virtual]
```

Returns:

A Boolean value indicating whether or not the function is successful.

```
virtual bool CRenderStyle::CopyRenderStyle( CRenderStyle * pSrcRenderStyle ) [pure virtual]
```

Parameters:

pSrcRenderStyle

Returns:

A Boolean value indicating whether or not the function is successful.

```
virtual uint32 CRenderStyle::DecRefCount( ) [inline, virtual]
```

```
virtual bool CRenderStyle::GetDirect3D_Options( RSD3DOptions * pOptions ) [inline, virtual]
```

Parameters:

pOptions [Return parameter] A pointer to the [RSD3DOptions](#) structure set for this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Retrieves the Direct3D options set for this render style.

See also:

[CRenderStyle::SetDirect3D_Options\(\)](#)

```
const char* CRenderStyle::GetFilename( ) const [inline]
```


virtual bool CRenderStyle::GetLightingMaterial([LightingMaterial](#) * *pLightMaterial*) [pure virtual]

Parameters:

pLightMaterial [Return parameter] A pointer to the structure set for this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Retrieves the lighting material set for this render style.

virtual [uint32](#) CRenderStyle::GetRefCount() [inline, virtual]

Returns:

ref count.

Get the reference count.

virtual bool CRenderStyle::GetRenderPass([uint32](#) *iPass*,
[RenderPassOp](#) * *pRenderPass*
) [pure virtual]

Parameters:

iPass An index into the **CRenderStyle** instance's list of [RenderPassOp](#) structures.

pRenderPass [Return parameter] A pointer to the [RenderPassOp](#) structure.

Retrieves one of the [RenderPassOp](#) structures set for this render style.

See also:

[CRenderStyle::SetRenderPass\(\)](#)

virtual bool CRenderStyle::GetRenderPass_D3DOptions([uint32](#) *iPass*,
[RSD3DRenderPass](#) * *pD3DRenderPass*
) [inline, virtual]

Parameters:

iPass An index into the **CRenderStyle** instance's list of [RSD3DRenderPassOp](#) structures.

pD3DRenderPass [Return parameter] A pointer to one of the [RSD3DRenderPass](#) structures applied to this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Retrieves one of the [RSD3DRenderPass](#) structures applied to this render style.

See also:

[CRenderStyle::GetRenderPass_D3DOptions\(\)](#)

virtual [uint32](#) CRenderStyle::GetRenderPassCount() [pure virtual]

Returns:

A uint32 value identifying how many [RenderPassOp](#) structures apply to this render style.

Returns the count of [RenderPassOp](#) structures applied to this render style.

virtual [uint32](#) CRenderStyle::IncRefCount() [inline, virtual]

virtual bool CRenderStyle::IsSupportedOnDevice() [pure virtual]

Check for support on the current platform.

virtual bool CRenderStyle::Load_LTBDData([ILTStream](#) * *pFileStream*) [pure virtual]

Parameters:

pFileStream The LTB render style file.

Returns:

A Boolean value indicating whether or not the function is successful.

Loads a compiled LTB render style file.

virtual bool CRenderStyle::RemoveRenderPass([uint32](#) *iPass*) [pure virtual]

Parameters:

iPass An index into the **CRenderStyle** instance's list of [RenderPassOp](#) structures.

Returns:

A Boolean value indicating whether or not the function is successful.

Removes a [RenderPassOp](#) structure from this render style.

See also:

[CRenderStyle::AddRenderPass\(\)](#)

virtual void CRenderStyle::SetDefaults() [pure virtual]

Set to default.

virtual bool CRenderStyle::SetDirect3D_Options([RSD3DOptions](#) & *Options*) [inline, virtual]

Parameters:

Options The address of the [RSD3DOptions](#) structure to apply to this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Sets Direct3D options for this render style.

See also:

[CRenderStyle::GetDirect3D_Options\(\)](#)

void CRenderStyle::SetFilename(const char * *new_name*) [inline]

virtual bool CRenderStyle::SetLightingMaterial([LightingMaterial](#) & *LightMaterial*) [pure virtual]

Parameters:

LightMaterial The address of the [LightingMaterial](#) structure for this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Sets the lighting material for this render style.

virtual bool CRenderStyle::SetRenderPass([uint32](#) *iPass*,
[RenderPassOp](#) & *RenderPass*
) [pure virtual]

Parameters:

iPass An index into the CRenderStyle instance's list of [RenderPassOp](#) structures.

RenderPass The address of the [RenderPassOp](#) structure.

Returns:

A Boolean value indicating whether or not the function is successful.

Sets a structure for this render style.

See also:

[CRenderStyle::GetRenderPass\(\)](#)

virtual bool CRenderStyle::SetRenderPass_D3DOptions([uint32](#) *iPass*,
[RSD3DRenderPass](#) * *pD3DRenderPass*
) [inline, virtual]

Parameters:

iPass An index into the CRenderStyle instance's list of [RSD3DRenderPassOp](#) structures.

pD3DRenderPass The address of the [RSD3DRenderPass](#) structure to apply to this render style.

Returns:

A Boolean value indicating whether or not the function is successful.

Sets a [RSD3DRenderPass](#) structure to this render style.

See also:

[CRenderStyle::GetRenderPass_D3DOptions\(\)](#)

Field Documentation

[uint32](#) CRenderStyle::m_iRefCnt [protected]

char* CRenderStyle::m_pFilename [private]

The documentation for this class was generated from the following file:

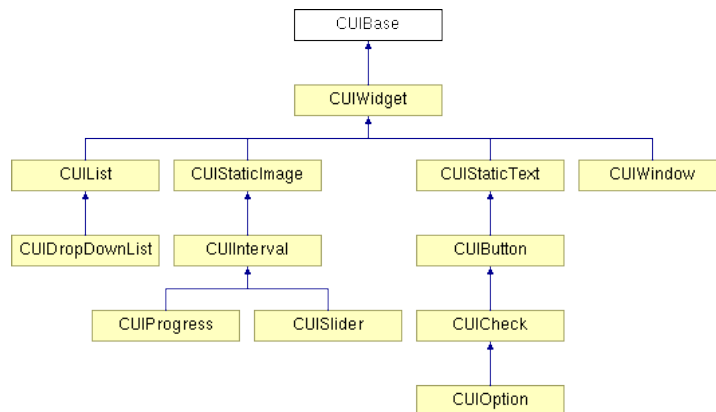
- [ltrenderstyle.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIBase Class Reference

Inheritance diagram for CUIBase:



Public Member Functions

```

virtual ~CUIBase ()
virtual CUI_WIDGETTYPE GetType ()
virtual const char * GetClassName ()
virtual CUIGUID GetGUID ()
virtual CUIBase * GetParent ()
virtual CUIBase * GetFirstChild ()
virtual CUIBase * GetNextChild ()
virtual CUI_RESULTTYPE GetPosition (float *pX, float *pY, CUIBase *pRelativeTo)
virtual CUI_RESULTTYPE GetSize (float *pW, float *pH)
virtual CUI_RESULTTYPE GetRect (CUIRECT *pRect, CUIBase *pRelativeTo)
virtual HTEXTURE GetTexture (CUI_ELEMENTTYPE elm)
virtual CUI_RESULTTYPE SetPosition (float x, float y, CUIBase *pRelativeTo, bool bMoveChildren)
virtual CUI_RESULTTYPE SetSize (float w, float h)
virtual CUI_RESULTTYPE SetRect (CUIRECT *pRect, CUIBase *pRelativeTo, bool bMoveChildren)
virtual CUI_RESULTTYPE SetUserCallback (CUI_CALLBACKTYPE pUserCallback, void *pUserCallbackObject, void *pUserCallbackData)
virtual CUI_CALLBACKTYPE GetUserCallback (void **ppUserCallbackObject, void **ppUserCallbackData)
virtual void * GetUserCallbackObject ()
virtual void * GetUserCallbackData ()
virtual CUI_RESULTTYPE SetParent (CUIBase *pParent)
virtual CUI_RESULTTYPE Show (bool bShowChildren)
virtual CUI_RESULTTYPE Hide (bool bHideChildren)
virtual CUI_RESULTTYPE SetState (uint32 flags)
virtual CUI_RESULTTYPE UnsetState (uint32 flags)
virtual uint32 GetState ()
virtual CUI_RESULTTYPE SetColor (CUI_ELEMENTTYPE elm, uint32 argb)=0
virtual CUI_RESULTTYPE SetColors (CUI_ELEMENTTYPE elm, uint32 argb0, uint32 argb1, uint32 argb2, uint32 argb3)=0
virtual CUI_RESULTTYPE SetTexture (CUI_ELEMENTTYPE elm, HTEXTURE hTex, bool bTile)=0
virtual CUI_RESULTTYPE SetGutter (int16 left, int16 right, int16 top, int16 bottom)
virtual CUI_RESULTTYPE GetGutter (int16 *pLeft, int16 *pRight, int16 *pTop, int16 *pBottom)
virtual CUI_RESULTTYPE BringToFront ()
virtual CUI_RESULTTYPE SendToBack ()
virtual void Render ()
virtual void Destroy ()
virtual CUI_RESULTTYPE HandleMessage (CUIMessage *pMessage)
virtual CUI_RESULTTYPE SendMessageToChildren (CUIMessage *pMessage)
CUIBase_Impl * GetImpl ()
  
```

Protected Attributes

CUIBase_Impl * [m_pImpl](#)

Detailed Description

CUIBase class.

See also:interface [ILTWidgetManager](#)

Used for: Text and UI.

Constructor & Destructor Documentation

virtual CUIBase::~CUIBase() [virtual]

Virtual Destructor.

Member Function Documentation

virtual [CUI_RESULTTYPE](#) CUIBase::BringToFront() [virtual]**Returns:**

CUI_RESULTTYPE

Brings a widget to the front (top) of its local display list. A parent widget will never draw above its children, but the child widgets can be re-ordered.

Used for: Text and UI.

virtual void CUIBase::Destroy() [virtual]

Returns:

CUI_RESULTTYPE

Causes a widget be added to the destroy queue. When the next [ILTWidgetManager::Update\(\)](#) is called, the widget (and all its children) will be destroyed.

Used for: Text and UI.

virtual const char* CUIBase::GetClassName() [virtual]

Returns:

Widget's name as a const char*

This function is used to query the class name of a widget.

Used for: Text and UI.

virtual [CUIBase*](#) CUIBase::GetFirstChild() [virtual]**Returns:**

CUIBase* address of first child widget

This function returns the address of the first child widget. If no child exists, then this function will return NULL. To continue iterating through the list of child widgets, call [CUIBase::GetNextChild\(\)](#) until a NULL pointer is returned.

NOTE: calling any function which changes the z-order of widgets or changes their parent while iterating through a list of children will cause unpredictable results! (i.e., [CUIBase::BringToFront\(\)](#), [CUIBase::SendToBack\(\)](#), or [CUIBase::SetParent\(\)](#))

Used for: Text and UI.

virtual [CUIGUID](#) CUIBase::GetGUID() [virtual]**Returns:**

Widget's CUIGUID

This function is used to query the CUIGUID of a widget.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::GetGutter( int16 * pLeft,
                                           int16 * pRight,
                                           int16 * pTop,
                                           int16 * pBottom
                                           ) [virtual]
```

Parameters:

pLeft address of variable to receive gutter distance from the left side.
pRight address of variable to receive gutter distance from the right side.
pTop address of variable to receive gutter distance from the top.
pBottom address of variable to receive gutter distance from the bottom.

Returns:

CUI_RESULTTYPE.

Many widget types have sub-elements that can be spaced away from the edges by using a gutter. The gutter specifies how far (in pixels) from the edges any interior parts of the UI are drawn.

Used for: Text and UI.

```
CUIBase_Impl* CUIBase::GetImpl( ) [inline]
```

```
virtual CUIBase* CUIBase::GetNextChild( ) [virtual]
```

Returns:

[CUIBase](#)* address of next child widget

This function returns the address of the next child widget each time it is called. If no child exists, then this function will return NULL. To continue iterating through the list of child widgets, call this function again until a NULL pointer is returned. To retrieve the first child widget, use [CUIBase::GetFirstChild\(\)](#).

NOTE: calling any function which changes the z-order of widgets or changes their parent while iterating through a list of children will cause unpredictable results! (i.e., [CUIBase::BringToFront\(\)](#), [CUIBase::SendToBack\(\)](#), or [CUIBase::SetParent\(\)](#))

Used for: Text and UI.

```
virtual CUIBase* CUIBase::GetParent( ) [virtual]
```

Returns:

[CUIBase](#)* address of parent widget

This function returns the address of a widget's parent. If no parent exists, then this function will return NULL.

See also:

[CUIBase::SetParent\(CUIBase*\)](#)

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::GetPosition( float *   pX,
                                           float *   pY,
                                           CUIBase* pRelativeTo
                                           ) [virtual]
```

Parameters:

pX (out) address of variable to receive the x-coordinate of the widget.
pY (out) address of variable to receive the y-coordinate of the widget.
pRelativeTo (in) specifies the widget that the retrieved (x,y) coords. are relative to.

Returns:

CUIR_OK if successful.

This function retrieves the (x,y) coordinates of the upper left corner of a widget (in pixels), relative to the widget specified as *pRelativeTo*. If *pRelativeTo* is NULL, (x,y) will be relative to the upper left corner of the screen (0,0).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::GetRect( CUIRECT* pRect,
                                         CUIBase* pRelativeTo
                                         ) [virtual]
```

Parameters:

pRect (out) address of [CUIRECT](#) struct to receive the widget's bounding box.
pRelativeTo (in) specifies the widget that the retrieved (x,y) coords. are relative to.

Returns:

CUIR_OK if successful.

This function retrieves the current bounding rectangle (in pixels) of a widget, relative to the widget specified as *pRelativeTo*. if *pRelativeTo* is NULL, (x,y) will be relative to the upper left corner of the screen (0,0).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::GetSize( float * pW,
                                         float * pH
                                         ) [virtual]
```

Parameters:

pW (out) address of variable to receive the width of the widget.
pH (out) address of variable to receive the height of the widget.

Returns:

CUIR_OK if successful.

This function retrieves the current width and hieght (in pixels) of a widget.

Used for: Text and UI.

virtual [uint32](#) CUIBase::GetState() [virtual]

Returns:

current widget state flags.

Used to query widget's current state. Returns a state bitmask composed of CUI_STATETYPE.

Used for: Text and UI.

virtual [HTEXTURE](#) CUIBase::GetTexture([CUI_ELEMENTTYPE](#) *elm*) [virtual]

Parameters:

elm CUI_ELEMENTTYPE to query

Returns:

HTEXTURE

This function retrieves the current HTEXTURE associated with a CUI_ELEMENTTYPE. CUI_ELEMENTTYPE is defined in [cuitypes.h](#), and breaks a widget down into texturable components. For example, a [CUIWindow](#) supports texturing the following CUI_ELEMENTTYPES:

CUIE_BG CUIE_TOP CUIE_BOTTOM CUIE_LEFT CUIE_RIGHT CUIE_TR_CORNER CUIE_TL_CORNER CUIE_BR_CORNER
CUIE_BL_CORNER

See also:

[HTEXTURE](#)

Used for: Text and UI.

Reimplemented in [CUIWidget](#).

virtual [CUI_WIDGETTYPE](#) CUIBase::GetType() [virtual]

Returns:

CUI Widget Type (as defined in [cuitypes.h](#))

This function is used to query the type of a widget.

Used for: Text and UI.

virtual [CUI_CALLBACKTYPE](#) CUIBase::GetUserCallback(void ** *ppUserCallbackObject*,
void ** *ppUserCallbackData*
) [virtual]

Parameters:

ppUserCallbackObject (out) address of user-defined callback object
ppUserCallbackData (out) address of user-defined callback data

Returns:

address of user-defined callback function

This function retrieves the user-defined callback, object, and data specified when the user callback was set.

See also:

[SetUserCallback\(\)](#).

virtual void* CUIBase::GetUserCallbackData() [virtual]

Returns:

user-defined callback data

This function retrieves the user-defined callback data specified when the user callback was set.

See also:

[SetUserCallback\(\)](#).

virtual void* CUIBase::GetUserCallbackObject() [virtual]

Returns:

user-defined callback object

This function retrieves the user-defined callback object specified when the user callback was set.

See also:

[SetUserCallback\(\)](#).

virtual [CUI_RESULTTYPE](#) CUIBase::HandleMessage([CUIMessage](#) * *pMessage*) [virtual]

Parameters:

pMessage [CUIMessage](#).

Returns:

CUI_RESULTTYPE

This function is used to cause a widget to respond to a CUIMESSAGE, and is generally used internally by the [ILTWidgetManager](#).

See also:

[ILTWidgetManager](#).

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIBase::Hide(bool *bHideChildren*) [virtual]

Parameters:

bHideChildren true or false.

Returns:

CUIR_OK if successful.

Use this function to hide a widget. If *bHideChildren* is true, the widget's children will also be hidden.

Used for: Text and UI.

virtual void CUIBase::Render() [virtual]

Returns:

CUI_RESULTTYPE

Causes a widget to draw immediately, and then render its children. Widgets that are part of the main display list (i.e., can trace their lineage back to the root widget) will be rendered when [ILTWidgetManager::Update\(\)](#) is called. For widgets that are not part of the internal hierarchy, the widget's own render function must be called.

Used for: Text and UI.

[CUI_RESULTTYPE](#) CUIBase::SendMessageToChildren([CUIMessage](#) * *pMessage*)

Parameters:

pMessage [CUIMessage](#).

Returns:

CUI_RESULTTYPE

This function is used to pass a [CUIMessage](#) to all of a widget's children. This function is generally called from within a widget's [HandleMessage\(\)](#) function.

See also:

[HandleMessage\(\)](#)

[ILTWidgetManager](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIBase::SendToBack() [virtual]

Returns:

CUI_RESULTTYPE

Sends a widget to the back (bottom) of its local display list. A parent widget will never draw above its children, but the child widgets can be re-ordered.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIBase::SetColor([CUI_ELEMENTTYPE](#) *elm*,
[uint32](#) *argb*
) [pure virtual]

Parameters:

elm CUI Element whose color should be set.

argb 32-bit color value of the form 0xAARRGGBB.

Returns:

CUI_RESULTTYPE

Use this function to set the background color of a CUI_ELEMENTTYPE. CUI_ELEMENTTYPEs are defined in [cuitypes.h](#).

Used for: Text and UI.

Implemented in [CUIWidget](#).

```
virtual CUI\_RESULTTYPE CUIBase::SetColors( CUI\_ELEMENTTYPE elm,
                                           uint32      argb0,
                                           uint32      argb1,
                                           uint32      argb2,
                                           uint32      argb3
                                           ) [pure virtual]
```

Parameters:

elm CUI Element to set the color of.
argb0 32-bit color value of the form 0xAARRGGBB.
argb1 32-bit color value of the form 0xAARRGGBB.
argb2 32-bit color value of the form 0xAARRGGBB.
argb3 32-bit color value of the form 0xAARRGGBB.

Returns:

[CUI_RESULTTYPE](#)

Use this function to set separate vertex colors for a [CUI_ELEMENTTYPE](#). [CUI_ELEMENTTYPE](#)s are defined in [cuiypes.h](#).

Used for: Text and UI.

Implemented in [CUIWidget](#).

```
virtual CUI\_RESULTTYPE CUIBase::SetGutter( int16 left,
                                           int16 right,
                                           int16 top,
                                           int16 bottom
                                           ) [virtual]
```

Parameters:

left gutter distance from the left side.
right gutter distance from the right side.
top gutter distance from the top.
bottom gutter distance from the bottom.

Returns:

[CUI_RESULTTYPE](#).

Many widget types have sub-elements that can be spaced away from the edges by using a gutter. The gutter specifies how far (in pixels) from the edges any interior parts of the UI are drawn.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::SetParent( CUIBase * pParent ) [virtual]
```

Parameters:

pParent the address of the desired parent widget.

Returns:

[CUIR_OK](#) if successful.

Use this function to set a widget's parent. This will modify the widget's render order and message handling.

See also:

[GetParent\(\)](#)

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::SetPosition( float x,
                                           float y,
                                           CUIBase * pRelativeTo,
                                           bool bMoveChildren
                                           ) [virtual]
```

Parameters:

x the new x-coordinate of the widget (in pixels).
y the new y-coordinate of the widget (in pixels).
pRelativeTo specifies the widget that the incoming (x,y) coords. are relative to
bMoveChildren specifies if the move should apply to child widgets also

Returns:

[CUIR_OK](#) if successful.

This function moves a widget to new (x,y) coordinates. If *pRelativeTo* is a valid widget address, the (x,y) coordinates will be relative to that widget's screen position. If *pRelativeTo* is [NULL](#), the (x,y) coords. will be relative to the upper left corner of the screen (0,0).

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIBase::SetRect( CUIRECT * pRect,
                                           CUIBase * pRelativeTo,
```



```

        bool      bMoveChildren
    ) [virtual]

```

Parameters:

pRect the new bounding box of the widget.
pRelativeTo specifies the widget that the (x,y) coords. are relative to
bMoveChildren specifies if the move should apply to child widgets also

Returns:

CUIR_OK if successful.

This function sets the (x,y) coordiantes of a widget, as well as the width and height. All units are in pixels.

See also:

[SetPosition\(\)](#)

Used for: Text and UI.

```

virtual CUI_RESULTTYPE CUIBase::SetSize( float    w,
                                         float    h
                                         ) [virtual]

```

Parameters:

w the new width of the widget (in pixels).
h the new height of the widget (in pixels).

Returns:

CUIR_OK if successful.

This function re-sizes a widget to a new width and height.

Used for: Text and UI.

```

virtual CUI_RESULTTYPE CUIBase::SetState( uint32 flags ) [virtual]

```

Parameters:

flags window state flags to set.

Returns:

CUIR_OK if successful.

Used to set a state bitmask composed of CUI_STATETYPE.

Used for: Text and UI.

```

virtual CUI_RESULTTYPE CUIBase::SetTexture( CUI_ELEMENTTYPE elm,
                                             HTEXTURE        hTex,
                                             bool             bTile
                                             ) [pure virtual]

```

Parameters:

elm CUI Element whose texture should be set.
hTex Texture to use.
bTile Whether to tile or not

Returns:

CUI_RESULTTYPE

Use this function to set the texture of a CUI_ELEMENTTYPE. CUI_ELEMENTTYPES are defined in [cuitypes.h](#). If *bTile* is true, the texture will tile across the element. If false, the texture will stretch to fit. Note that the amount of times a texture will tile is limited by the system architecture, and further note that not all CUI_ELEMENTTYPES support tiled textures.

Used for: Text and UI.

Implemented in [CUIWidget](#).

```

virtual CUI_RESULTTYPE CUIBase::SetUserCallback( CUI_CALLBACKTYPE pUserCallback,
                                                  void *             pUserCallbackObject,
                                                  void *             pUserCallbackData
                                                  ) [virtual]

```

Parameters:

pUserCallback the callback function pointer.
pUserCallbackObject
pUserCallbackData

Returns:

CUI_RESULTTYPE

When a widget handles a message, the widget will attempt to pass the [CUIMessage](#) on to any user-defined callback that exists. If the user-defined

callback handles the message, it should return something other than CUIR_NOT_HANDLED. If the callback funtion returns CUIR_NOT_HANDLED, the message will continue to propagate.

For a further description of pUserCallbackObject and pUserCallbackData, see [ILTWidgetManager::SetUserCallback\(\)](#).

See also:

CUI_CALLBACK

[CUIMessage](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIBase::Show(bool *bShowChildren*) [virtual]

Parameters:

bShowChildren true or false.

Returns:

CUIR_OK if successful.

Use this function to make a hidden widget visible. If *bShowChildren* is true, the widget's children will also be shown.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIBase::UnsetState([uint32](#) *flags*) [virtual]

Parameters:

flags state flags to remove.

Returns:

CUIR_OK if successful.

Used to remove a state bitmask composed of CUI_STATETYPE.

Used for: Text and UI.

Field Documentation

CUIBase_Imp!* [CUIBase::m_pImpl](#) [protected]

The documentation for this class was generated from the following file:

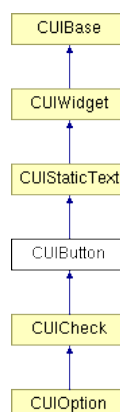
- [cuibase.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIButton Class Reference

Inheritance diagram for CUIButton:



Public Member Functions

[CUIButton\(\)](#)
[CUIButton\(CUIGUID guid\)](#)
virtual [~CUIButton\(\)](#)

Detailed Description

CUIButton is one of the CUI widgets. It displays a string of text (using an internal [CUIPolyString](#)) over one of the following CUI_ELEMENTTYPEs based on its state:

CUIE_BG, CUIE_BUTTON_PRESSED, CUIE_BUTTON_HIGHLIGHTED, CUIE_BUTTON_DISABLED.

See also:

[ILTWidgetManager](#)

[CUI_ELEMENTTYPE](#)

[SetState\(\)](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIButton::CUIButton()

CUIButton::CUIButton([CUIGUID](#) guid)

virtual CUIButton::~~CUIButton() [virtual]

The documentation for this class was generated from the following file:

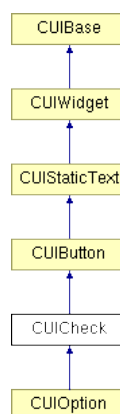
- [cuibutton.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUICheck Class Reference

Inheritance diagram for CUICheck:



Public Member Functions

[CUICheck\(\)](#)
[CUICheck\(CUIGUID guid\)](#)

virtual [~CUICheck](#) ()
virtual bool [SetValue](#) (bool value)
virtual bool [GetValue](#) ()
virtual bool [Toggle](#) ()

Detailed Description

CUICheck is one of the CUI widgets. It displays a string of text (using an internal [CUIPolyString](#)), a background image and one of the following CUI_ELEMENTTYPEs based on its state:

CUIE_BG, CUIE_CHECK_ON, CUIE_CHECK_OFF, CUIE_CHECK_DISABLED.

See also:

[ILTWidgetManager](#)

[CUI_ELEMENTTYPE](#)

[SetState\(\)](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUICheck::CUICheck()

CUICheck::CUICheck([CUIGUID](#) guid)

virtual CUICheck::~~CUICheck() [virtual]

Member Function Documentation

virtual bool CUICheck::GetValue() [virtual]

Returns:

the current value

Use this function to query a CUICheck widget's state. The return is **true** if the widget has the state CUIS_PRESSED set, or **false** otherwise.

virtual bool CUICheck::SetValue(bool value) [virtual]

Parameters:

value true or false

Returns:

the previous value

Use this function to set a CUICheck widget's state. If *value* is **true**, this function is equivalent to calling CUIBase::SetState(CUIS_PRESSED). if *value* is **false**, this function is equivalent to calling CUIBase::UnsetState(CUIS_PRESSED).

virtual bool CUICheck::Toggle() [virtual]

Returns:

the previous value

Use this function to toggle a CUICheck widget's state. If the widget currently has the state CUIS_PRESSED set, the return will be **true** and the state will be unset. If CUIS_PRESSED is not currently set, **false** will be returned and the state will be set.

The documentation for this class was generated from the following file:

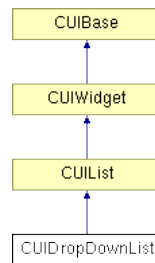
- [cuicheck.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIDropDownList Class Reference

Inheritance diagram for CUIDropDownList:



Public Member Functions

[CUIDropDownList](#) ([CUIGUID](#) guid)
 virtual [~CUIDropDownList](#) ()
 virtual [CUI_RESULTTYPE](#) [Open](#) ()
 virtual [CUI_RESULTTYPE](#) [Close](#) ()
 virtual [CUI_RESULTTYPE](#) [SetDisplayNumber](#) ([uint8](#) display)

Detailed Description

[CUIList](#) class. This class uses a collection of CUIPolyStrings to display a list of text items over a background. If the list is expanded, an additional expansion image is drawn.

CUIE_BG CUIE_CUIE_DROPDOWN_BG

See also:

class [CUIPolyString](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIDropDownList::CUIDropDownList([CUIGUID](#) guid)

virtual CUIDropDownList::~CUIDropDownList() [virtual]

Member Function Documentation

virtual [CUI_RESULTTYPE](#) CUIDropDownList::Close() [virtual]

Returns:

CUIR_OK

Use this function to close (raise) a drop-down list.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIDropDownList::Open() [virtual]

Returns:

CUIR_OK

Use this function to open (drop) a drop-down list.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIDropDownList::SetDisplayNumber([uint8](#) display) [virtual]

Returns:

CUIR_OK

Use this function to set the number of items displayed when a drop-down list is open.

Used for: Text and UI.

The documentation for this class was generated from the following file:

- [cuidropdownlist.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIFont Class Reference

Public Member Functions

```
virtual const char * GetClassName ()=0
    virtual ~CUIFont ()
    virtual bool IsValid ()=0
    virtual bool IsProportional ()=0
virtual HTEXTURE GetTexture ()=0
    virtual uint32 GetAttributes ()=0
    virtual uint8 GetDefCharScreenWidth ()=0
    virtual uint8 GetDefCharScreenHeight ()=0
    virtual uint8 GetCharTexWidth ()=0
    virtual uint8 GetCharTexHeight ()=0
    virtual uint8 GetDefSpacingH ()=0
    virtual uint8 GetDefSpacingV ()=0
    virtual int8 GetDefSlant ()=0
    virtual int8 GetDefBold ()=0
    virtual uint16 * GetFontTable ()=0
    virtual uint8 * GetFontMap ()=0
    virtual uint32 * GetDefColors ()=0
virtual CUI\_RESULTTYPE SetTexture (HTEXTURE hTex, uint8 cwidth=0, uint8 cheight=0)=0
virtual CUI\_RESULTTYPE SetTexture (HTEXTURE hTex, uint16 *pTable=NULL, uint8 cheight=0)=0
    virtual void SetMap (uint8 *pMap)=0
    virtual void SetAttributes (uint32 attrs)=0
    virtual void UnsetAttributes (uint32 attrs)=0
    virtual void SetDefCharWidth (uint8 width)=0
    virtual void SetDefCharHeight (uint8 height)=0
    virtual void SetDefSpacingH (uint8 hspacing)=0
    virtual void SetDefSpacingV (uint8 vspacing)=0
    virtual void SetDefSlant (int8 slant)=0
    virtual void SetDefBold (int8 bold)=0
    virtual void SetDefColor (uint32 argb)=0
    virtual void SetDefColors (uint32 argb0, uint32 argb1, uint32 argb2, uint32 argb3)=0
    virtual void DrawString (float x, float y, char *text)=0
    virtual void Apply (CUIPolyString *pPolyStr, int16 index=0, int16 num=0)=0
```

Detailed Description

The CUIFont class is used to store font resources. Fonts can be monospace or proportional, and can be created from HTEXTURES or TrueType resources.

See also:

interface [ILTFontManager](#)

class [CUIPolyString](#)

Used for: Text and UI.

Constructor & Destructor Documentation

```
virtual CUIFont::~CUIFont( ) [inline, virtual]
```

Member Function Documentation

```
virtual void CUIFont::Apply( CUIPolyString * pPolyStr,
    int16 index = 0,
    int16 num = 0
) [pure virtual]
```

Parameters:

pPolyString [CUIPolyString](#) to modify.
index (optional) index of the first character to modify.
num (optional) number of characters to modify.

Applies a CUIFont to a [CUIPolyString](#). If *index* is not specified, it will be the first character of the string. If *num* is not specified, all characters from *index* to the end of the string will be modified.

Apply allows you to modify all or part of a string. If you modify only part of a [CUIPolyString](#), however, you may *only* change the color, size, or attributes of the font. If you change the font's texture or font table and then apply the font to only a portion of the string, the results could be garbled on screen.

Note: This function is deprecated and will be removed in a future release. Use [CUIPolyString::ApplyFont\(\)](#) instead.

See also:

[CUIPolyString](#)

Used for: Text and UI.

```
virtual void CUIFont::DrawString( float    x,
                                float    y,
                                char *    text
                                ) [pure virtual]
```

Parameters:

x horizontal screen coordinate (in pixels).
y vertical screen coordinate (in pixels).
text a string of text.

Draws a string of text to the screen. This function essentially allocates a [CUIPolyString](#), draws it, and then frees it. For persistent text, it is recommended to create and store your own [CUIPolyString](#) objects -- it will be much more resource friendly.

See also:

class [CUIPolyString](#)

[ILTFontManager::CreatePolyString\(\)](#)

Used for: Text and UI.

```
virtual uint32 CUIFont::GetAttributes( ) [pure virtual]
```

Returns:

uint32 flags.

This function is used to query the attribute flags associated with a font.

See also:

[SetAttributes\(\)](#)

[UnsetAttributes\(\)](#)

[CUI_FONTFXTYPE](#)

Used for: Text and UI.

```
virtual uint8 CUIFont::GetCharTexHeight( ) [pure virtual]
```

Returns:

height of one character's texture (in pixels).

This is the value that is passed in when creating monospace or proportional bitmap fonts.

Used for: Text and UI.

```
virtual uint8 CUIFont::GetCharTexWidth( ) [pure virtual]
```

Returns:

width of one character's texture (in pixels).

This is the value that is passed in when creating monospace bitmap fonts.

Used for: Text and UI.

```
virtual const char* CUIFont::GetClassName( ) [pure virtual]
```

```
virtual int8 CUIFont::GetDefBold( ) [pure virtual]
```

Returns:

The font's default bold value (in pixels).

See also:

[CUIFont::SetDefBold\(\)](#)

Used for: Text and UI.

virtual [uint8](#) CUIFont::GetDefCharScreenHeight() [pure virtual]

Returns:

default height of one character on screen (in pixels).

This is not a per-character value. This is the font's height. The height can be set with the [CUIFont::SetDefCharHeight\(\)](#) function.

Used for: Text and UI.

virtual [uint8](#) CUIFont::GetDefCharScreenWidth() [pure virtual]

return default width of one character on screen (in pixels).

For proportional fonts, function returns the default width of a space character. This width can be set with the [CUIFont::SetDefCharWidth\(\)](#) function.

Used for: Text and UI.

virtual [uint32](#)* CUIFont::GetDefColors() [pure virtual]

Returns:

pointer to an array of 4 default uint32 colors of the form 0xAARRGGBB.

See also:

[CUIFont::SetDefColor\(\)](#)

[CUIFont::SetDefColors\(\)](#)

Used for: Text and UI.

virtual [int8](#) CUIFont::GetDefSlant() [pure virtual]

Returns:

The font's default italic slant (in pixels).

See also:

[CUIFont::SetDefSlant\(\)](#)

Used for: Text and UI.

virtual [uint8](#) CUIFont::GetDefSpacingH() [pure virtual]

Returns:

default horizontal spacing between characters (in pixels).

See also:

[CUIFont::SetDefSpacing\(\)](#)

Used for: Text and UI.

virtual [uint8](#) CUIFont::GetDefSpacingV() [pure virtual]

Returns:

default vertical spacing between characters (in pixels).

See also:

[CUIFont::SetDefSpacingV\(\)](#)

Used for: Text and UI.

virtual [uint8](#)* CUIFont::GetFontMap() [pure virtual]

Returns:

The font's character map.

See also:

[ILTFontManager](#)

Used for: Text and UI.

virtual [uint16](#)* CUIFont::GetFontTable() [pure virtual]

Returns:

The font's character uv lookup table.

See also:[ILTFontManager](#)

Used for: Text and UI.

virtual [HTEXTURE](#) CUIFont::GetTexture() [pure virtual]**Returns:**

HTEXTURE.

This function returns a handle to the texture associated with the font.

See also:[ILTTexInterface](#)

Used for: Text and UI.

virtual bool CUIFont::IsProportional() [pure virtual]

Returns:

bool.

This function is used to query whether or not the font in question is proportional (as opposed to monospace).

Used for: Text and UI.

virtual bool CUIFont::IsValid() [pure virtual]

Returns:

bool.

This function is used to query whether or not the font in question is valid. This is used internally by ILFontManager.

Used for: Text and UI.

virtual void CUIFont::SetAttributes([uint32](#) *attrs*) [pure virtual]**Parameters:***attrs* New font flags.

This function is used to set the attribute flags for a font. More than one flag can be set at a time, e.g.:

[CUIFont::SetAttributes\(CUI_FONT_ITALIC | CUI_FONT_BOLD\);](#)**See also:**[GetAttributes\(\)](#)[UnsetAttributes\(\)](#)[CUI_FONTTEXTYPE](#)

Used for: Text and UI.

virtual void CUIFont::SetDefBold([int8](#) *bold*) [pure virtual]**Parameters:***bold* Sets the font's default bold amount (in pixels).*Bold* is the amount that a character's width is increased (or decreased) from the normal character width.**Note:** Simply setting *bold* for a font will not cause the font to be emboldened. You must set the font attribute CUI_FONT_BOLD.**See also:**[SetAttributes\(\)](#)

Used for: Text and UI.

virtual void CUIFont::SetDefCharHeight([uint8](#) *height*) [pure virtual]**Parameters:***height* Sets the font's default character height on screen (in pixels).This function affects both monospace and proportional fonts. When a [CUIPolyString](#) or a CUI widget that displays text is created with this font, this is default character height.

Used for: Text and UI.

virtual void CUIFont::SetDefCharWidth([uint8](#) *width*) [pure virtual]**Parameters:***width* Sets the font's default character width on screen (in pixels).

For proportional fonts, this function sets the number of pixels that a 'space' character uses. For monospace fonts, the screen width of every character is set to *width*. When a [CUIPolyString](#) or a CUI widget that displays text is created, this is default character width.

Used for: Text and UI.

```
virtual void CUIFont::SetDefColor( uint32 argb ) [pure virtual]
```

Parameters:

argb Sets the font's default color.

Sets a single color for each character of the entire font.

Used for: Text and UI.

```
virtual void CUIFont::SetDefColors( uint32 argb0,
                                   uint32 argb1,
                                   uint32 argb2,
                                   uint32 argb3
                                   ) [pure virtual]
```

Parameters:

argb0 Sets the font's character color for vertex 0.

argb1 Sets the font's character color for vertex 1.

argb2 Sets the font's character color for vertex 2.

argb3 Sets the font's character color for vertex 3.

Sets the default vertex colors for each character of the entire font (each character is represented internally as an [LT_POLYGT4](#)). Vertices are numbered clockwise beginning with vertex 0 at the top left corner.

See also:

struct [LT_POLYGT4](#)

interface [ILTDrawPrim](#)

Used for: Text and UI.

```
virtual void CUIFont::SetDefSlant( int8 slant ) [pure virtual]
```

Parameters:

slant Sets the font's default italic slant (in pixels).

Slant is the distance that a character's top is offset horizontally from its bottom when italic characters are being drawn. A positive value for *slant* will cause the characters to lean to the right, while a negative value for *slant* will cause the characters to lean to the left.

Note: Simply setting the *slant* of a font will not cause the font to be italicized. You must set the font attribute CUI_FONT_ITALIC.

See also:

[SetAttributes\(\)](#)

Used for: Text and UI.

```
virtual void CUIFont::SetDefSpacingH( uint8 hspacing ) [pure virtual]
```

Parameters:

spacing Sets the font's default horizontal character spacing on screen (in pixels).

Spacing is the distance between the left edge of a character and the right edge of the preceeding character.

Used for: Text and UI.

```
virtual void CUIFont::SetDefSpacingV( uint8 vspacing ) [pure virtual]
```

Parameters:

spacing Sets the font's default vertical character spacing on screen (in pixels).

Spacing is the distance between the top edge of a character and the bottom edge of the character above (in a multi-line string).

Used for: Text and UI.

```
virtual void CUIFont::SetMap( uint8 * pMap ) [pure virtual]
```

Parameters:

pMap pointer to a font map.

Returns:

none.

This function is used to set a font's map. A font map is an array of uint8 which maps characters to their positions in font table (which in turn determines their u,v coordinates). For example, if the element [32] of the font map array was equal to 10, then ascii character '!' would map to entry

[10] in the font table.

This is useful in such cases as using European characters with accents and umlauts--such characters are sprinkled throughout the upper 128 ascii indices and you may not wish to use up valuable VRAM by including unprintable or unused characters in your font. Or, you may wish to map all lowercase characters to their capital counterparts for example.

NOTE: If you provide a font map, you are responsible for allocating and freeing the memory.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIFont::SetTexture( HTEXTURE hTex,
                                           uint16* pTable = NULL,
                                           uint8 height = 0
                                           ) [pure virtual]
```

Parameters:

hTex New texture.
pTable (optional) Address of new font table.
height (optional) New character height (in pixels, relative to the texture map).

Returns:

CUIR_OK if successful.

This function is used to set a proportional font's texture map and optionally set a new character height (within that texture, not onscreen). If *height* is not specified, the characters' texture height coordinates will not be changed from their original values.

See also:

ILTFontManager::CreateFont(HTEXTURE, uint8*, uint8)

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIFont::SetTexture( HTEXTURE hTex,
                                           uint8 cwidth = 0,
                                           uint8 height = 0
                                           ) [pure virtual]
```

Parameters:

hTex New texture.
cwidth (optional) New character width (in pixels, relative to the texture map).
height (optional) New character height (in pixels, relative to the texture map).

Returns:

CUIR_OK if successful.

This function is used to set a monospace font's texture map and optionally set new character dimensions (relative to the texture map, not on-screen). If *cwidth* or *height* are not specified, the character dimensions will not be changed from their original values.

Used for: Text and UI.

```
virtual void CUIFont::UnsetAttributes( uint32 attrs ) [pure virtual]
```

Parameters:

attrs Font flags to unset.

This function is used to unset the attribute flags for a font. More than one flag can be unset at a time, e.g.:

```
CUIFont::UnsetAttributes(CUI\_FONT\_ITALIC | CUI\_FONT\_BOLD) ;
```

See also:

[GetAttributes\(\)](#)

[SetAttributes\(\)](#)

[CUI_FONTFXTYPE](#)

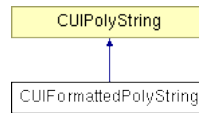
Used for: Text and UI.

The documentation for this class was generated from the following file:

- [cuifont.h](#)

CUIFormattedPolyString Class Reference

Inheritance diagram for CUIFormattedPolyString:



Public Member Functions

[CUIFormattedPolyString](#) ([CUIFont](#) *pFont, const char *pBuf=NULL, float x=0.0, float y=0.0, [CUI_ALIGNMENTTYPE](#) alignment=CUI_HALIGN_LEFT)
 virtual const char * [GetClassName](#) ()
 virtual [~CUIFormattedPolyString](#) ()
 virtual [CUI_RESULTTYPE](#) [SetAlignmentH](#) ([CUI_ALIGNMENTTYPE](#) halign)
 virtual [CUI_RESULTTYPE](#) [SetWrapWidth](#) ([uint16](#) wrap)

Detailed Description

CUIFormattedPolyString class. This class is used to create and store collections of polygons that have been mapped to font textures via the [CUIFont](#) class. In addition to the functionality of the regular [CUIPolyString](#), the CUIFormattedPolyString adds support for newlines, word-wrap, and justification.

See also:

class [CUIFont](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIFormattedPolyString::CUIFormattedPolyString([CUIFont](#) * *pFont*,
 const char * *pBuf*=NULL,
 float *x* = 0.0,
 float *y* = 0.0,
[CUI_ALIGNMENTTYPE](#) *alignment* = CUI_HALIGN_LEFT
)

virtual CUIFormattedPolyString::~CUIFormattedPolyString() [virtual]

Member Function Documentation

virtual const char* CUIFormattedPolyString::GetClassName() [virtual]

Reimplemented from [CUIPolyString](#).

virtual [CUI_RESULTTYPE](#) CUIFormattedPolyString::SetAlignmentH([CUI_ALIGNMENTTYPE](#) *halign*) [virtual]

Parameters:

align new text alignment.

Returns:

CUIR_OK if successful.

Sets the horizontal alignment of a CUIFormattedPolyString. For a CUIFormattedPolyString, the horizontal alignment is based on the string's (x,y) coordinate. If the *align* is CUI_HALIGN_LEFT, the (x,y) will refer to the upper left corner of the string. If *align* is CUI_HALIGN_CENTER, the string's x coordinate will refer to the horizontal midpoint of the string. If *align* is CUI_HALIGN_RIGHT, (x,y) will be the upper right corner of the string. if *align* is CUI_HALIGN_JUSTIFIED, (x,y) will be the upper left corner and the string's text will spread out to fill the space between x and (x + wrapwidth)

See also:

[SetWrapWidth\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIFormattedPolyString::SetWrapWidth([uint16](#) *wrap*) [virtual]

Parameters:

wrap wrap-width for each line.

Returns:

CUIR_OK if successful.

Sets the word-wrap width (in pixels) for a CUIFormattedPolyString. The string will honor newlines (' ') and will attempt to perform word wrapping. If *wrap* is zero, word wrap will be turned off for this poly string (and as a side-effect, the string will no longer honor newline characters). The wrap width also determines the right margin when a CUIFormattedPolystring's alignment is set to CUI_HALIGN_JUSTIFIED.

See also:

[SetAlignmentH\(\)](#)

Used for: Text and UI.

The documentation for this class was generated from the following file:

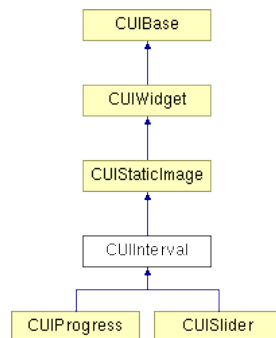
- [cuiformattedpolystring.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIInterval Class Reference

Inheritance diagram for CUIInterval:



Public Member Functions

```

virtual ~CUIInterval ()
virtual int32 GetMaxValue ()
virtual int32 GetMinValue ()
virtual CUI\_RESULTTYPE GetRange (int32 *pMin, int32 *pMax)
virtual int32 GetCurrentValue ()
virtual CUI\_ORIENTTYPE GetOrientation ()
virtual CUI\_RESULTTYPE SetMaxValue (int32 max)
virtual CUI\_RESULTTYPE SetMinValue (int32 min)
virtual CUI\_RESULTTYPE SetRange (int32 min, int32 max)
virtual CUI\_RESULTTYPE SetCurrentValue (int32 val)
virtual CUI\_RESULTTYPE SetOrientation (CUI\_ORIENTTYPE orient)
virtual CUI\_RESULTTYPE Increment (int32 inc)
virtual CUI\_RESULTTYPE IncrementPercent (int32 percent)
virtual CUI\_RESULTTYPE QueryPoint (int16 x, int16 y)
  
```

Detailed Description

CUIInterval is a class containing member data and functions which are common to widgets such as [CUISlider](#) and [CUIProgress](#). This class cannot be directly instantiated.

See also:

[ILTWidgetManager](#)

[CUISlider](#)

[CUIProgress](#)

Used for: Text and UI.

Constructor & Destructor Documentation

virtual CUIInterval::~CUIInterval() [virtual]

Member Function Documentation

virtual [int32](#) CUIInterval::GetCurrentValue() [virtual]

Returns:

the object's current value.

See also:

[SetCurrentValue\(\)](#)

Used for: Text and UI.

virtual [int32](#) CUIInterval::GetMaxValue() [virtual]

Returns:

the object's maximum value.

See also:

[SetMaxValue\(\)](#)

Used for: Text and UI.

virtual [int32](#) CUIInterval::GetMinValue() [virtual]

Returns:

the object's minimum value.

See also:

[SetMinValue\(\)](#)

Used for: Text and UI.

virtual [CUI_ORIENTATIONTYPE](#) CUIInterval::GetOrientation() [virtual]

Returns:

the object's screen orientation.

Returns either CUI_ORIENT_HORIZONTAL or CUI_ORIENT_VERTICAL.

See also:

[SetOrientation\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::GetRange([int32](#) * *pMin*,
[int32](#) * *pMax*
) [virtual]

Parameters:

pMin address of variable to receive the object's minimum value

pMax address of variable to receive the object's maximum value

Returns:

CUIR_OK.

GetRange retrieves the maximum and minimum values from the object.

See also:

[SetRange\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::Increment([int32](#) *inc*) [virtual]

Parameters:

inc the amount by which to increment the object's current value.

Returns:

CUI_RESULTTYPE.

The object's current value will be incremented by *inc*. *Inc* can be positive or negative. An object's current value will never go outside the range of its maximum or minimum values.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::IncrementPercent([int32](#) *percent*) [virtual]

Parameters:

inc the percentage by which to increment the object's current value.

Returns:

CUI_RESULTTYPE.

The object's current value will be incremented by $(\text{max_value} - \text{min_value}) * \text{percent}$. *Percent* can be positive or negative. An object's current value will never go outside the range of its maximum or minimum values.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::QueryPoint([int16](#) *x*,
[int16](#) *y*
) [virtual]

Parameters:

x x-coordinate to test.

y y-coordinate to test.

Returns:

CUI_RESULTTYPE.

Use this function to test a point (screen coordinates) inside the object. If the object is a [CUISlider](#), possible return values are:

CUIR_SLIDER_MIN if the point is in the area that represents the range from *min_value* to *current_value*. *CUIR_SLIDER_BAR* if the point is within the slider bar's movable element (*current_value*). *CUIR_SLIDER_MAX* if the point is in the area that represents the range from *current_value* to *max_value*. *CUIR_ERROR* if the point is outside the confines of the [CUISlider](#) widget.

If the object is a [CUIProgress](#), possible return values are: *CUIR_PROGRESS_LESS* if the point is in the area that represents the range from *min_value* to *current_value*. *CUIR_PROGRESS_GREATER* if the point is in the area that represents the range from *current_value* to *max_value*. *CUIR_ERROR* if the point is outside the confines of the [CUIProgress](#) widget.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::SetCurrentValue([int32](#) *val*) [virtual]

Parameters:

val the object's new current value.

Returns:

CUI_RESULTTYPE.

See also:

[GetCurrentValue\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::SetMaxValue([int32](#) *max*) [virtual]

Parameters:

max new maximum value.

Returns:

CUI_RESULTTYPE.

Sets the object's maximum value.

See also:

[GetMaxValue\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::SetMinValue([int32](#) *min*) [virtual]

Parameters:

min new minimum value.

Returns:

CUI_RESULTTYPE.

Sets the object's minimum value.

See also:

[GetMinValue\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::SetOrientation([CUI_ORIENTATIONTYPE](#) *orient*) [virtual]

Parameters:

orient the object's new screen orientation.

Returns:

CUI_RESULTTYPE.

Specifies whether the object is drawn vertically (CUI_ORIENT_VERTICAL) or horizontally (CUI_ORIENT_HORIZONTAL).

See also:

[GetOrientation\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIInterval::SetRange([int32](#) *min*,
 [int32](#) *max*
) [virtual]

Parameters:

min new minimum value.
max new maximum value.

Returns:

CUI_RESULTTYPE.

Sets the object's minimum and maximum values.

See also:

[GetRange\(\)](#)

Used for: Text and UI.

The documentation for this class was generated from the following file:

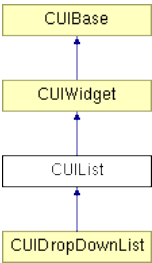
- [cuiinterval.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIList Class Reference

Inheritance diagram for CUIList:



Public Member Functions

[CUIList](#) ()
[CUIList](#) ([CUIGUID](#) guid)
virtual [~CUIList](#) ()
virtual [CUIFont](#) * [GetFont](#) ()
virtual [CUI_RESULTTYPE](#) [SetFont](#) ([CUIFont](#) *pFont)
virtual [CUI_RESULTTYPE](#) [SetCharHeight](#) ([uint8](#) height)
virtual [CUI_RESULTTYPE](#) [AddItem](#) (const char *pText, [int32](#) index=-1)
virtual [CUI_RESULTTYPE](#) [RemoveItem](#) ([int32](#) index)
virtual [int32](#) [FindItem](#) (const char *pText)
virtual const char * [GetItem](#) ([int32](#) index)
virtual [CUI_RESULTTYPE](#) [SetSelection](#) ([int32](#) index)
virtual [int32](#) [GetSelection](#) ()
virtual [uint32](#) [GetItemCount](#) ()
virtual [int32](#) [QueryPoint](#) (float x, float y)
virtual [CUI_RESULTTYPE](#) [Scroll](#) ([int32](#) number)

Detailed Description

CUIList class. This class is uses a collection of CUIPolyStrings to display a list of text items over a background.

CUIE_BG

See also:

class [CUIPolyString](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIList::CUIList()

CUIList::CUIList([CUIGUID](#) *guid*)

virtual CUIList::~CUIList() [virtual]

Member Function Documentation

virtual [CUI_RESULTTYPE](#) CUIList::AddItem(const char * *pText*,
[int32](#) *index* = -1
) [virtual]

Parameters:

pText text item to add

index (optional) position at which to add the new item.

Use this function to add a text item to the CUIList widget. If *index* is -1, the item will be added at the end of the list. If *index* is greater than the number of items currently in the list, the new item will be added at the end.

Used for: Text and UI.

virtual [int32](#) CUIList::FindItem(const char * *pText*) [virtual]

Parameters:

pText text item to find

Use this function to locate a text item in the list. The entire string is matched and the match is case sensitive.

Returns:

index of the matching item, or -1 if the search item does not exist in the list.

Used for: Text and UI.

virtual [CUIFont*](#) CUIList::GetFont() [virtual]

Returns:

current font

Use this function to retrieve the address of the [CUIFont](#) which is being used by the CUIList widget.

Used for: Text and UI.

virtual const char* CUIList::GetItem([int32](#) *index*) [virtual]

Parameters:

index item to retrieve

Use this function to retrieve the text of an item in the list.

Returns:

item text

Used for: Text and UI.

virtual [uint32](#) CUIList::GetItemCount() [virtual]

Use this function to get the total number of items in the list.

Returns:

number of items

Used for: Text and UI.

virtual [int32](#) CUIList::GetSelection() [virtual]

Use this function to get the current selection in the list. Only one item can be selected at a time.

Returns:

index of the currently selected item.

Used for: Text and UI.

virtual [int32](#) CUIList::QueryPoint(float x,
float y
) [virtual]

Parameters:

x horizontal coordinate to test

y vertical coordinate to test

Use this function to test whether a point on the screen is within list items.

Returns:

index of the item containing specified coords. or -1 if x and y do not lie inside an item's bounding box.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIList::RemoveItem([int32](#) index) [virtual]

Parameters:

index index of item to remove

Use this function to remove a list item. *Index* is a zero-based (i.e., the first item is index zero).

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIList::Scroll([int32](#) number) [virtual]

Parameters:

number number of lines to scroll

Use this function to scroll the list up (*number* is negative) or down (*number* is positive). You cannot scroll past the beginning or the end of the list.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIList::SetCharHeight([uint8](#) height) [virtual]

Parameters:

height height of text

Use this function to set the height of text items displayed by the CUIList widget.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIList::SetFont([CUIFont](#) * pFont) [virtual]

Parameters:

pFont address of font to set

Use this function to set the [CUIFont](#) used by the CUIList widget.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIList::SetSelection([int32](#) index) [virtual]

Parameters:

index index of the item to select

Use this function to set the current selection in the list. Only one item can be selected at a time.

Used for: Text and UI.

The documentation for this class was generated from the following file:

- [cuilist.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIMessage Class Reference

Public Member Functions

[CUIMessage](#) ()
[CUIMessage](#) ([CUI_MESSAGETYPE](#) message, [CUIBase](#) *pSender, [CUIBase](#) *pRecipient, [uint32](#) data1, [uint32](#) data2)

Data Fields

void * [m_pUserObject](#)
 void * [m_pUserData](#)
[CUI_MESSAGETYPE](#) [m_Message](#)
[CUIBase](#) * [m_pSender](#)
[CUIBase](#) * [m_pRecipient](#)
[uint32](#) [m_Data1](#)
[uint32](#) [m_Data2](#)

Detailed Description

CUIMessage class. This is used to pass CUI system messages among widgets in the ILTWidgetManager's hierarchy. Widget's have almost no default message handling behavior. It is up to the Client Shell to enqueue messages and respond to them in a callback function.

For example: To deal with a *mouse click*, you must first detect the mouse click in the routine(s) where you poll for input from the mouse, keyboard, joystick, etc. You would then enqueue a CUIM_ON_MOUSE_CLICK message with [ILTWidgetManager::EnqueueMessage\(\)](#). In one of your UI callback functions, you receive the CUIM_ON_MOUSE_CLICK message, check to see if it occurred within the boundaries of the receiving widget, and take appropriate action in your client code.

There are 2 ways to receive message callbacks, if you want them. You must set up one or more callback functions, and then install it either with [CUIBase::SetUserCallback\(\)](#) or [ILTWidgetManager::SetUserCallback\(\)](#). If you install a user callback for the [ILTWidgetManager](#), you will receive one instance of every message in the queue during the update. If you install one or more [CUIBase](#) callbacks, you will receive an instance of each message for every widget that has installed the same callback. For example: If you were to create 3 CUIButtons and a *ButtonCallback()* function, and call [CUIBase::SetUserCallback\(\)](#) for each [CUIButton](#), then you would receive 3 instances of every message, each one with the [CUIMessage::m_pRecipient](#) set to one of the three CUIButtons.

How you handle message callbacks (or *if* you handle them) is completely up to you.

See also:

interface [ILTWidgetManager](#)

ILTWidgetManaget::SetUserCallback

[CUIBase::SetUserCallback](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIMessage::CUIMessage() [inline]

A CUIMessage can be constructed with all members initialised to NULL, and the message set to CUIM_NO_MESSAGE.

See also:

[CUI_MESSAGETYPE](#)

Used for: Text and UI.

CUIMessage::CUIMessage([CUI_MESSAGETYPE](#) message,
 [CUIBase](#) * pSender,
 [CUIBase](#) * pRecipient,
 [uint32](#) data1,
 [uint32](#) data2
) [inline]

Parameters:

message CUI_MESSAGETYPE for this message to carry.
pSender address of the sending widget (can be NULL)
pRecipient address of the current receiving widget (can be NULL).

data1 This can be a pointer or a value.
data2 This can be a pointer or a value.

A CUIMessage can be constructed by passing in appropriate initialization values.

See also:

[CUI_MESSAGE_TYPE](#)

[CUIBase](#)

Used for: Text and UI.

Field Documentation

[uint32 CUIMessage::m_Data1](#)

One of two data storage members.

Used for: Text and UI.

[uint32 CUIMessage::m_Data2](#)

One of two data storage members.

Used for: Text and UI.

[CUI_MESSAGE_TYPE CUIMessage::m_Message](#)

The message type.

See also:

[CUI_MESSAGE_TYPE](#)

Used for: Text and UI.

[CUIBase* CUIMessage::m_pRecipient](#)

This is not the address of a message's intended target! It is the address of the widget that is currently handling the message. If you know the widget who should specifically receive a message, you can call that widget's [CUIBase::HandleMessage\(\)](#) function.

Used for: Text and UI.

[CUIBase* CUIMessage::m_pSender](#)

The address of the widget who sent the message (if any)

Used for: Text and UI.

[void* CUIMessage::m_pUserData](#)

This optional user-data member is filled in by the caller.

See also:

[ILTWidgetManager::SetUserCallback](#)

[CUIBase::SetUserCallback](#)

Used for: Text and UI.

[void* CUIMessage::m_pUserObject](#)

Since non-static member functions cannot be used as callbacks, A CUIMessage stores a pointer to an object that can later be cast to the desired object type within a static member function. **m_pObject** can also be used as user specified data.

See also:

[ILTWidgetManager::SetUserCallback](#)

[CUIBase::SetUserCallback](#)

Used for: Text and UI.

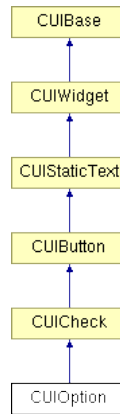
The documentation for this class was generated from the following file:

- [cuimessage.h](#)

API Reference

CUIOption Class Reference

Inheritance diagram for CUIOption:



Public Member Functions

[CUIOption](#) ([CUIGUID](#) guid)
virtual [~CUIOption](#) ()

Detailed Description

CUIOption is one of the CUI widgets. It displays a string of text (using an internal [CUIPolyString](#)) and displays the following CUI_ELEMENTTYPEs based on its state:

CUIE_BG, CUIE_OPTION_ON, CUIE_OPTION_OFF, CUIE_OPTION_DISABLED.

CUIOption is almost identical to CUICheck, except that only one sibling at a time can be set to *true*. Calling SetValue(**true**) will cause all other CUIOption siblings to change their value to false.

See also:

[ILTWidgetManager](#)

[CUI_ELEMENTTYPE](#)

[SetState\(\)](#)

[SetValue\(\)](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIOption::CUIOption([CUIGUID](#) guid)

virtual CUIOption::~~CUIOption() [virtual]

The documentation for this class was generated from the following file:

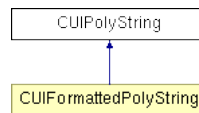
- [cuioption.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIPolyString Class Reference

Inheritance diagram for CUIPolyString:



Public Member Functions

```

CUIPolyString ()
CUIPolyString (CUIFont *pFont, const char *pBuffer=NULL, float x=0.0, float y=0.0)
virtual ~CUIPolyString ()
virtual const char * GetClassName ()
virtual CUI_RESULTTYPE SetText (const char *pBuffer)
virtual CUI_RESULTTYPE SetColor (uint32 argb)
virtual CUI_RESULTTYPE SetColors (uint32 argb0, uint32 argb1, uint32 argb2, uint32 argb3)
virtual CUI_RESULTTYPE SetCharScreenSize (uint8 height, uint8 width=0)
virtual CUI_RESULTTYPE SetCharScreenWidth (uint8 width)
virtual CUI_RESULTTYPE SetCharScreenHeight (uint8 height)
    virtual uint8 GetCharScreenWidth ()
    virtual uint8 GetCharScreenHeight ()
virtual CUI_RESULTTYPE SetPosition (float x, float y)
virtual CUI_RESULTTYPE SetFont (CUIFont *pFont)
    virtual const char * GetText ()
    virtual LT_POLYGT4 * GetPolys ()
    virtual CUIFont * GetFont ()
virtual CUI_RESULTTYPE GetDims (float *pWidth, float *pHeight)
virtual CUI_RESULTTYPE GetPosition (float *pX, float *pY)
virtual CUI_RESULTTYPE GetRect (CUIRECT *pRect)
    virtual float GetHeight ()
    virtual float GetWidth ()
    virtual float GetX ()
    virtual float GetY ()
    virtual uint16 GetLength ()
virtual CUI_RESULTTYPE ApplyFont (CUIFont *pFont, int16 index=0, int16 num=0, bool bProcessRemainder=true)
virtual CUI_RESULTTYPE Render (int32 start=0, int32 end=-1)
virtual CUI_RESULTTYPE RenderClipped (CUIRECT *pClipRect, int32 start=0, int32 end=-1)
    virtual bool IsValid ()
CUIPolyString_Impl * GetImpl ()
  
```

Protected Attributes

CUIPolyString_Impl * [m_pImpl](#)

Detailed Description

CUIPolyString class. This class is used to create and store collections of polygons that have been mapped to font textures via the [CUIFont](#) class.

See also:

class [CUIFont](#)

class [CUIFormattedPolyString](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIPolyString::CUIPolyString()

```

CUIPolyString::CUIPolyString( CUIFont * pFont,
    const char * pBuffer = NULL,
    float      x = 0.0,
    float      y = 0.0
)
  
```

virtual CUIPolyString::~~CUIPolyString() [virtual]

Member Function Documentation

```
virtual CUI\_RESULTTYPE CUIPolyString::ApplyFont( CUIFont * pFont,
                                                int16      index = 0,
                                                int16      num = 0,
                                                bool        bProcessRemainder = true
                                                ) [virtual]
```

Parameters:

pFont [CUIFont](#) to use.
index (optional) index of the first character to modify.
num (optional) number of characters to modify.
bProcessRemainder (optional) adjust positions of chars following the modification.

Applies a [CUIFont](#) to a CUIPolyString. If *index* is not specified, it will be the first character of the string. If *num* is not specified, all characters from *index* to the end of the string will be modified. If **ProcessRemainder** is not specified, all characters following the modification will be adjusted horizontally.

This version of ApplyFont allows you to modify all or part of a string. If you modify only part of a CUIPolyString, however, you may *only* change the color, size, or attributes of the font. If you change the font's texture or font table and then apply the font to only a portion of the string, the results could be garbled on screen.

Used for: Text and UI.

```
virtual uint8 CUIPolyString::GetCharScreenHeight( ) [virtual]
```

Returns:

character screen height.

Sets the screen height of the CUIPolyString's characters.

Used for: Text and UI.

```
virtual uint8 CUIPolyString::GetCharScreenWidth( ) [virtual]
```

Returns:

character screen width.

retrieves the screen width of the CUIPolyString's characters.

Used for: Text and UI.

```
virtual const char* CUIPolyString::GetClassName( ) [virtual]
```

Reimplemented in [CUIFormattedPolyString](#).

```
virtual CUI\_RESULTTYPE CUIPolyString::GetDims( float * pWidth,
                                                float * pHeight
                                                ) [virtual]
```

Parameters:

pWidth address of variable to receive the width (in pixels)
pHeight address of variable to receive the height (in pixels)

Returns:

CUI_RESULTTYPE.

Gets the current screen dimensions of the CUIPolyString

See also:

[GetPosition\(\)](#)

[GetRect\(\)](#)

Returns CUIR_OK.

Used for: Text and UI.

```
virtual CUIFont* CUIPolyString::GetFont( ) [virtual]
```

Returns:

CUIFont*.

Returns a pointer to the current font associated with the CUIPolyString. If you wish to change the font, use [CUIFont::Apply\(\)](#)

See also:

[CUIFont::Apply\(\)](#)

Used for: Text and UI.

```
virtual float CUIPolyString::GetHeight( ) [virtual]
```

Returns:

float.

Gets the current screen height of the CUIPolyString (in pixels).

Used for: Text and UI.

CUIPolyString_Impl* CUIPolyString::GetImpl() [inline]

virtual [uint16](#) CUIPolyString::GetLength() [virtual]

Returns:

int16.

Gets the length of the ascii representation of the CUIPolyString (in characters).

Used for: Text and UI.

virtual [LT_POLYGT4](#)* CUIPolyString::GetPolys() [virtual]

Returns:

LT_POLYGT4*.

Returns a pointer to the polygons which make up a CUIPolyString. These polygons can be directly manipulated at runtime for various visual effects.

See also:

class [ILTDrawPrim](#)

struct [LT_POLYGT4](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::GetPosition(float * *pX*,
float * *pY*
) [virtual]

Parameters:

pX address of variable to receive the x coord. (in pixels)

pY address of variable to receive the y coord. (in pixels)

Returns:

CUI_RESULTTYPE.

For a CUIPolyString, returns the (x,y) screen coordinates of the upper left corner of the string. For a [CUIFormattedPolyString](#), (x,y) can be the top-left, top-center, or top-right, depending on the alignment.

See also:

[CUIFormattedPolyString::SetAlignmentH\(\)](#)

[GetDims\(\)](#)

[GetRect\(\)](#)

Returns CUIR_OK.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::GetRect([CUIRECT](#) * *pRect*) [virtual]

Parameters:

pRect address of variable to receive the bounding rectangle (in pixels)

Returns:

CUI_RESULTTYPE.

Gets the current bounding box of the CUIPolyString. If you are querying a [CUIFormattedPolyString](#), be aware that based on the alignment of the string, the resulting *pRect->x* and *pRect->y* values might be different from the (x,y) coords. returned by [GetPosition\(\)](#), [GetX\(\)](#), and [GetY\(\)](#).

See also:

[CUIFormattedPolyString::SetAlignmentH\(\)](#)

[GetDims\(\)](#)

[GetPosition\(\)](#)

Returns CUIR_OK.

Used for: Text and UI.

Returns:
const char*.

See also: [SetText\(\)](#)

Returns:
float.

Returns:
float.

Returns:
float.

Returns:
true or false.

Parameters:

- start* (optional) drawing starts from this character index.
- end* (optional) drawing ends at this character index.

Returns:
none.

Parameters:

- clip* Rectangle to clip to.
- start* (optional) drawing starts from this character index.
- end* (optional) drawing ends at this character index.

Returns:
none.

NOTE: precise clipping is performed (i.e., you will see partial letters if the clip rectangle intersects character polygons).

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetCharScreenHeight([uint8](#) *height*) [virtual]

Parameters:

height desired screen height (in pixels) of each character.

Returns:

CUIR_OK if successful.

Sets the screen height of the CUIPolyString's characters.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetCharScreenSize([uint8](#) *height*,
[uint8](#) *width* = 0
) [virtual]

Parameters:

height desired screen height (in pixels) of each character.

width (optional) desired screen width (in pixels) of each character.

Returns:

CUIR_OK if successful.

Sets the screen size of the CUIPolyString's characters.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetCharScreenWidth([uint8](#) *width*) [virtual]

Parameters:

width desired screen width (in pixels) of each character.

Returns:

CUIR_OK if successful.

Sets the screen width of the CUIPolyString's characters.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetColor([uint32](#) *argb*) [virtual]

Parameters:

argb uint32 color of the form 0xAARRGGBB.

Sets a single color for each character of the entire CUIPolyString.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetColors([uint32](#) *argb0*,
[uint32](#) *argb1*,
[uint32](#) *argb2*,
[uint32](#) *argb3*
) [virtual]

Parameters:

argb0 uint32 color of the form 0xAARRGGBB.

argb1 uint32 color of the form 0xAARRGGBB.

argb2 uint32 color of the form 0xAARRGGBB.

argb3 uint32 color of the form 0xAARRGGBB.

span true or false

Sets the vertex colors for each character of the entire CUIPolyString (each character is represented internally as an [LT_POLYGT4](#)).

See also:

struct [LT_POLYGT4](#)

interface [ILTDrawPrim](#)

Vertices are numbered clockwise beginning with vertex 0 at the top left corner.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIPolyString::SetFont([CUIFont](#) * *pFont*) [virtual]

Parameters:

pFont the new font for the CUIPolyString

Returns:

CUI_RESULTTYPE.

Sets a new font for the entire CUIPolyString.

Returns CUIR_OK if successful.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIPolyString::SetPosition( float    x,
                                                    float    y
                                                    ) [virtual]
```

Parameters:*x* the new x coordinate of the CUIPolyString (onscreen, in pixels)*y* the new y coordinate of the CUIPolyString (onscreen, in pixels)**Returns:**

CUI_RESULTTYPE.

Sets new screen coordinates for the CUIPolyString.

Returns CUIR_OK if successful.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIPolyString::SetText( const char * pBuffer ) [virtual]
```

Parameters:*pBuffer* A NULL-terminated string of text (no formatting characters).**Returns:**

CUI_RESULTTYPE.

Sets the text of a CUIPolyString. When function is used, the entire text of the CUIPolyString will be formatted with the last [CUIFont](#) which was applied to the CUIPolyString.Returns CUIR_OK if successful, CUIR_NO_FONT if there is no associated [CUIFont](#), CUIR_OUT_OF_MEMORY if there are memory allocation problems, otherwise returns CUIR_ERROR.**See also:**[ApplyFont\(\)](#)

Used for: Text and UI.

Field Documentation

CUIPolyString_Impl* [CUIPolyString::m_pImpl](#) [protected]

The documentation for this class was generated from the following file:

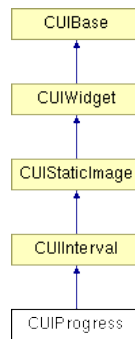
- [cuipolystring.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIProgress Class Reference

Inheritance diagram for CUIProgress:



Public Member Functions

[CUIProgress](#) ([CUIGUID](#) guid)
 virtual [~CUIProgress](#) ()
 virtual [CUI_RESULTTYPE](#) [SetStretchMode](#) ([CUI_STRETCHMODE](#) stretch)
 virtual [CUI_STRETCHMODE](#) [GetStretchMode](#) ()
 virtual [CUI_RESULTTYPE](#) [SetFillMode](#) ([CUI_FILLMODE](#) fill)
 virtual [CUI_FILLMODE](#) [GetFillMode](#) ()

Detailed Description

CUIProgress is a CUI widgets which displays a filling or stretching image over a background to indicate progress.

CUIProgress starts with a default *min* of 0, *max* of 100, and a *current value* of 50.

CUIE_BG CUIE_PROGRESS_INDICATOR

See also:

[ILTWidgetManager](#)

[CUI_ELEMENTTYPE](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIProgress::CUIProgress([CUIGUID](#) guid)

virtual CUIProgress::~~CUIProgress() [virtual]

Member Function Documentation

virtual [CUI_FILLMODE](#) CUIProgress::GetFillMode() [virtual]

Returns:

CUI_FILLMODE.

Returns whether the objects texture is drawn in a stretch or fill mode.

See also:

[SetFillMode\(\)](#)

Used for: Text and UI.

virtual [CUI_STRETCHMODE](#) CUIProgress::GetStretchMode() [virtual]

Returns:

CUI_STRETCH_TRUE or CUI_STRETCH_FALSE.

Returns whether the objects texture is drawn in a stretch or fill mode.

See also:

[SetStretchMode\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIProgress::SetFillMode([CUI_FILLMODE](#) fill) [virtual]

Parameters:

stretch CUI_FILLMODE.

Returns:

CUI_RESULTTYPE.

If *fill* is CUI_FILL_LEFTTORIGHT, the CUIE_PROGRESS_INDICATOR element draws from left to right when the orientation is CUI_ORIENT_HORIZONTAL.

If *fill* is CUI_FILL_RIGHTTOLEFT, the CUIE_PROGRESS_INDICATOR element draws from right to left when the orientation is CUI_ORIENT_HORIZONTAL.

If *fill* is CUI_FILL_TOPTOBOTTOM, the CUIE_PROGRESS_INDICATOR element draws from top to bottom when the orientation is CUI_ORIENT_VERTICAL.

If *fill* is CUI_FILL_BOTTOMTOTOP, the CUIE_PROGRESS_INDICATOR element draws from bottom to top when the orientation is CUI_ORIENT_VERTICAL.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIProgress::SetStretchMode([CUI_STRETCHMODE](#) *stretch*) [virtual]

Parameters:

stretch CUI_STRETCHMODE.

Returns:

CUI_RESULTTYPE.

If *stretch* is CUI_STRETCH_TRUE, the CUIE_PROGRESS_INDICATOR element's texture stretches to fit the width of the indicator, and grows or shrinks based on the object's current value.

If *stretch* is CUI_STRETCH_FALSE, the CUIE_PROGRESS_INDICATOR element draws a partial texture. The amount of the texture drawn is based on the object's current value.

See also:

[SetTexture\(\)](#)

Used for: Text and UI.

The documentation for this class was generated from the following file:

- [cuiprogress.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIRECT Class Reference

Public Member Functions

[CUIRECT](#) ()

[CUIRECT](#) (float ix, float iy, float iwidth, float iheight)

[CUIRECT](#) & [operator=](#) (const [CUIRECT](#) &srcRect)

Data Fields

float [x](#)

float [y](#)

float [width](#)

float [height](#)

Detailed Description

CUI Rectangle struct.

Constructor & Destructor Documentation

CUIRECT::CUIRECT() [inline]

CUIRECT::CUIRECT(float *ix*,
float *iy*,
float *iwidth*,
float *iheight*
) [inline]

Member Function Documentation

[CUIRECT](#)& CUIRECT::operator=(const [CUIRECT](#) & *srcRect*) [inline]

Field Documentation

float [CUIRECT::height](#)
height.

float [CUIRECT::width](#)
width.

float [CUIRECT::x](#)
x coordinate

float [CUIRECT::y](#)
y coordinate.

The documentation for this class was generated from the following file:

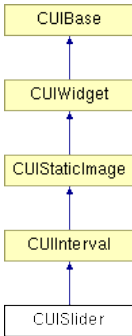
- [cuitypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUISlider Class Reference

Inheritance diagram for CUISlider:



Public Member Functions

[CUISlider](#) ([CUIGUID](#) guid)
virtual [~CUISlider](#) ()
virtual [uint32](#) [GetSliderSizePercent](#) ()
virtual [CUI_RESULTTYPE](#) [SetSliderSizePercent](#) ([uint16](#) percent)

Detailed Description

CUISlider is a CUI widgets that can be used as a scroll-bar, a volume slider, or anything else that is adjustable and has a min and a max value. This widget displays a background and a slider object.

CUISlider starts with a default *min* of 0, *max* of 100, *current value* of 50, and *slider size percent* of 25.

CUIE_BG CUIE_SLIDER

See also:

[ILTWidgetManager](#)

[CUI_ELEMENTTYPE](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUISlider::CUISlider([CUIGUID](#) *guid*)

virtual CUISlider::~CUISlider() [virtual]

Member Function Documentation

virtual [uint32](#) CUISlider::GetSliderSizePercent() [virtual]

Returns:

the size of the slider element.

Returns the size of the slider element as a percentage of the current width of the CUISlider object.

See also:

[SetSliderSizePercent\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUISlider::SetSliderSizePercent([uint16](#) *percent*) [virtual]

Parameters:

percent sets the size of the slider element.

Returns:

CUI_RESULTTYPE.

Sets the size of the slider element as a percentage of the current width of the CUISlider object.

See also:

[GetSliderSizePercent\(\)](#)

Used for: Text and UI.

The documentation for this class was generated from the following file:

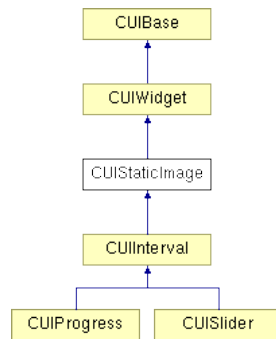
- [cuislidder.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

CUIStaticImage Class Reference

Inheritance diagram for CUIStaticImage:



Public Member Functions

[CUIStaticImage\(\)](#)
[CUIStaticImage\(CUIGUID guid\)](#)
virtual [~CUIStaticImage\(\)](#)

Detailed Description

The CUIStaticImage class is one of the CUI Widgets. This class holds a single static image.

CUIE_BG

See also:
interface [ILTWidgetManager](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIStaticImage::CUIStaticImage()

CUIStaticImage::CUIStaticImage([CUIGUID](#) guid)

virtual CUIStaticImage::~CUIStaticImage() [virtual]

The documentation for this class was generated from the following file:

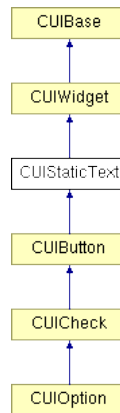
- [cuistaticimage.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIStaticText Class Reference

Inheritance diagram for CUIStaticText:



Public Member Functions

```

    CUIStaticText()
    CUIStaticText(CUIGUID guid)
    virtual ~CUIStaticText()
    virtual const char* GetText()
    virtual CUIFont* GetFont()
    virtual CUI_RESULTTYPE SetText(const char *pBuffer)
    virtual CUI_RESULTTYPE SetFont(CUIFont *pFont)
    virtual CUI_RESULTTYPE SetCharHeight(uint8 height)
    virtual CUI_RESULTTYPE SetWrapWidth(uint16 wrap)
  
```

Detailed Description

CUIStaticText class. This class is uses a [CUIFormattedPolyString](#) to provide a way of displaying formatted text within the widget hierarchy.

See also:

class [CUIPolyString](#)

Used for: Text and UI.

Constructor & Destructor Documentation

CUIStaticText::CUIStaticText()

CUIStaticText::CUIStaticText([CUIGUID](#) *guid*)

virtual CUIStaticText::~CUIStaticText() [virtual]

Member Function Documentation

virtual [CUIFont](#)* CUIStaticText::GetFont() [virtual]

Returns:

[CUIFont](#).

This function returns the [CUIFont](#) associated with a static text widget

Used for: Text and UI.

virtual const char* CUIStaticText::GetText() [virtual]

Returns:

NULL-terminated, immutable string

This function gets the ascii text stored in a static text widget

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIStaticText::SetCharHeight([uint8](#) *height*) [virtual]

Parameters:

height desired screen height of characters.

Returns:

CUIR_OK if successful.

This function sets the height of the text contained in the widget.

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIStaticText::SetFont([CUIFont](#) * *pFont*) [virtual]

Parameters:

pFont [CUIFont](#) to set.

Returns:

CUIR_OK if successful.

This function sets the [CUIFont](#) associated with a static text widget. The font must be set before calling [SetText\(\)](#).

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIStaticText::SetText(const char * *pBuffer*) [virtual]

Parameters:

pBuffer string to set.

Returns:

CUIR_OK if successful.

This function sets the ascii text stored in a static text widget. The font must be set before calling [SetText\(\)](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIStaticText::SetWrapWidth([uint16](#) *wrap*) [virtual]

Parameters:

wrap wrap-width.

Returns:

CUI_RESULTTYPE.

Sets the wrap-width of the widget. This defines where a multi-line CUIStaticText widget will break text.

Returns CUIR_OK if successful.

Used for: Text and UI.

The documentation for this class was generated from the following file:

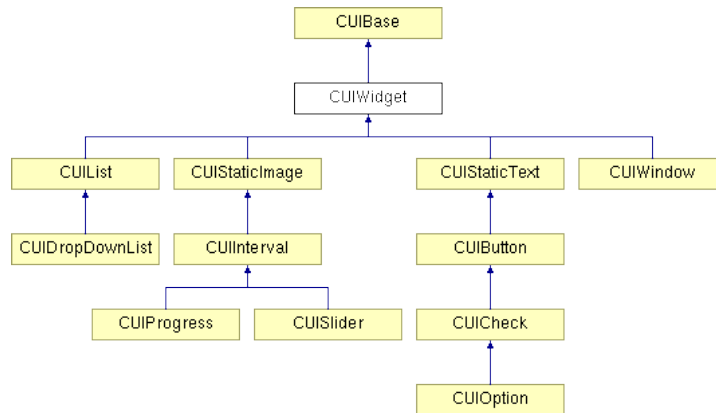
- [cuistatictext.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIWidget Class Reference

Inheritance diagram for CUIWidget:



Public Member Functions

virtual [~CUIWidget](#) ()
 virtual [HTEXTURE](#) [GetTexture](#) ([CUI_ELEMENTTYPE](#) elm)
 virtual [CUI_RESULTTYPE](#) [SetColor](#) ([CUI_ELEMENTTYPE](#) elm, [uint32](#) argb)
 virtual [CUI_RESULTTYPE](#) [SetColors](#) ([CUI_ELEMENTTYPE](#) elm, [uint32](#) argb0, [uint32](#) argb1, [uint32](#) argb2, [uint32](#) argb3)
 virtual [CUI_RESULTTYPE](#) [SetText](#) ([CUI_ELEMENTTYPE](#) elm, [HTEXTURE](#) hTex, bool tile)
 virtual [CUI_RESULTTYPE](#) [SetText](#) ([CUI_ELEMENTTYPE](#) elm, [HTEXTURE](#) hTex, [CUIRECT](#) *pRect)
 virtual [CUI_RESULTTYPE](#) [SetAlignmentH](#) ([CUI_ALIGNMENTTYPE](#) align)
 virtual [CUI_RESULTTYPE](#) [SetAlignmentV](#) ([CUI_ALIGNMENTTYPE](#) align)

Detailed Description

CUIWidget class. A widget cannot be instantiated. It is the base class for other widget types

See also:

interface [ILTWidgetManager](#)

Used for: Text and UI.

Constructor & Destructor Documentation

virtual CUIWidget::~CUIWidget() [virtual]

Member Function Documentation

virtual [HTEXTURE](#) CUIWidget::GetTexture([CUI_ELEMENTTYPE](#) elm) [virtual]

Parameters:

elm CUI Element to query

Returns:

HTEXTURE

This function retrieves the current Texture associated with a CUI_ELEMENTTYPE. CUI_ELEMENTTYPEs are defined in [cuitypes.h](#), and break a widget down into texturable components. For example, a [CUIWindow](#) has the following CUI Elements:

CUIE_BG CUIE_TOP CUIE_BOTTOM CUIE_LEFT CUIE_RIGHT CUIE_TR_CORNER CUIE_TL_CORNER CUIE_BR_CORNER
 CUIE_BL_CORNER

Used for: Text and UI.

Reimplemented from [CUIBase](#).

virtual [CUI_RESULTTYPE](#) CUIWidget::SetAlignmentH([CUI_ALIGNMENTTYPE](#) align) [virtual]

Parameters:

align new alignment.

Returns:

CUIR_OK if successful

Use this function to set the horizontal alignment of a widget (not all widgets support the concept of alignment. Furthermore, of those that do support alignment, not all support every kind of alignment. i.e., setting a CUIStaticImage's alignment to CUI_HALIGN_JUSTIFY makes no sense, but setting a [CUIStaticText](#) widget's alignment to CUI_HALIGN_JUSTIFY will work).

valid horizontal alignments are: CUI_HALIGN_LEFT CUI_HALIGN_CENTER CUI_HALIGN_RIGHT CUI_HALIGN_JUSTIFY

See also:

[CUI_ALIGNMENTTYPE](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIWidget::SetAlignmentV([CUI_ALIGNMENTTYPE](#) *align*) [virtual]

Parameters:

align new alignment.

Returns:

CUIR_OK if successful

Use this function to set the vertical alignment of a widget (not all widgets support the concept of alignment. Furthermore, of those that do support alignment, not all support every kind of alignment).

valid vertical alignments are: CUI_VALIGN_TOP CUI_VALIGN_CENTER CUI_VALIGN_BOTTOM

See also:

[CUI_ALIGNMENTTYPE](#)

Used for: Text and UI.

virtual [CUI_RESULTTYPE](#) CUIWidget::SetColor([CUI_ELEMENTTYPE](#) *elm*,
[uint32](#) *argb*
) [virtual]

Parameters:

elm CUI Element whose color should be set.

argb 32-bit color value of the form 0xAARRGGBB.

Returns:

CUI_RESULTTYPE

Use this function to set the background color of a CUI_ELEMENTTYPE. CUI_ELEMENTTYPEs are defined in [cuitypes.h](#).

Used for: Text and UI.

Implements [CUIBase](#).

virtual [CUI_RESULTTYPE](#) CUIWidget::SetColors([CUI_ELEMENTTYPE](#) *elm*,
[uint32](#) *argb0*,
[uint32](#) *argb1*,
[uint32](#) *argb2*,
[uint32](#) *argb3*
) [virtual]

Parameters:

elm CUI Element to set the color of.

argb0 32-bit color value of the form 0xAARRGGBB.

argb1 32-bit color value of the form 0xAARRGGBB.

argb2 32-bit color value of the form 0xAARRGGBB.

argb3 32-bit color value of the form 0xAARRGGBB.

Returns:

CUI_RESULTTYPE

Use this function to set separate vertex colors for a CUI_ELEMENTTYPE. CUI_ELEMENTTYPEs are defined in [cuitypes.h](#).

Used for: Text and UI.

Implements [CUIBase](#).

virtual [CUI_RESULTTYPE](#) CUIWidget::SetTexture([CUI_ELEMENTTYPE](#) *elm*,
[HTEXTURE](#) *hTex*,
[CUIRECT](#) * *pRect*
) [virtual]

Parameters:

elm CUI Element to set the texture of.

hTex Texture to use.

pRect address of [CUIRECT](#) containing uv coords. of within hTex

Returns:

CUI_RESULTTYPE

Use this function to set the texture of a CUI Element. CUI Elements are defined in [cuitypes.h](#). Use the [CUIRECT](#) to specify a portion of the

HTEXTURE to use for the element. Furthermore, the height and width of the element will be changed to reflect the rectangle size.

Used for: Text and UI.

```
virtual CUI\_RESULTTYPE CUIWidget::SetTexture( CUI\_ELEMENTTYPE elm,  
                                             HTEXTURE hTex,  
                                             bool tile  
                                             ) [virtual]
```

Parameters:

- elm* CUI_ELEMENTTYPE to set the texture of.
- hTex* Texture to use.
- tile* if true, the texture tiles, if false, it stretches to fir the element

Returns:

CUI_RESULTTYPE

Use this function to set the texture of a CUI_ELEMENTTYPE. CUI Elements are defined in [cuiatypes.h](#). Note that the amount of times a texture will tile is limited by the system architecture, and further note that not all CUI_ELEMENTTYPEs support tiled textures.

Used for: Text and UI.

Implements [CUIBase](#).

The documentation for this class was generated from the following file:

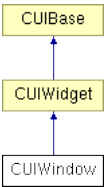
- [cuiwidget.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

CUIWindow Class Reference

Inheritance diagram for CUIWindow:



Public Member Functions

```
CUIWindow (CUIGUID guid, HTEXTURE skin=NULL, char *table=NULL)  
virtual ~CUIWindow ()
```

Detailed Description

CUIWindow class.

See also:

interface [ILTWidgetManager](#)

Used for: Text and UI.

Constructor & Destructor Documentation

```
CUIWindow::CUIWindow( CUIGUID guid,  
                      HTEXTURE skin = NULL,  
                      char * table = NULL  
                      )
```

```
virtual CUIWindow::~CUIWindow( ) [virtual]
```

The documentation for this class was generated from the following file:

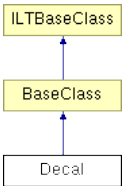
- [cuiwindow.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Decal Class Reference

Inheritance diagram for Decal:



Detailed Description

Decal class.

The documentation for this class was generated from the following file:

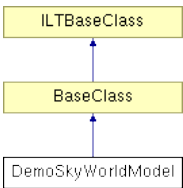
- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

DemoSkyWorldModel Class Reference

Inheritance diagram for DemoSkyWorldModel:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

[LTVector](#) [m_SkyDims](#)
float [m_InnerPercentX](#)
float [m_InnerPercentY](#)
float [m_InnerPercentZ](#)
[int32](#) [m_Index](#)

Detailed Description

DemoSkyWorldModel class.

This is a **WorldModel** that adds itself to the sky object list and has properties that level designers can edit to set the sky dimensions. If the sky box is set to zero, then it won't set it. (This is so you can have multiple **DemoSkyWorldModels** in the world, and only one sets the sky box). The **DemoSkyWorldModel** uses **InnerPercentX**, Y, and Z as a percentage of the **SkyDims** box for the inner box. Each sky world model must have a (unique) index from 0-30. Ones with lower indices get drawn first.

Member Function Documentation

```
virtual uint32 DemoSkyWorldModel::EngineMessageFn( uint32 messageID,
                                                void * pData,
                                                float  lData
                                                ) [virtual]
```

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

[int32](#) [DemoSkyWorldModel::m_Index](#)

float [DemoSkyWorldModel::m_InnerPercentX](#)

float [DemoSkyWorldModel::m_InnerPercentY](#)

float [DemoSkyWorldModel::m_InnerPercentZ](#)

[LTVector](#) [DemoSkyWorldModel::m_SkyDims](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

DeviceBinding Struct Reference

Data Fields

char [strDeviceName](#) [100]
 name of device for this trigger

```

char strTriggerName [100]
    name of this trigger (device object name)
char strRealName [100]
    real name of this trigger ("##21" instead of "Space bar")

```

```

uint32 m\_nObjectId

```

```

    Object Id.

```

```

float nScale

```

```

    scale to multiply input by

```

```

float nRangeScaleMin

```

```

    min for range scale

```

```

float nRangeScaleMax

```

```

    max for range scale

```

```

float nRangeScalePreCenterOffset

```

```

    offset of cent value for input data

```

```

GameAction * pActionHead

```

```

    list of actions bound to this trigger

```

```

DeviceBinding * pNext

```

```

    next in list

```

Field Documentation

[uint32 DeviceBinding::m_nObjectId](#)

Object Id.

float [DeviceBinding::nRangeScaleMax](#)

max for range scale

float [DeviceBinding::nRangeScaleMin](#)

min for range scale

float [DeviceBinding::nRangeScalePreCenterOffset](#)

offset of cent value for input data

float [DeviceBinding::nScale](#)

scale to multiply input by

[GameAction](#)* [DeviceBinding::pActionHead](#)

list of actions bound to this trigger

[DeviceBinding](#)* [DeviceBinding::pNext](#)

next in list

char [DeviceBinding::strDeviceName](#)[100]

name of device for this trigger

char [DeviceBinding::strRealName](#)[100]

real name of this trigger ("##21" instead of "Space bar")

char [DeviceBinding::strTriggerName](#)[100]

name of this trigger (device object name)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

DeviceInput Struct Reference

Data Fields

[uint32 m_DeviceType](#)
type of input device (DEVICETYPE_ defines above)
char [m_DeviceName](#) [128]
name of input device

[uint32 m_ControlType](#)
type of control that changed (CONTROLTYPE_ defines above)
char [m_ControlName](#) [64]
name of control that changed

[uint16 m_ControlCode](#)
object instance code for the control that changed (DIK_ code for control)

[uint32 m_nObjectId](#)
Device object id. Used with [DeviceObject::m_nObjectId](#).

[uint32 m_InputValue](#)
input value for the control

Field Documentation

[uint16 DeviceInput::m_ControlCode](#)
object instance code for the control that changed (DIK_ code for control)

char [DeviceInput::m_ControlName](#) [64]
name of control that changed

[uint32 DeviceInput::m_ControlType](#)
type of control that changed (CONTROLTYPE_ defines above)

char [DeviceInput::m_DeviceName](#) [128]
name of input device

[uint32 DeviceInput::m_DeviceType](#)
type of input device (DEVICETYPE_ defines above)

[uint32 DeviceInput::m_InputValue](#)
input value for the control

[uint32 DeviceInput::m_nObjectId](#)
Device object id. Used with [DeviceObject::m_nObjectId](#).

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

DeviceObject Struct Reference

Data Fields

[uint32 m_DeviceType](#)
type of input device (DEVICETYPE_ defines above)

char [m_DeviceName](#) [128]
name of input device ("Keyboard", "Mouse", etc.)

[uint32 m_ObjectType](#)
type of object (CONTROLTYPE_ defines above)

char [m_ObjectName](#) [64]
name of object ("x-axis", "button 0", etc.)

[uint32 m_nObjectId](#)
Object Id.

float [m_RangeLow](#)
low end of range for this object

float [m_RangeHigh](#)
high end of range for this object

[DeviceObject](#) * [m_pNext](#)

Detailed Description

Device Object for listing objects on a particular device

Field Documentation

char [DeviceObject::m_DeviceName](#)[128]
name of input device ("Keyboard", "Mouse", etc.)

[uint32 DeviceObject::m_DeviceType](#)
type of input device (DEVICETYPE_ defines above)

[uint32 DeviceObject::m_nObjectId](#)
Object Id.

char [DeviceObject::m_ObjectName](#)[64]
name of object ("x-axis", "button 0", etc.)

[uint32 DeviceObject::m_ObjectType](#)
type of object (CONTROLTYPE_ defines above)

[DeviceObject*](#) [DeviceObject::m_pNext](#)

float [DeviceObject::m_RangeHigh](#)
high end of range for this object

float [DeviceObject::m_RangeLow](#)
low end of range for this object

The documentation for this struct was generated from the following file:

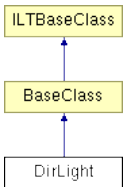
- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

DirLight Class Reference

Inheritance diagram for DirLight:



Detailed Description

DirLight class.

The documentation for this class was generated from the following file:

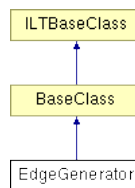
- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

EdgeGenerator Class Reference

Inheritance diagram for EdgeGenerator:



Detailed Description

EdgeGenerator class.

The documentation for this class was generated from the following file:

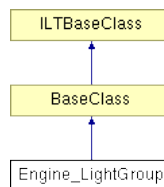
- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Engine_LightGroup Class Reference

Inheritance diagram for Engine_LightGroup:



Detailed Description

Engine_LightGroup class. Defines the grouping for a set of world lights which can be turned on and off. Note that this is engine-side and is intended to be inherited from on the game side for controlling purposes.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

EulerAngles_t Struct Reference

Public Member Functions

[EulerAngles_t](#) (float nx=0.0f, float ny=0.0f, float nz=0.0f, float nw=0.0f)
void [Init](#) (float nx, float ny, float nz, float nw)

Data Fields

float [x](#)
float [y](#)
float [z](#)
float [w](#)

Constructor & Destructor Documentation

```
EulerAngles_t::EulerAngles_t( float nx = 0.0f,
                             float ny = 0.0f,
                             float nz = 0.0f,
                             float nw = 0.0f
                             ) [inline]
```

Member Function Documentation

```
void EulerAngles_t::Init( float nx,
                         float ny,
                         float nz,
                         float nw
                         ) [inline]
```

Field Documentation

float [EulerAngles_t::w](#)

float [EulerAngles_t::x](#)

float [EulerAngles_t::y](#)

float [EulerAngles_t::z](#)

The documentation for this struct was generated from the following file:

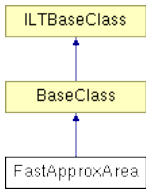
- [LTEulerAngles.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006. All Rights Reserved.

API Reference

FastApproxArea Class Reference

Inheritance diagram for FastApproxArea:



Detailed Description

FastApproxArea class. This defines an area where only fast approximation vising will occur. The area is bounded by hullmakers and portals.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

FileEntry Struct Reference

Data Fields

int [m_Type](#)
char * [m_pBaseFilename](#)
Directory or file?
char * [m_pFullFilename](#)
pic1.pcx
[FileEntry](#) * [m_pNext](#)
interface/bitmaps/pic1.pcx

Field Documentation

char* [FileEntry::m_pBaseFilename](#)
Directory or file?

char* [FileEntry::m_pFullFilename](#)
pic1.pcx

[FileEntry](#)* [FileEntry::m_pNext](#)
interface/bitmaps/pic1.pcx

int [FileEntry::m_Type](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

FourFloatColor Struct Reference

Public Member Functions

[FourFloatColor](#) ()

[FourFloatColor](#) (float nr, float ng, float nb, float na)

Data Fields

float [r](#)

float [g](#)

float [b](#)

float [a](#)

Detailed Description

FourFloatColor

Constructor & Destructor Documentation

FourFloatColor::FourFloatColor() [inline]

```
FourFloatColor::FourFloatColor( float  nr,
                                float  ng,
                                float  nb,
                                float  na
                                ) [inline]
```

Field Documentation

float [FourFloatColor::a](#)

float [FourFloatColor::b](#)

float [FourFloatColor::g](#)

float [FourFloatColor::r](#)

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

FourFloatVector Struct Reference

Public Member Functions

[FourFloatVector](#) ()

[FourFloatVector](#) (float nx, float ny, float nz, float nw)

Data Fields

float [x](#)

float [y](#)

float [z](#)

float [w](#)

Detailed Description

FourFloatVector

Constructor & Destructor Documentation

FourFloatVector::FourFloatVector() [inline]

FourFloatVector::FourFloatVector(float *nx*,
float *ny*,
float *nz*,
float *nw*
) [inline]

Field Documentation

float [FourFloatVector::w](#)

float [FourFloatVector::x](#)

float [FourFloatVector::y](#)

float [FourFloatVector::z](#)

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

FrameLocator Class Reference

Public Member Functions

void [Clear](#) ()

Data Fields

[uint16 m_iAnim](#)

[uint16 m_iFrame](#)

[uint32 m_Time](#)

Detailed Description

All the info on where we are in the animation.

Member Function Documentation

void FrameLocator::Clear() [inline]

Field Documentation

[uint16 FrameLocator::m_iAnim](#)

[uint16 FrameLocator::m_iFrame](#)

[uint32 FrameLocator::m_Time](#)

The documentation for this class was generated from the following file:

- [ltanimtracker.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

GameAction Struct Reference

Data Fields

[int32 nActionCode](#)
action number from config file
char [strActionName](#) [30]
name of action from config file
float [nRangeLow](#)
low range if used (zero if not used)
float [nRangeHigh](#)
high range if used (zero if not used)

[GameAction](#) * [pNext](#)
next in list

Field Documentation

[int32 GameAction::nActionCode](#)
action number from config file

float [GameAction::nRangeHigh](#)
high range if used (zero if not used)

float [GameAction::nRangeLow](#)
low range if used (zero if not used)

[GameAction](#)* [GameAction::pNext](#)
next in list

char [GameAction::strActionName](#)[30]
name of action from config file

The documentation for this struct was generated from the following file:

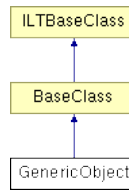
- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

GenericObject Class Reference

Inheritance diagram for GenericObject:



Public Member Functions

virtual [uint32 EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

float [m_LightRadius](#)
 float [m_ColorR](#)
 float [m_ColorG](#)
 float [m_ColorB](#)
 float [m_ColorA](#)

Detailed Description

This has functionality to be almost any type of engine object.

Member Function Documentation

virtual [uint32](#) GenericObject::EngineMessageFn([uint32](#) messageID,
 void * pData,
 float lData
) [virtual]

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

float [GenericObject::m_ColorA](#)

float [GenericObject::m_ColorB](#)

float [GenericObject::m_ColorG](#)

float [GenericObject::m_ColorR](#)

float [GenericObject::m_LightRadius](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

GenericProp Struct Reference

Public Member Functions

[GenericProp](#) ()
[GenericProp](#) (const [GenericProp](#) &cOther)
[GenericProp](#) & operator= (const [GenericProp](#) &cOther)
[GenericProp](#) (const [LTVector](#) &vValue, [uint32](#) nType)
[GenericProp](#) (const char *pValue, [uint32](#) nType)
[GenericProp](#) (float fValue, [uint32](#) nType)
[GenericProp](#) ([int32](#) nValue, [uint32](#) nType=LT_PT_LONGINT)
[GenericProp](#) (const [LTRotation](#) &cRotation, [uint32](#) nType)
[GenericProp](#) (bool bValue, [uint32](#) nType)
[GenericProp](#) (const [LTVector](#) &vValue)
[GenericProp](#) (const char *pValue)
[GenericProp](#) (float fValue)
[GenericProp](#) ([int32](#) nValue)
[GenericProp](#) (const [LTRotation](#) &cRotation)
[GenericProp](#) (bool bValue)

Data Fields

[uint32](#) m_Type
[LTVector](#) m_Vec
[LTVector](#) m_Color
char m_String [256+1]
[LTRotation](#) m_Rotation
[int32](#) m_Long
float m_Float
bool m_Bool

Constructor & Destructor Documentation

[GenericProp::GenericProp](#)() [inline]

[GenericProp::GenericProp](#)(const [GenericProp](#) & cOther) [inline]

[GenericProp::GenericProp](#)(const [LTVector](#) & vValue,
 [uint32](#) nType
) [inline]

[GenericProp::GenericProp](#)(const char * pValue,
 [uint32](#) nType
) [inline]

[GenericProp::GenericProp](#)(float fValue,
 [uint32](#) nType
) [inline]

```
GenericProp::GenericProp( int32 nValue,  
                          uint32 nType = LT PT LONGINT  
                        ) [inline]
```

```
GenericProp::GenericProp( const LTRotation & cRotation,  
                          uint32 nType  
                        ) [inline]
```

```
GenericProp::GenericProp( bool bValue,  
                          uint32 nType  
                        ) [inline]
```

```
GenericProp::GenericProp( const LTVector & vValue ) [inline]
```

```
GenericProp::GenericProp( const char * pValue ) [inline]
```

```
GenericProp::GenericProp( float fValue ) [inline]
```

```
GenericProp::GenericProp( int32 nValue ) [inline]
```

```
GenericProp::GenericProp( const LTRotation & cRotation ) [inline]
```

```
GenericProp::GenericProp( bool bValue ) [inline]
```

Member Function Documentation

[GenericProp](#)& GenericProp::operator=(const [GenericProp](#) & cOther) [inline]

Field Documentation

bool [GenericProp::m_Bool](#)

[LTVector](#) [GenericProp::m_Color](#)

float [GenericProp::m_Float](#)

[int32](#) [GenericProp::m_Long](#)

[LTRotation](#) [GenericProp::m_Rotation](#)

char [GenericProp::m_String](#)[256+1]

[uint32](#) [GenericProp::m_Type](#)

[LTVector](#) [GenericProp::m_Vec](#)

The documentation for this struct was generated from the following file:

- [ltproperty.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

GenericPropList Class Reference

Public Member Functions

[GenericPropList](#) ()
[GenericPropList](#) (const [GenericPropList](#) &cOther)
[~GenericPropList](#) ()
[GenericPropList](#) & [operator=](#) (const [GenericPropList](#) &cOther)
[LTVector](#) [GetPropVector](#) (const char *pName, const [LTVector](#) &vDefault) const
[LTVector](#) [GetPropColor](#) (const char *pName, const [LTVector](#) &vDefault) const
 const char * [GetPropString](#) (const char *pName, const char *pDefault) const
[LTRotation](#) [GetPropRotation](#) (const char *pName, const [LTRotation](#) &rDefault) const
[int32](#) [GetPropLongInt](#) (const char *pName, [int32](#) nDefault) const
[int32](#) [GetPropFlags](#) (const char *pName, [int32](#) nDefault) const
 float [GetPropReal](#) (const char *pName, float fDefault) const
 bool [GetPropBool](#) (const char *pName, bool bDefault) const
 void [Reset](#) ()
[uint32](#) [GetNumProps](#) () const
[uint32](#) [GetMaxProps](#) () const
 bool [ReserveProps](#) ([uint32](#) nMax, bool bPreserveProps)
 bool [AddProp](#) (const char *pszName, const [GenericProp](#) &Prop)
 const char * [GetPropName](#) ([uint32](#) nProp) const
 const [GenericProp](#) * [GetProp](#) ([uint32](#) nProp) const
 const [GenericProp](#) * [GetProp](#) (const char *pszName) const

Protected Member Functions

void [FreeProps](#) ()

Protected Attributes

[uint32](#) [m_nNumProps](#)
[uint32](#) [m_nMaxProps](#)
[SPropEntry](#) * [m_pProps](#)
[SPropEntry](#) ** [m_pPropIndices](#)

Data Structures

struct [SPropEntry](#)

Constructor & Destructor Documentation

[GenericPropList::GenericPropList](#)() [inline]

[GenericPropList::GenericPropList](#)(const [GenericPropList](#) & *cOther*) [inline]

[GenericPropList::~~GenericPropList](#)() [inline]

Member Function Documentation

bool [GenericPropList::AddProp](#)(const char * *pszName*,
 const [GenericProp](#) & *Prop*
) [inline]

void [GenericPropList::FreeProps](#)() [inline, protected]

[uint32](#) GenericPropList::GetMaxProps() const [inline]

[uint32](#) GenericPropList::GetNumProps() const [inline]

const [GenericProp](#)* GenericPropList::GetProp(const char * *pszName*) const [inline]
Get the property associated with pName. Returns LTNUL if that property was not found.

const [GenericProp](#)* GenericPropList::GetProp([uint32](#) *nProp*) const [inline]

bool GenericPropList::GetPropBool(const char * *pName*,
bool *bDefault*
) const [inline]
Get the LT_PT_BOOL property associated with pName. Returns bDefault if that property was not found.

[LTVector](#) GenericPropList::GetPropColor(const char * *pName*,
const [LTVector](#) & *vDefault*
) const [inline]
Get the LT_PT_COLOR property associated with pName. Returns vDefault if that property was not found.

[int32](#) GenericPropList::GetPropFlags(const char * *pName*,
[int32](#) *nDefault*
) const [inline]
Get the LT_PT_FLAGS property associated with pName. Returns nDefault if that property was not found.

[int32](#) GenericPropList::GetPropLongInt(const char * *pName*,
[int32](#) *nDefault*
) const [inline]
Get the LT_PT_LONGINT property associated with pName. Returns nDefault if that property was not found.

const char* GenericPropList::GetPropName([uint32](#) *nProp*) const [inline]

float GenericPropList::GetPropReal(const char * *pName*,
float *fDefault*
) const [inline]
Get the LT_PT_REAL property associated with pName. Returns fDefault if that property was not found.

[LTRotation](#) GenericPropList::GetPropRotation(const char * *pName*,
const [LTRotation](#) & *rDefault*
) const [inline]
Get the LT_PT_ROTATION property associated with pName. Returns rDefault if that property was not found.

const char* GenericPropList::GetPropString(const char * *pName*,
const char * *pDefault*
) const [inline]
Get the LT_PT_STRING property associated with pName. Returns pDefault if that property was not found.

[LTVector](#) GenericPropList::GetPropVector(const char * *pName*,
const [LTVector](#) & *vDefault*
) const [inline]
Get the LT_PT_VECTOR property associated with pName. Returns vDefault if that property was not found.

[GenericPropList](#)& GenericPropList::operator=(const [GenericPropList](#) & *cOther*) [inline]

bool GenericPropList::ReserveProps([uint32](#) *nMax*,
bool *bPreserveProps*
) [inline]

void GenericPropList::Reset() [inline]

Field Documentation

[uint32 GenericPropList::m_nMaxProps](#) [protected]

[uint32 GenericPropList::m_nNumProps](#) [protected]

[SPropEntry** GenericPropList::m_pPropIndices](#) [protected]

[SPropEntry* GenericPropList::m_pProps](#) [protected]

The documentation for this class was generated from the following file:

- [ltproperty.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

[GenericPropList::SPropEntry](#)

GenericPropList::SPropEntry Struct Reference

Data Fields

char [m_pszName](#) [256]
[GenericProp m_Prop](#)

Field Documentation

[GenericProp GenericPropList::SPropEntry::m_Prop](#)

char [GenericPropList::SPropEntry::m_pszName](#)[256]

The documentation for this struct was generated from the following file:

- [ltproperty.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HATTACHMENT_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HATTACHMENT_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HCLASS_t Struct Reference

Data Fields

int [__nothing](#)

Detailed Description

Handle definitions. It defines them in this weird way so that you can't accidentally cast between them.

Field Documentation

int [HCLASS t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HCLIENT_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HCLIENT t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HCLIENTREF_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HCLIENTREF_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [iltserver.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HCONSOLEVAR_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HCONSOLEVAR_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HLTFONT_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HLTFONT_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HLTLINE_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HLTLINE_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [iltclient.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HLTSOUND_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HLTSOUND_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HNETSERVICE_t Struct Reference

Data Fields

int [__nothing](#)

Field Documentation

int [HNETSERVICE_t::__nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

HPOLY Struct Reference

Public Member Functions

[HPOLY](#) ([uint32](#) nWorldIndex, [uint32](#) nPolyIndex)
[HPOLY](#) ()
[HPOLY](#) & [operator=](#) (const [HPOLY](#) &hOther)
 bool [operator==](#) (const [HPOLY](#) &hOther)
 bool [operator!=](#) (const [HPOLY](#) &hOther)

Data Fields

[uint32 m_nPolyIndex](#)
[uint32 m_nWorldIndex](#)

Detailed Description

Poly reference.

Constructor & Destructor Documentation

HPOLY::HPOLY([uint32](#) nWorldIndex,
 [uint32](#) nPolyIndex
) [inline]

HPOLY::HPOLY() [inline]

Member Function Documentation

bool HPOLY::operator!=(const [HPOLY](#) & hOther) [inline]

[HPOLY](#)& HPOLY::operator=(const [HPOLY](#) & hOther) [inline]

bool HPOLY::operator==(const [HPOLY](#) & hOther) [inline]

Field Documentation

[uint32 HPOLY::m_nPolyIndex](#)

[uint32 HPOLY::m_nWorldIndex](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

HSTRING_t Struct Reference

Data Fields

int [_nothing](#)

Field Documentation

int [HSTRING_t::_nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

HSURFACE_t Struct Reference

Data Fields

int [_nothing](#)

Field Documentation

int [HSURFACE_t::_nothing](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IAggregate Class Reference

Public Member Functions

[IAggregate](#) ()
virtual [~IAggregate](#) ()
virtual [uint32](#) [EngineMessageFn](#) ([ILTBaseClass](#) *pObject, [uint32](#) messageID, void *pData, [LTFLOAT](#) lData)
virtual [uint32](#) [ObjectMessageFn](#) ([ILTBaseClass](#) *pObject, [HOBJECT](#) hSender, [ILTMessage_Read](#) *pMsg)

Data Fields

[IAggregate](#) * [m_pNextAggregate](#)

Detailed Description

This header file defines the base aggregate structure. Every aggregate must have a link to its next one and a pointer to certain functions that an aggregate must supply. When implementing an aggregate, you *must* derive from this! (In C, just put it as the first member in your structure.)

Constructor & Destructor Documentation

`IAggregate::IAggregate( )` [inline]

`virtual IAggregate::~IAggregate( )` [inline, virtual]

Member Function Documentation

`virtual uint32 IAggregate::EngineMessageFn(ILTBaseClass * pObject,
 uint32 messageID,
 void * pData,
 LTFLOAT lData
)` [inline, virtual]

`virtual uint32 IAggregate::ObjectMessageFn(ILTBaseClass * pObject,
 HOBJECT hSender,
 ILTMessage_Read * pMsg
)` [inline, virtual]

Field Documentation

[IAggregate](#)* [IAggregate::m_pNextAggregate](#)

It is very important that these data members are *exactly* the same as the C version of this class (`Aggregate_t`).

The documentation for this class was generated from the following file:

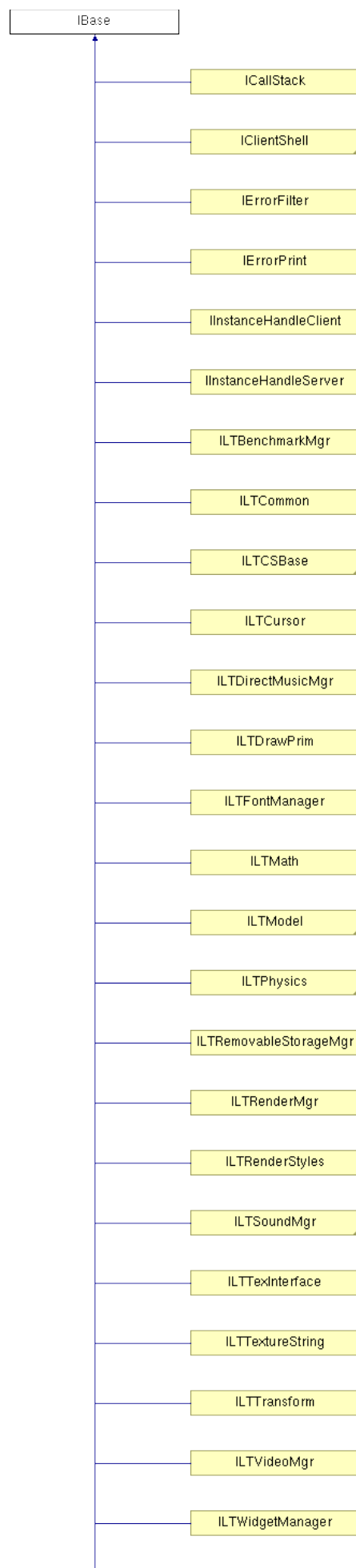
- [iaggregate.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IBase Class Reference

Inheritance diagram for IBase:



Public Member Functions

virtual const char * [InterfaceImplementation](#) ()=0

Detailed Description

IBase is the root class of all interfaces managed by the engine.

The members of the class are for internal use only. They are declared and defined for IBase derived class using the other Interface Manager macros.

Member Function Documentation

virtual const char* IBase::_InterfaceImplementation() [pure virtual]

The documentation for this class was generated from the following file:

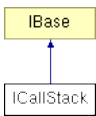
- [Itmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ICallStack Class Reference

Inheritance diagram for ICallStack:



Public Types

enum { [_ICallStack_VERSION_](#) = 0 }

Public Member Functions

virtual void [CallStackPush](#) ([CCallStackTracker](#) *tracker)=0
virtual void [CallStackPop](#) ([CCallStackTracker](#) *tracker)=0
virtual void [CallStackConfirmTop](#) ([CCallStackTracker](#) *tracker)=0
virtual void [CallStackPrint](#) ([uint32](#) color, const char *category)=0

Member Enumeration Documentation

anonymous enum

Enumeration values:

[_ICallStack_VERSION_](#)

Member Function Documentation

virtual void ICallStack::CallStackConfirmTop([CCallStackTracker](#) * tracker) [pure virtual]

virtual void ICallStack::CallStackPop([CCallStackTracker](#) * tracker) [pure virtual]

virtual void ICallStack::CallStackPrint([uint32](#) color,
const char * category
) [pure virtual]

virtual void ICallStack::CallStackPush([CCallStackTracker](#) * *tracker*) [pure virtual]

The documentation for this class was generated from the following file:

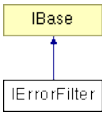
- [Iterror.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IErrorFilter Class Reference

Inheritance diagram for IErrorFilter:



Public Types

enum { [IErrorFilter_VERSION](#) = 0 }

Public Member Functions

virtual bool [CheckCategory](#)(const char * *category*)=0

Member Enumeration Documentation

anonymous enum

Enumeration values:

IErrorFilter_VERSION

Member Function Documentation

virtual bool IErrorFilter::CheckCategory(const char * *category*) [pure virtual]

The documentation for this class was generated from the following file:

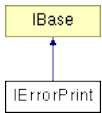
- [Iterror.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IErrorPrint Class Reference

Inheritance diagram for IErrorPrint:



Public Types

enum { [IErrorPrint_VERSION](#) = 0 }

Public Member Functions

virtual void [PrintError](#) ([uint32](#) color, const char *category, const char *message)=0

virtual void [PrintError](#) ([uint32](#) color, const char *category, const char *format, va_list args)=0

Member Enumeration Documentation

anonymous enum

Enumeration values:

[_IErrorPrint_VERSION_](#)

Member Function Documentation

virtual void IErrorPrint::PrintError([uint32](#) color,
const char * category,
const char * format,
va_list args
) [pure virtual]

virtual void IErrorPrint::PrintError([uint32](#) color,
const char * category,
const char * message
) [pure virtual]

The documentation for this class was generated from the following file:

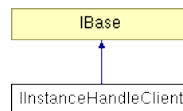
- [Ierror.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IInstanceHandleClient Class Reference

Inheritance diagram for IInstanceHandleClient:



Public Types

enum { [_IInstanceHandleClient_VERSION_](#) = 4 }

Public Member Functions

virtual void [SetInstanceHandle](#) (void *handle)=0

virtual void * [GetInstanceHandle](#) ()=0

Detailed Description

Interface class that manages the client shell DLL's windows instance handle.

Member Enumeration Documentation

anonymous enum

Enumeration values:

[IInstanceHandleClient_VERSION](#)

Member Function Documentation

virtual void* IInstanceHandleClient::GetInstanceHandle() [pure virtual]

Call this function to retrieve the instance handle that was given by the engine.

virtual void IInstanceHandleClient::SetInstanceHandle(void * *handle*) [pure virtual]

Called by the engine if the interface is defined. If the client shell is in a windows DLL, the instance handle is passed in. On other platforms, or if the client shell is not in a DLL, NULL is passed in.

The documentation for this class was generated from the following file:

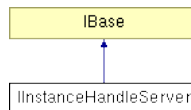
- [iclientshell.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IInstanceHandleServer Class Reference

Inheritance diagram for IInstanceHandleServer:



Public Types

enum { [_IInstanceHandleServer_VERSION](#) = 1 }

Public Member Functions

virtual void [SetInstanceHandle](#) (void *handle)=0
virtual void * [GetInstanceHandle](#) ()=0

Detailed Description

Interface class that manages the server shell DLL's windows instance handle.

Member Enumeration Documentation

anonymous enum

Enumeration values:

[_IInstanceHandleServer_VERSION](#)

Member Function Documentation

virtual void* IInstanceHandleServer::GetInstanceHandle() [pure virtual]

Call this function to retrieve the instance handle that was given by the engine.

virtual void IInstanceHandleServer::SetInstanceHandle(void * *handle*) [pure virtual]

Called by the engine if the interface is defined. If the server shell is in a windows DLL, the instance handle is passed in. On other platforms, or if the server shell is not in a DLL, NULL is passed in.

The documentation for this class was generated from the following file:

- [iservershell.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

InitSoundInfo Struct Reference

Public Member Functions

void [Init](#) ()

Data Fields

char [m_sz3DProvider](#) [_MAX_PATH+1]
 uint16 [m_nNumSWVoices](#)
 uint16 [m_nNum3DVoices](#)
 unsigned short [m_nSampleRate](#)
 unsigned short [m_nBitsPerSample](#)
 unsigned long [m_dwFlags](#)
 unsigned long [m_dwResults](#)
 unsigned short [m_nVolume](#)
 float [m_fDistanceFactor](#)
 float [m_fDopplerFactor](#)

Member Function Documentation

void InitSoundInfo::Init() [inline]

Initialize the **InitSoundInfo** structure to the default values.

Used for: Audio.

Field Documentation

unsigned long [InitSoundInfo::m_dwFlags](#)

A combination of any of the **INITSOUNDINFOFLAG_**xxx flags.

unsigned long [InitSoundInfo::m_dwResults](#)

The sound engine will fill in this parameter inside the [ILTClientSoundMgr::InitSound\(\)](#) call. It can be a combination of any of the **INITSOUNDINFORESULTS_**xxx values.

float [InitSoundInfo::m_fDistanceFactor](#)

This specifies a conversion factor for the sound engine to understand what each game unit represents in meters. Pass a number that is units of meters/game_unit. For instance, if 1 game unit equals 0.5 meters, then fill this parameter with 0.5/1.0 or just 0.5.

float [InitSoundInfo::m_fDopplerFactor](#)

This specifies the initial Doppler factor to use Ranges from 0 (no Doppler effect) to 10 (10 times real-world Doppler)

unsigned short [InitSoundInfo::m_nBitsPerSample](#)

The output bit depth for the primary sound buffer. This value can be 8 or 16.

uint16 [InitSoundInfo::m_nNum3DVoices](#)

The maximum number of 3D provider voices that will be rendered at one time. A 3D provider voice is a 3D sound or a sound that has certain enhancements that are handled by a 3D provider. A good example of an enhancement is hardware reverb. If this number is exceeded, the new voices will be played as non-3D provider voices, which means that the **m_nNumSWVoices** must also be checked.

uint16 [InitSoundInfo::m_nNumSWVoices](#)

The maximum number of non-3D provider voices that will be rendered at one time. If the number of active sounds exceeds this number, the extra sounds will still be tracked. If a rendered sounds stops, then any waiting sounds will begin rendering from the correct offset in time within the sound.

unsigned short [InitSoundInfo::m_nSampleRate](#)

The output sample rate (in hertz) for the primary sound buffer. This value can be 8000, 11025, 22050 or 44100.

unsigned short [InitSoundInfo::m_nVolume](#)

Initial output volume. This can be a number between 0 and 100.

char [InitSoundInfo::m_sz3DProvider](#)[_MAX_PATH+1]

A string containing the name of the 3D provider. The sound engine will render all 3D sounds through this 3D provider, if it is available. To have no 3D provider, set this parameter to an empty string. If no 3D provider is available for a sound or none is specified, then 3D sounds will be simulated in software with volume fading and panning. The list of providers available can be obtained by calling **ILTClientSoundMgr::GetSound3DproviderLists()**.

The documentation for this struct was generated from the following file:

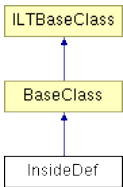
- [iltsoundmgr.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

InsideDef Class Reference

Inheritance diagram for InsideDef:



Detailed Description

InsideDef. These are used by the preprocessor to define 'inside' areas.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

IntersectInfo Struct Reference

Public Member Functions

[IntersectInfo](#) ()

Data Fields

[LTVector](#) [m_Point](#)
[LTPlane](#) [m_Plane](#)
[HOBJECT](#) [m_hObject](#)
[HPOLY](#) [m_hPoly](#)
[uint32](#) [m_SurfaceFlags](#)
[HMODELNODE](#) [m_hNode](#)

Constructor & Destructor Documentation

`IntersectInfo::IntersectInfo( ) [inline]`

Field Documentation

[HMODELNODE](#) [IntersectInfo::m_hNode](#)

Model node hit by the intersection of ray to OBB. Value is INVALID_MODEL_NODE if it didn't hit one.

[HOBJECT](#) [IntersectInfo::m_hObject](#)

Object it hit.

[HPOLY IntersectInfo::m_hPoly](#)

The polygon it hit (if it's a world poly). Value is INVALID_HPOLY if it didn't hit one.

[LTPlane IntersectInfo::m_Plane](#)

Plane of intersection.

[LTVector IntersectInfo::m_Point](#)

Point of intersection.

[uint32 IntersectInfo::m_SurfaceFlags](#)

Surface flags of what it hit (these aren't object flags, and are only set if it hit the world or a WorldModel).

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

IntersectQuery Class Reference

Public Member Functions

[IntersectQuery\(\)](#)

Data Fields

[LTVector m_From](#)

[LTVector m_To](#)

[LTVector m_Direction](#)

[uint32 m_Flags](#)

[ObjectFilterFn m_FilterFn](#)

[ObjectFilterFn m_FilterActualIntersectFn](#)

[PolyFilterFn m_PolyFilterFn](#)

void * [m_pUserData](#)

void * [m_pActualIntersectUserData](#)

Detailed Description

Pass this in to the IntersectSegment routine.

Constructor & Destructor Documentation

`IntersectQuery::IntersectQuery( )` [inline]

Field Documentation

[LTVector IntersectQuery::m_Direction](#)

Only used for CastRay, this is the direction the ray should be cast in. This doesn't need to be normalized.

[ObjectFilterFn IntersectQuery::m_FilterActualIntersectFn](#)

If this is not LTNUL, then it will call this function when it has an ACTUAL object intersection. If you return false from this function, then it will ignore the object and continue on.

[ObjectFilterFn IntersectQuery::m_FilterFn](#)

If this is not LTNUL, then it will call this function when it has a POSSIBLE object intersection (it doesn't know if it really intersects yet when it calls this). If you return false from this function, then it will ignore the object and continue on.

[uint32 IntersectQuery::m_Flags](#)

A combination of the intersect flags (in [ltcodes.h](#)).

[LTVector IntersectQuery::m_From](#)

Start and end points.

[void* IntersectQuery::m_pActualIntersectUserData](#)

Passed into pUserData of m_FilterActualIntersectFn function

[PolyFilterFn IntersectQuery::m_PolyFilterFn](#)

If this is not LTNULL, then it will call this function when it has a poly intersection. If you return false from this function, then it will ignore the poly and continue on.

[void* IntersectQuery::m_pUserData](#)

Passed into pUserData of m_Filter function.

[LTVector IntersectQuery::m_To](#)

The documentation for this class was generated from the following file:

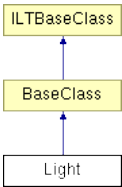
- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Light Class Reference

Inheritance diagram for Light:



Detailed Description

Light class.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LightingMaterial Struct Reference

Data Fields

[FourFloatColor Ambient](#)

[FourFloatColor Diffuse](#)

[FourFloatColor Emissive](#)
[FourFloatColor Specular](#)
float [SpecularPower](#)

Detailed Description

[Light](#) Material values (attenuation values for dynamic lighting).

Field Documentation

[FourFloatColor LightingMaterial::Ambient](#)
Ambient attenuation.

[FourFloatColor LightingMaterial::Diffuse](#)
Diffuse attenuation.

[FourFloatColor LightingMaterial::Emissive](#)
Emissive attenuation.

[FourFloatColor LightingMaterial::Specular](#)
Specular attenuation.

float [LightingMaterial::SpecularPower](#)
Specular Power.

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_LINEF Struct Reference

Data Fields

[LT_VERTF verts](#) [2]
[LT_VERTRGBA rgba](#)

Detailed Description

Contains vertex structure information and color information for a flat shaded line.

Field Documentation

[LT_VERTRGBA LT_LINEF::rgba](#)

[LT_VERTF LT_LINEF::verts](#)[2]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.

[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_LINEFT Struct Reference

Data Fields

[LT_VERTFT verts](#) [2]
[LT_VERTRGBA rgba](#)

Detailed Description

Contains vertex structure information and color information for a textured, flat-shaded line.

Field Documentation

[LT_VERTRGBA](#) [LT_LINEFT::rgba](#)

[LT_VERTFT](#) [LT_LINEFT::verts](#)[2]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_LINEG Struct Reference

Data Fields

[LT_VERTG verts](#) [2]

Detailed Description

Contains vertex structure information for a Gouraud shaded line.

Field Documentation

[LT_VERTG](#) [LT_LINEG::verts](#)[2]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_LINEGT Struct Reference

Data Fields

[LT_VERTGT](#) [verts](#) [2]

Detailed Description

Contains vertex structure information for a textured, Gouraud shaded line.

Field Documentation

[LT_VERTGT](#) [LT_LINEGT::verts](#)[2]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYF3 Struct Reference

Data Fields

[LT_VERTF](#) [verts](#) [3]
[LT_VERTRGBA](#) [rgba](#)

Detailed Description

Contains vertex structure information and color informaiton for a flat-shaded triangle.

Field Documentation

[LT_VERTRGBA](#) [LT_POLYF3::rgba](#)

[LT_VERTF](#) [LT_POLYF3::verts](#)[3]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYF4 Struct Reference

Data Fields

[LT_VERTEX](#) [verts](#) [4]
[LT_VERTEXRGBA](#) [rgba](#)

Detailed Description

Contains vertex structure information and color information for flat shaded quadrilateral.

Field Documentation

[LT_VERTEXRGBA](#) [LT_POLYF4::rgba](#)

[LT_VERTEX](#) [LT_POLYF4::verts](#)[4]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYFT3 Struct Reference

Data Fields

[LT_VERTEXFT](#) [verts](#) [3]
[LT_VERTEXRGBA](#) [rgba](#)

Detailed Description

Contains vertex structure information and color information for a textured, flat-shaded triangle.

Field Documentation

[LT_VERTEXRGBA](#) [LT_POLYFT3::rgba](#)

[LT_VERTEXFT](#) [LT_POLYFT3::verts](#)[3]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYFT4 Struct Reference

Data Fields

[LT_VERTFT_verts](#) [4]
[LT_VERTRGBA_rgba](#)

Detailed Description

Contains vertex structure information and color information for a textured, flat-shaded triangle.

Field Documentation

[LT_VERTRGBA](#) [LT_POLYFT4::rgba](#)

[LT_VERTFT](#) [LT_POLYFT4::verts](#)[4]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYG3 Struct Reference

Data Fields

[LT_VERTG_verts](#) [3]

Detailed Description

Contains vertex structure information for a Gouraud shaded triangle.

Field Documentation

[LT_VERTG](#) [LT_POLYG3::verts](#)[3]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYG4 Struct Reference

Data Fields

[LT_VERTG_verts](#) [4]

Detailed Description

Contains vertex structure information for a Gouraud shaded quadrilateral.

Field Documentation

[LT_VERTG](#) [LT_POLYG4::verts](#)[4]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYGT3 Struct Reference

Data Fields

[LT_VERTGT](#) [verts](#) [3]

Detailed Description

Contains vertex structure information for a textured, Gouraud shaded triangle.

Field Documentation

[LT_VERTGT](#) [LT_POLYGT3::verts](#)[3]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_POLYGT4 Struct Reference

Data Fields

[LT_VERTGT](#) [verts](#) [4]

Detailed Description

Contains vertex structure information for a textured, Gouraud shaded quadrilateral.

Field Documentation

[LT_VERTGT](#) [LT_POLYGT4::verts](#)[4]

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_VERTF Struct Reference

Public Member Functions

[LT_VERTF](#) ()

Data Fields

float [x](#)
float [y](#)
float [z](#)

Detailed Description

Contains coordinate information for a flat-shaded vertex.

Constructor & Destructor Documentation

[LT_VERTF::LT_VERTF](#)() [[inline](#)]

Field Documentation

float [LT_VERTF::x](#)

float [LT_VERTF::y](#)

float [LT_VERTF::z](#)

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_VERTFT Struct Reference

Public Member Functions

[LT_VERTFT](#) ()

Data Fields

float [x](#)
float [y](#)
float [z](#)
float [u](#)
float [v](#)

Detailed Description

Contains coordinate and texture information for a textured, flat-shaded vertex.

Constructor & Destructor Documentation

LT_VERTFT::LT_VERTFT() [inline]

Field Documentation

float [LT_VERTFT::u](#)

float [LT_VERTFT::v](#)

float [LT_VERTFT::x](#)

float [LT_VERTFT::y](#)

float [LT_VERTFT::z](#)

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_VERTG Struct Reference

Public Member Functions

[LT_VERTG](#) ()

Data Fields

float [x](#)
float [y](#)
float [z](#)
[LT_VERTRGBA](#) [rgba](#)

Detailed Description

Contains coordinate and color information for a Gouraud-shaded vertex.

Constructor & Destructor Documentation

LT_VERTG::LT_VERTG() [inline]

Field Documentation

[LT_VERTRGBA](#) [LT_VERTG::rgba](#)

float [LT_VERTG::x](#)

float [LT_VERTG::y](#)

float [LT_VERTG::z](#)

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_VERTGT Struct Reference

Public Member Functions

[LT_VERTGT](#) ()

Data Fields

float [x](#)

float [y](#)

float [z](#)

[LT_VERTRGBA](#) [rgba](#)

float [u](#)

float [v](#)

Detailed Description

Contains coordinate, color, and texture information for a textured, Gouraud shaded vertex.

Constructor & Destructor Documentation

LT_VERTGT::LT_VERTGT() [inline]

Field Documentation

[LT_VERTRGBA](#) [LT_VERTGT::rgba](#)

float [LT_VERTGT::u](#)

float [LT_VERTGT::v](#)

float [LT_VERTGT::x](#)

float [LT_VERTGT::y](#)

float [LT_VERTGT::z](#)

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LT_VERTRGBA Struct Reference

Public Member Functions

[uint32 Compact](#) ()

Data Fields

[uint8 b](#)

[uint8 g](#)

[uint8 r](#)

[uint8 a](#)

Detailed Description

Contains the RGBA data for a vertex. Also provides the Compact operator to convert the RGBA data to a uint32. Gouraud shaded vertices use this structure.

Member Function Documentation

[uint32](#) LT_VERTRGBA::Compact() `[inline]`

Field Documentation

[uint8](#) LT_VERTRGBA::a

[uint8](#) LT_VERTRGBA::b

[uint8](#) LT_VERTRGBA::g

[uint8](#) LT_VERTRGBA::r

The documentation for this struct was generated from the following file:

- [iltdrawprim.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTAnimTracker Class Reference

Public Member Functions

```

    LTAnimTracker ()
    bool IsValid ()
    ModelAnim* GetCurAnim ()
    ModelAnim* GetPrevAnim ()
    AnimInfo* GetCurAnimInfo ()
    AnimInfo* GetPrevAnimInfo ()
    AnimKeyFrame* GetCurFrame ()
    AnimKeyFrame* GetPrevFrame ()
    bool AllowInvalid ()
    LTAnimTracker* GetNext ()
    void SetNext (LTAnimTracker*pNext)
    ModelInstance* GetModelInstance ()
    void SetModelInstance (ModelInstance*pInst)
    Model* GetModel ()
    void SetModel (Model*pModel)
  
```

Data Fields

```

    LTLink m_Link
    StringKeyCallback m_StringKeyCallback
    uint32 m_CurKey
    uint16 m_Flags
    uint16 m_InterpolationMS
    float m_RateModifier
    AnimTimeRef m_TimeRef
    ANIMTRACKERID m_ID
    HMODELNODE m_hHintNode
    HMODELANIM m_hLastHintAnim
    uint32 m_dwLastHintTime
    LTransform m_tfLastHint
    bool m_bDirty
  
```

Detailed Description

Animation tracker for model instances.

Note: Game code should *never* touch the internals of this structure or call any of its functions. It should use the [ILTModel](#) functions to access these.

Constructor & Destructor Documentation

```

LTAnimTracker::LTAnimTracker( ) [inline]
  Movement encoding hint.
  
```

Member Function Documentation

```
bool LTAnimTracker::AllowInvalid( )
```

```
ModelAnim* LTAnimTracker::GetCurAnim( )
```

```
AnimInfo* LTAnimTracker::GetCurAnimInfo( )
```

```
AnimKeyFrame* LTAnimTracker::GetCurFrame( )
```


Model* LTAAnimTracker::GetModel() [inline]

ModelInstance* LTAAnimTracker::GetModelInstance() [inline]

[LTAAnimTracker](#)* LTAAnimTracker::GetNext() [inline]

Get/set its linked list info.

ModelAnim* LTAAnimTracker::GetPrevAnim()

AnimInfo* LTAAnimTracker::GetPrevAnimInfo()

AnimKeyFrame* LTAAnimTracker::GetPrevFrame()

bool LTAAnimTracker::IsValid()

Tells if the frame locators are on valid frames.

void LTAAnimTracker::SetModel(Model * *pModel*) [inline]

void LTAAnimTracker::SetModelInstance(ModelInstance * *pInst*) [inline]

void LTAAnimTracker::SetNext([LTAAnimTracker](#) * *pNext*) [inline]

Field Documentation

bool [LTAAnimTracker::m_bDirty](#)

Tracker dirty?

[uint32 LTAAnimTracker::m_CurKey](#)

Stuff used to track where it is in the keyframes.

[uint32 LTAAnimTracker::m_dwLastHintTime](#)

[uint16 LTAAnimTracker::m_Flags](#)

Combination of **AT_** flags above.

[HMODELNODE LTAAnimTracker::m_hHintNode](#)

Movement encoding hint.

[HMODELANIM LTAAnimTracker::m_hLastHintAnim](#)

[ANIMTRACKERID LTAAnimTracker::m_ID](#)

Reference / identifier for engine-managed tracker

[uint16 LTAAnimTracker::m_InterpolationMS](#)

How far we have ve interpolated between **prev** and **cur** (when switching animations). Set to **NOT_INTERPOLATING** if not interpolating.

[LTLink LTAAnimTracker::m_Link](#)

float [LTAAnimTracker::m_RateModifier](#)

Factor with which to change the animation rate, default = 1.0.

[StringKeyCallback LTAAnimTracker::m_StringKeyCallback](#)

Set this if you want to be notified of string keys.

[LTransform](#) [LAnimTracker::m_tfLastHint](#)

[AnimTimeRef](#) [LAnimTracker::m_TimeRef](#)

The documentation for this class was generated from the following file:

- [ltanimtracker.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTBenchCPUResult_t Struct Reference

Data Fields

[uint32 m_dwSize](#)

[uint32 m_dwPerformanceIndex](#)

[LTFLOAT m_fTestDuration](#)

Detailed Description

Benchmark CPU Test Result Structure

Field Documentation

[uint32 LTBenchCPUResult t::m_dwPerformanceIndex](#)

[uint32 LTBenchCPUResult t::m_dwSize](#)

[LTFLOAT LTBenchCPUResult t::m_fTestDuration](#)

The documentation for this struct was generated from the following file:

- [iltbenchmark.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTBenchCPUTest_t Struct Reference

Data Fields

[uint32 m_dwSize](#)

[uint32 m_dwMinPerformanceIndex](#)

[uint32 m_dwMaxPerformanceIndex](#)

Detailed Description

Benchmark CPU Test Structure

Field Documentation

[uint32 LTBenchCPUTest t::m_dwMaxPerformanceIndex](#)

[uint32 LTBenchCPUTest t::m_dwMinPerformanceIndex](#)

[uint32 LTBenchCPUTest t::m_dwSize](#)

The documentation for this struct was generated from the following file:

- [iltbenchmark.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTBenchGraphicsInfo_t Struct Reference

Data Fields

[uint32 m_dwSize](#)

[uint32 m_dwMinFrameRate](#)

[uint32 m_dwMaxFrameRate](#)

[uint32 m_dwAverageFrameRate](#)

[uint32 m_dwMinPolyCount](#)

[uint32 m_dwMaxPolyCount](#)

[uint32 m_dwAveragePolyCount](#)

[LTFLOAT m_fTestDuration](#)

Detailed Description

Benchmark Graphics Test Structure

Field Documentation

[uint32 LTBenchGraphicsInfo t::m_dwAverageFrameRate](#)

[uint32 LTBenchGraphicsInfo t::m_dwAveragePolyCount](#)

[uint32 LTBenchGraphicsInfo t::m_dwMaxFrameRate](#)

[uint32 LTBenchGraphicsInfo t::m_dwMaxPolyCount](#)

[uint32 LTBenchGraphicsInfo t::m_dwMinFrameRate](#)

[uint32 LTBenchGraphicsInfo t::m_dwMinPolyCount](#)

[uint32 LTBenchGraphicsInfo t::m_dwSize](#)

[LTFLOAT LTBenchGraphicsInfo t::m_fTestDuration](#)

The documentation for this struct was generated from the following file:

- [iltbenchmark.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTCounter Struct Reference

Data Fields

unsigned long [m_Data](#) [4]

Detailed Description

Used in the counter functions.

Field Documentation

unsigned long [LTCounter::m_Data](#)[4]

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTCrashHandler Class Reference

Public Member Functions

[LTCrashHandler](#) (bool init)

Protected Member Functions

virtual MINIDUMP_USER_STREAM_INFORMATION * [WriteMiniDump](#) (LONG [_ExceptionPointers](#) *pExceptionInfo)
[GetUserStreamInfo](#) ()

Static Protected Member Functions

static LONG WINAPI [Handler](#) (struct _EXCEPTION_POINTERS *pExceptionInfo)

Protected Attributes

_EXCEPTION_POINTERS * [m_pExceptionInfo](#)
time_t [m_nTime](#)
TCHAR [m_szDumpPath](#) [_MAX_PATH]

Static Protected Attributes

static [LTCrashHandler](#)* [gpHandler](#)

Constructor & Destructor Documentation

LTCrashHandler::LTCrashHandler(bool *init*)

Member Function Documentation

virtual MINIDUMP_USER_STREAM_INFORMATION* LTCrashHandler::GetUserStreamInfo() [inline, protected, virtual]

static LONG WINAPI LTCrashHandler::Handler(struct _EXCEPTION_POINTERS * *pExceptionInfo*) [static, protected]

LONG LTCrashHandler::WriteMiniDump(_EXCEPTION_POINTERS * *pExceptionInfo*) [protected]

Field Documentation

[LTCrashHandler](#)* [LTCrashHandler::gpHandler](#) [static, protected]

time_t [LTCrashHandler::m_nTime](#) [protected]

_EXCEPTION_POINTERS* [LTCrashHandler::m_pExceptionInfo](#) [protected]

TCHAR [LTCrashHandler::m_szDumpPath](#)[_MAX_PATH] [protected]

The documentation for this class was generated from the following file:

- [lcrashhandler.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTEffectShader Class Reference

Public Types

enum { [EFFECTSHADER_INVALID](#) = 0xFFFFFFFF }

Public Member Functions

int [GetID](#) () const

effect shader ID

const char* [GetName](#) () const

name of effect shader (filename if loaded from file)

[LTEffectShader](#)* [GetNext](#) ()

next in list

virtual [LTRESULT](#) [SetBool](#) (const char* szParam, [LTBOOL](#) bBool) const =0

virtual [LTRESULT](#) [SetBoolArray](#) (const char* szParam, [LTBOOL](#)* bBool, int nCount) const =0

virtual [LTRESULT](#) [SetFloat](#) (const char* szParam, float fFloat) const =0

virtual [LTRESULT](#) [SetFloatArray](#) (const char* szParam, float* fFloat, int nCount) const =0

virtual [LTRESULT](#) [SetInt](#) (const char* szParam, int nInt) const =0

virtual [LTRESULT](#) [SetIntArray](#) (const char* szParam, int* nInt, int nCount) const =0

virtual [LTRESULT](#) [SetMatrix](#) (const char* szParam, [LTMatrix](#)& mMatrix) const =0

virtual [LTRESULT](#) [SetMatrixArray](#) (const char* szParam, [LTMatrix](#)* mMatrix, int nCount) const =0

virtual [LTRESULT](#) [SetMatrixTranspose](#) (const char* szParam, [LTMatrix](#)& mMatrix) const =0

virtual [LTRESULT](#) [SetMatrixTransposeArray](#) (const char *szParam, [LTMatrix](#) *mMatrix, int nCount) const =0
 virtual [LTRESULT](#) [SetString](#) (const char *szParam, const char *szString) const =0
 virtual [LTRESULT](#) [SetTechnique](#) (const char *szTechnique) const =0
 virtual [LTRESULT](#) [ValidateTechnique](#) (const char *szTechnique) const =0
 virtual [LTRESULT](#) [FindFirstValidTechnique](#) ([LTTechniqueInfo](#) *pInfo) const =0
 virtual [LTRESULT](#) [SetTexture](#) (const char *szParam, [HTEXTURE](#) hTexture) const =0
 virtual [LTRESULT](#) [SetTextureRT](#) (const char *szParam, [HRENDERTARGET](#) hRenderTarget) const =0
 virtual [LTRESULT](#) [SetVector](#) (const char *szParam, float *fFloat) const =0
 virtual [LTRESULT](#) [SetVectorArray](#) (const char *szParam, float *fFloat, int nCount) const =0

Protected Member Functions

[LTEffectShader](#) ()

Protected Attributes

int [m_ShaderID](#)
 char [m_FileName](#) [_MAX_PATH]
[LTEffectShader](#) * [m_pNext](#)

Member Enumeration Documentation

anonymous enum

Enumeration values:

EFFECTSHADER_INVALID invalid effect shader ID

Constructor & Destructor Documentation

[LTEffectShader](#)::[LTEffectShader](#)() [[inline](#), [protected](#)]

Member Function Documentation

virtual [LTRESULT](#) [LTEffectShader](#)::[FindFirstValidTechnique](#)([LTTechniqueInfo](#) * *pInfo*) const [[pure virtual](#)]

int [LTEffectShader](#)::[GetID](#)() const [[inline](#)]

effect shader ID

const char* [LTEffectShader](#)::[GetName](#)() const [[inline](#)]

name of effect shader (filename if loaded from file)

[LTEffectShader](#)* [LTEffectShader](#)::[GetNext](#)() [[inline](#)]

next in list

virtual [LTRESULT](#) [LTEffectShader](#)::[SetBool](#)(const char * *szParam*,
[LTBOOL](#) * *bBool*,
) const [[pure virtual](#)]

virtual [LTRESULT](#) [LTEffectShader](#)::[SetBoolArray](#)(const char * *szParam*,
[LTBOOL](#) * *bBool*,
 int *nCount*
) const [[pure virtual](#)]

virtual [LTRESULT](#) [LTEffectShader](#)::[SetFloat](#)(const char * *szParam*,
 float * *fFloat*
) const [[pure virtual](#)]

virtual [LTRESULT](#) [LTEffectShader](#)::[SetFloatArray](#)(const char * *szParam*,
 float * *fFloat*,
 int *nCount*
) const [[pure virtual](#)]

virtual [LTRESULT](#) [LTEffectShader](#)::[SetInt](#)(const char * *szParam*,
 int *nInt*

```
) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetIntArray( const char * szParam,
                                           int *      nInt,
                                           int        nCount
                                           ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetMatrix( const char * szParam,
                                           LTMATRIX & mMatrix
                                           ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetMatrixArray( const char * szParam,
                                                LTMATRIX * mMatrix,
                                                int        nCount
                                                ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetMatrixTranspose( const char * szParam,
                                                    LTMATRIX & mMatrix
                                                    ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetMatrixTransposeArray( const char * szParam,
                                                         LTMATRIX * mMatrix,
                                                         int        nCount
                                                         ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetString( const char * szParam,
                                           const char * szString
                                           ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetTechnique( const char * szTechnique ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetTexture( const char * szParam,
                                           HTEXTURE hTexture
                                           ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetTextureRT( const char *      szParam,
                                              HRENDERTARGET hRenderTarget
                                              ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetVector( const char * szParam,
                                           float *      fFloat
                                           ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::SetVectorArray( const char * szParam,
                                                float *      fFloat,
                                                int          nCount
                                                ) const [pure virtual]
```

```
virtual LTRESULT LEffectShader::ValidateTechnique( const char * szTechnique ) const [pure virtual]
```

Field Documentation

```
char LEffectShader::m\_FileName[_MAX_PATH] [protected]
```

[LTEffectShader*](#) [LTEffectShader::m_pNext](#) [protected]

int [LTEffectShader::m_ShaderID](#) [protected]

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTFloatPt Struct Reference

Data Fields

float [x](#)
float [y](#)

Field Documentation

float [LTFloatPt::x](#)

float [LTFloatPt::y](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTFontParams Class Reference

Public Member Functions

[LTFontParams](#) ()

Data Fields

long [Weight](#)
unsigned char [CharSet](#)
unsigned char [OutPrecision](#)
unsigned char [ClipPrecision](#)
unsigned char [Quality](#)
unsigned char [PitchAndFamily](#)
unsigned char [Italic](#)
unsigned char [Underline](#)
unsigned char [StrikeOut](#)

Constructor & Destructor Documentation

[LTFontParams::LTFontParams\(\)](#) [inline]

Field Documentation

unsigned char [LTFontParams::CharSet](#)

unsigned char [LTFontParams::ClipPrecision](#)

unsigned char [LTFontParams::Italic](#)

unsigned char [LTFontParams::OutPrecision](#)

unsigned char [LTFontParams::PitchAndFamily](#)

unsigned char [LTFontParams::Quality](#)

unsigned char [LTFontParams::StrikeOut](#)

unsigned char [LTFontParams::Underline](#)

long [LTFontParams::Weight](#)

The documentation for this class was generated from the following file:

- [LTFontParams.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTGraphicsCaps Class Reference

Public Member Functions

[LTGraphicsCaps](#) ()

Data Fields

[uint32](#) [VertexShaderVersion](#)

[uint32](#) [PixelShaderVersion](#)

Constructor & Destructor Documentation

[LTGraphicsCaps::LTGraphicsCaps](#)() `[inline]`

Field Documentation

[uint32](#) [LTGraphicsCaps::PixelShaderVersion](#)

[uint32 LTGraphicsCaps::VertexShaderVersion](#)

The documentation for this class was generated from the following file:

- [ltgraphicscaps.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTGUID Union Reference

Public Member Functions

bool [operator<](#)(const [LTGUID](#) & _X) const
 bool [operator==](#)(const [LTGUID](#) & _X) const

Data Fields

[LTGUID guid](#)
[LTGUID_64 guid_64](#)

Member Function Documentation

bool LTGUID::operator<(const [LTGUID](#) & _X) const [inline]

bool LTGUID::operator==(const [LTGUID](#) & _X) const [inline]

Field Documentation

[LTGUID LTGUID::guid](#)

[LTGUID_64 LTGUID::guid_64](#)

The documentation for this union was generated from the following file:

- [ltbasetypes.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTIntPt Struct Reference

Public Member Functions

[LTIntPt](#) (int a=0, int b=0)

Data Fields

int [x](#)
 int [y](#)

Constructor & Destructor Documentation

```
LTIntPt::LTIntPt( int a = 0,  
                 int b = 0  
                 ) [inline]
```

Field Documentation

int [LTIntPt::x](#)

int [LTIntPt::y](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTLine Struct Reference

Data Fields

[LTLinePt m_Points](#) [2]

Detailed Description

The line structure for line systems.

Used for: Special FX.

Field Documentation

[LTLinePt LTLine::m_Points](#)[2]

The documentation for this struct was generated from the following file:

- [iltelient.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTLinePt Struct Reference

Data Fields

[LTVector m_Pos](#)

float [r](#)

Position (relative to system).

float [g](#)
Position (relative to system).

float [b](#)
Position (relative to system).

float [a](#)
Position (relative to system).

Detailed Description

The line point structure for line systems.

Used for: Special FX.

Field Documentation

float [LTLinePt::a](#)
Position (relative to system).

float [LTLinePt::b](#)
Position (relative to system).

float [LTLinePt::g](#)
Position (relative to system).

[LTVector](#) [LTLinePt::m_Pos](#)

float [LTLinePt::r](#)
Position (relative to system).

The documentation for this struct was generated from the following file:

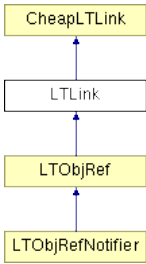
- [iltclient.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTLink Class Reference

Inheritance diagram for LTLink:



Public Member Functions

[LTLink](#) ()
[LTLink](#) ([LTLinkCommand](#) cmd)
void [Init2](#) (void *pData)

Data Fields

void * [m_pData](#)

Constructor & Destructor Documentation

LTLINK::LTLINK() [inline]

LTLINK::LTLINK([LTLINKCommand](#) cmd) [inline]

Member Function Documentation

void LTLINK::Init2(void * pData) [inline]

Field Documentation

void* [LTLINK::m_pData](#)

The documentation for this class was generated from the following file:

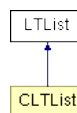
- [ltlink.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTLIST Class Reference

Inheritance diagram for LTLIST:



Public Member Functions

[LTLIST\(\)](#)

[LTLIST\(\[LTLINKCommand\]\(#\) cmd\)](#)

Data Fields

unsigned long [m_nElements](#)

[LTLINK m_Head](#)

Detailed Description

You can use this list structure to wrap up a linked list if you want to keep a count of the elements.

Constructor & Destructor Documentation

LTLIST::LTLIST() [inline]

LTLIST::LTLIST([LTLINKCommand](#) cmd) [inline]

Field Documentation

[LTLINK](#) [LTLIST::m_Head](#)

unsigned long [LTList::m_nElements](#)

The documentation for this class was generated from the following file:

- [lmlink.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTMatrix Struct Reference

Public Member Functions

```
void Apply (LTMatrix &B) const
void Apply (LTVector &p) const
void Apply (const LTVector &p, LTVector &d) const
void Apply4x4 (const LTVector &p, LTVector &d) const
void Apply3x3 (LTVector &v) const
void Apply3x3 (const LTVector &v, LTVector &d) const
const float &El (uint32 i, uint32 j) const
float &El (uint32 i, uint32 j)
void GetBasisVectors (LTVector *R0, LTVector *R1, LTVector *R2) const
LTVector GetScale () const
void GetTranslation (LTVector &t) const
void Identity ()
void Init (float m00, float m01, float m02, float m03, float m10, float m11, float m12, float m13, float m20, float m21, float m22, float m23,
float m30, float m31, float m32, float m33)
bool Inverse ()
LTMatrix MakeInverse () const
LTMatrix MakeInverseTransform () const
void Normalize ()
void Scale (float x, float y, float z)
void SetBasisVectors (const LTVector *R0, const LTVector *R1, const LTVector *R2)
void SetBasisVectors2 (const LTVector *R0, const LTVector *R1, const LTVector *R2)
void SetTranslation (const LTVector &t)
void SetTranslation (float x, float y, float z)
void SetupProjectionMatrix (const LTVector &O, const LTPlane &plane)
void SetupReflectionMatrix (const LTVector &n, const LTVector &p0)
void SetupRot (const LTVector &u, float a)
void SetupScalingMatrix (const LTVector &s)
void Transpose ()
void Transpose3x3 ()
LTPlane TransformPlane (const LTPlane &P)
LTVector operator \* (const LTVector &v) const
LTMatrix operator \* (const LTMatrix &M) const
LTMatrix operator~ () const
```

Data Fields

float [m](#) [4][4]

Member Function Documentation

```
void LTMatrix::Apply( const LTVector & p,
                    LTVector & d
                    ) const [inline]
```

```
void LTMatrix::Apply( LTVector & p ) const [inline]
```

```
void LTMatrix::Apply( LTMatrix & B ) const [inline]
```

```
void LTMMatrix::Apply3x3( const LTVector & v,
                        LTVector & d
                    ) const [inline]
```

```
void LTMMatrix::Apply3x3( LTVector & v ) const [inline]
```

```
void LTMMatrix::Apply4x4( const LTVector & p,
                        LTVector & d
                    ) const [inline]
```

```
float& LTMMatrix::El( uint32 i,
                    uint32 j
                ) [inline]
```

```
const float& LTMMatrix::El( uint32 i,
                        uint32 j
                    ) const [inline]
```

```
void LTMMatrix::GetBasisVectors( LTVector * R0,
                                LTVector * R1,
                                LTVector * R2
                            ) const [inline]
```

```
LTVector LTMMatrix::GetScale( ) const [inline]
```

```
void LTMMatrix::GetTranslation( LTVector & t ) const [inline]
```

```
void LTMMatrix::Identity( ) [inline]
```

```
void LTMMatrix::Init( float m00,
                    float m01,
                    float m02,
                    float m03,
                    float m10,
                    float m11,
                    float m12,
                    float m13,
                    float m20,
                    float m21,
                    float m22,
                    float m23,
                    float m30,
                    float m31,
                    float m32,
                    float m33
                ) [inline]
```

```
bool LTMMatrix::Inverse( ) [inline]
```

Preserve accuracy...

Preserve accuracy...

```
LTMMatrix LTMMatrix::MakeInverse( ) const [inline]
```

```
LTMMatrix LTMMatrix::MakeInverseTransform( ) const [inline]
```

```
void LTMMatrix::Normalize( ) [inline]
```

[LTMatrix](#) LTMatrix::operator *(const [LTMatrix](#) & *M*) const [inline]

[LTVector](#) LTMatrix::operator *(const [LTVector](#) & *v*) const [inline]

[LTMatrix](#) LTMatrix::operator ~() const [inline]

```
void LTMatrix::Scale( float  x,
                    float  y,
                    float  z
                    ) [inline]
```

```
void LTMatrix::SetBasisVectors( const LTVector * R0,
                               const LTVector * R1,
                               const LTVector * R2
                               ) [inline]
```

```
void LTMatrix::SetBasisVectors2( const LTVector * R0,
                                const LTVector * R1,
                                const LTVector * R2
                                ) [inline]
```

```
void LTMatrix::SetTranslation( float  x,
                              float  y,
                              float  z
                              ) [inline]
```

void LTMatrix::SetTranslation(const [LTVector](#) & *t*) [inline]

```
void LTMatrix::SetupProjectionMatrix( const LTVector & O,
                                     const LTPlane & plane
                                     ) [inline]
```

```
void LTMatrix::SetupReflectionMatrix( const LTVector & n,
                                     const LTVector & p0
                                     ) [inline]
```

```
void LTMatrix::SetupRot( const LTVector & u,
                        float          a
                        ) [inline]
```

void LTMatrix::SetupScalingMatrix(const [LTVector](#) & *s*) [inline]

[LTPlane](#) LTMatrix::TransformPlane(const [LTPlane](#) & *P*) [inline]

void LTMatrix::Transpose() [inline]

void LTMatrix::Transpose3x3() [inline]

Field Documentation

float [LTMatrix::m](#)[4][4]

The documentation for this struct was generated from the following file:

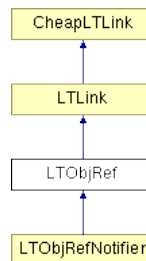
- [ltmatrix.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LObjRef Class Reference

Inheritance diagram for LObjRef:



Public Member Functions

[LObjRef](#) ()
[LObjRef](#) ([HOBJECT](#) hObj)
[LObjRef](#) (const [LObjRef](#) &cOther)
 virtual [~LObjRef](#) ()
[LObjRef](#) & [operator=](#) (const [LObjRef](#) &cOther)
[LObjRef](#) & [operator=](#) ([HOBJECT](#) hObj)
[operator HOBJECT](#) () const
 virtual void [OnObjDelete](#) ()
 virtual void [SetReceiver](#) ([LObjRefReceiver](#) &receiver)
 virtual [LObjRefReceiver](#) * [GetReceiver](#) () const

Private Member Functions

void [Link](#) ([HOBJECT](#) hObj)
 void [Unlink](#) ()

Constructor & Destructor Documentation

[LObjRef::LObjRef](#)() [inline]

[LObjRef::LObjRef](#)([HOBJECT](#) hObj) [inline]

[LObjRef::LObjRef](#)(const [LObjRef](#) & cOther) [inline]

virtual [LObjRef::~LObjRef](#)() [inline, virtual]

Member Function Documentation

virtual [LObjRefReceiver](#)* [LObjRef::GetReceiver](#)() const [inline, virtual]
 Reimplemented in [LObjRefNotifier](#).

void [LObjRef::Link](#)([HOBJECT](#) hObj) [private]

virtual void [LObjRef::OnObjDelete](#)() [inline, virtual]

LObjRef::operator [HOBJECT](#)() const [inline]

[LObjRef](#)& LObjRef::operator=([HOBJECT](#) hObj) [inline]

Reimplemented in [LObjRefNotifier](#).

[LObjRef](#)& LObjRef::operator=(const [LObjRef](#) & cOther) [inline]

virtual void LObjRef::SetReceiver([LObjRefReceiver](#) & receiver) [inline, virtual]

Reimplemented in [LObjRefNotifier](#).

void LObjRef::Unlink() [private]

The documentation for this class was generated from the following file:

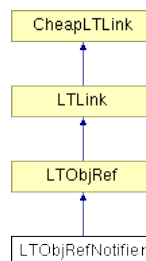
- [ltobjref.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LObjRefNotifier Class Reference

Inheritance diagram for LObjRefNotifier:



Public Member Functions

[LObjRefNotifier](#)()
[LObjRefNotifier](#)([HOBJECT](#) hObj)
[LObjRefNotifier](#)([LObjRefReceiver](#) &receiver)
[LObjRefNotifier](#)(const [LObjRefNotifier](#) &cOther)

[LObjRefNotifier](#) & [operator=](#)([LObjRefNotifier](#) const &cOther)

[LObjRefNotifier](#) & [operator=](#)([HOBJECT](#) hObj)

virtual void [SetReceiver](#)([LObjRefReceiver](#) &receiver)

virtual [LObjRefReceiver](#) * [GetReceiver](#)() const

Private Attributes

[LObjRefReceiver](#) * [m_pReceiver](#)

Constructor & Destructor Documentation

LObjRefNotifier::LObjRefNotifier() [inline]

LObjRefNotifier::LObjRefNotifier([HOBJECT](#) hObj) [inline]

LObjRefNotifier::LObjRefNotifier([LObjRefReceiver](#) & receiver) [inline]

LObjRefNotifier::LObjRefNotifier(const [LObjRefNotifier](#) & *cOther*) [inline]

Member Function Documentation

virtual [ILObjRefReceiver](#)* LObjRefNotifier::GetReceiver() const [inline, virtual]

Reimplemented from [LObjRef](#).

[LObjRefNotifier](#)& LObjRefNotifier::operator=([HOBJECT](#) *hObj*) [inline]

Reimplemented from [LObjRef](#).

[LObjRefNotifier](#)& LObjRefNotifier::operator=([LObjRefNotifier](#) const & *cOther*) [inline]

virtual void LObjRefNotifier::SetReceiver([ILObjRefReceiver](#) & *receiver*) [inline, virtual]

Reimplemented from [LObjRef](#).

Field Documentation

[ILObjRefReceiver](#)* [LObjRefNotifier::m_pReceiver](#) [private]

The documentation for this class was generated from the following file:

- [ltobjref.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTParticle Struct Reference

Data Fields

[LTVector](#) [m_Pos](#)

[LTVector](#) [m_Velocity](#)

Current position of the particle.

float [m_Size](#)

Current velocity of the particle.

[LTVector](#) [m_Color](#)

Particle size.

float [m_Alpha](#)

Color of the particle: RGB 0-255.

float [m_fAngle](#)

Alpha of the particle: 0-1.

float [m_fAngularVelocity](#)

Angle in radians of this particle.

float [m_Lifetime](#)

Velocity of the angular change in radians per second.

float [m_TotalLifetime](#)

Current lifetime left.

[uint32](#) [m_nUserData](#)

Total lifetime (i.e. initial value).

[LTParticle](#) * [m_pNext](#)

32 bits of user data...

[LTParticle](#) * [m_pPrev](#)

Linked list information.

Detailed Description

The particle structure for particle systems.

Used for: Special FX.

Field Documentation

float [LTParticle::m_Alpha](#)

Color of the particle: RGB 0-255.

[LTVector](#) [LTParticle::m_Color](#)

Particle size.

float [LTParticle::m_fAngle](#)

Alpha of the particle: 0-1.

float [LTParticle::m_fAngularVelocity](#)

Angle in radians of this particle.

float [LTParticle::m_Lifetime](#)

Velocity of the angular change in radians per second.

[uint32](#) [LTParticle::m_nUserData](#)

Total lifetime (i.e. initial value).

[LTParticle*](#) [LTParticle::m_pNext](#)

32 bits of user data...

[LTVector](#) [LTParticle::m_Pos](#)

[LTParticle*](#) [LTParticle::m_pPrev](#)

Linked list information.

float [LTParticle::m_Size](#)

Current velocity of the particle.

float [LTParticle::m_TotalLifetime](#)

Current lifetime left.

[LTVector](#) [LTParticle::m_Velocity](#)

Current position of the particle.

The documentation for this struct was generated from the following file:

- [iltclient.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTPerformanceInfo_t Struct Reference

Data Fields

[uint32](#) [m_dwSize](#)

[uint32](#) [m_dwFlags](#)

[uint32](#) [m_dwScreenWidth](#)

[uint32](#) [m_dwScreenHeight](#)

[uint32](#) [m_dwFps](#)

[uint32](#) [m_dwNumWorldPolys](#)

[uint32](#) [m_dwNumWorldPolysProcessed](#)

[uint32](#) [m_dwNumModelPolys](#)

[uint32](#) [m_dwReserved1](#)

Detailed Description

Enhanced run-time performance info.

Field Documentation

[uint32 LTPerformanceInfo t::m_dwFlags](#)

[uint32 LTPerformanceInfo t::m_dwFps](#)

[uint32 LTPerformanceInfo t::m_dwNumModelPolys](#)

[uint32 LTPerformanceInfo t::m_dwNumWorldPolys](#)

[uint32 LTPerformanceInfo t::m_dwNumWorldPolysProcessed](#)

[uint32 LTPerformanceInfo t::m_dwReserved1](#)

[uint32 LTPerformanceInfo t::m_dwScreenHeight](#)

[uint32 LTPerformanceInfo t::m_dwScreenWidth](#)

[uint32 LTPerformanceInfo t::m_dwSize](#)

The documentation for this struct was generated from the following file:

- [iltinfo.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTPixelShader Class Reference

Public Types

```
enum {
    PIXELSHADER\_ENVBUMPMAP = -52,
    PIXELSHADER\_SCREENGLOW = -51,
    PIXELSHADER\_SHADOWBLUR = -50,
    PIXELSHADER\_INVALID = 0xFFFFFFFF
}
enum { MAX\_CONSTANT\_REGISTERS = 8 }
max number of user-defined constants (4 floats each) More...
```

Public Member Functions

```
virtual bool IsValidShader () const =0
    is the shader valid?
int GetID () const
    pixel shader ID
const char * GetName () const
    name of shader (filename if loaded from file)
LTPixelShader * GetNext ()
```

next in list

virtual bool [GetConstant](#) (unsigned RegisterNum, float *pf0, float *pf1, float *pf2, float *pf3)=0
get the values in a constant register

virtual bool [SetConstant](#) (unsigned RegisterNum, float f0, float f1, float f2, float f3)=0
set the values in a constant register

virtual bool [SetConstant](#) (unsigned RegisterNum, const [LTMatrix](#) &Matrix)=0
copies the values in the given matrix to the four constant registers starting at RegisterNum

virtual float * [GetConstants](#) ()=0
get the constant array, there are 4 floats per register

unsigned [GetNumConstants](#) () const
get the number of constants, there are 4 floats per register

Protected Member Functions

[LTPixelShader](#) ()

Protected Attributes

int [m_ShaderID](#)
char [m_FileName](#) [_MAX_PATH]
[LTPixelShader](#) * [m_pNext](#)

Detailed Description

Pixel shader definition

Member Enumeration Documentation

anonymous enum

Enumeration values:

PIXELSHADER_ENVBUMPMAP reserved ID for world envbumpmap shader
PIXELSHADER_SCREENGLOW reserved ID for screen glow shader
PIXELSHADER_SHADOWBLUR reserved ID for shadow blur shader
PIXELSHADER_INVALID invalid pixel shader ID

anonymous enum

max number of user-defined constants (4 floats each)

Enumeration values:

MAX_CONSTANT_REGISTERS

Constructor & Destructor Documentation

[LTPixelShader::LTPixelShader](#)() [*inline*, *protected*]

Member Function Documentation

virtual bool [LTPixelShader::GetConstant](#)(unsigned *RegisterNum*,
float * *pf0*,
float * *pf1*,
float * *pf2*,
float * *pf3*
) [*pure virtual*]
get the values in a constant register

virtual float* [LTPixelShader::GetConstants](#)() [*pure virtual*]
get the constant array, there are 4 floats per register

int [LTPixelShader::GetID](#)() const [*inline*]
pixel shader ID

const char* [LTPixelShader::GetName](#)() const [*inline*]
name of shader (filename if loaded from file)

[LTPixelShader](#)* [LTPixelShader::GetNext](#)() [*inline*]
next in list

```
unsigned LTPixelShader::GetNumConstants( ) const [inline]
```

get the number of constants, there are 4 floats per register

```
virtual bool LTPixelShader::IsValidShader( ) const [pure virtual]
```

is the shader valid?

```
virtual bool LTPixelShader::SetConstant( unsigned RegisterNum,
                                         const LTMatrix & Matrix
                                         ) [pure virtual]
```

copies the values in the given matrix to the four constant registers starting at RegisterNum

```
virtual bool LTPixelShader::SetConstant( unsigned RegisterNum,
                                         float f0,
                                         float f1,
                                         float f2,
                                         float f3
                                         ) [pure virtual]
```

set the values in a constant register

Field Documentation

```
char LTPixelShader::m\_FileName[ MAX_PATH] [protected]
```

```
LTPixelShader\* LTPixelShader::m\_pNext [protected]
```

```
int LTPixelShader::m\_ShaderID [protected]
```

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTPlane Struct Reference

Public Member Functions

```
LTPlane ( )
LTPlane (const LTVector &n, const LTVector &p)
void Init (const LTVector &n, const LTVector &p)
LTPlane (float x, float y, float z, float d)
LTPlane (const LTVector &n, float d)
void Init (const LTVector &n, float d)
LTVector & Normal ( )
float & Dist ( )
float DistTo (const LTVector &p) const
const LTPlane operator- ( )
```

Data Fields

```
LTVector m\_Normal
float m\_Dist
```

Detailed Description

The LTPlane structure describes a plane in 3-dimensional space. Positive distances from the plane to a point are in the same direction as its normal.

Used for: Math.

Constructor & Destructor Documentation

LTPlane::LTPlane() [inline]

The default constructor.

Used for: Math.

LTPlane::LTPlane(const [LTVector](#) & *n*,
const [LTVector](#) & *P*
) [inline]

Parameters:

n a unit normal

P a point on the plane

Construct a plane with a unit normal *n* and a point *P* on the plane.

Used for: Math.

LTPlane::LTPlane(float *x*,
float *y*,
float *z*,
float *d*
) [inline]

Parameters:

x *n_x*

y *n_y*

z *n_z*

d the negative of the signed distance from the origin to the plane

Construct a plane with a unit normal $\mathbf{n} = (x, y, z)$ and the negative of the signed distance from the origin $\mathbf{O} = (0, 0, 0)$ to the plane.

Used for: Math.

LTPlane::LTPlane(const [LTVector](#) & *n*,
float *d*
) [inline]

Parameters:

n a unit normal

P a point on the plane

Construct a plane with a unit normal *n* and the negative of the signed distance from the origin $\mathbf{O} = (0, 0, 0)$ to the plane.

Used for: Math.

Member Function Documentation

float& LTPlane::Dist() [inline]

Returns:

the negative of the signed distance from the origin $\mathbf{O} = (0, 0, 0)$ to the plane

Used for: Math.

float LTPlane::DistTo(const [LTVector](#) & *p*) const [inline]

Parameters:

p a 3D point

Returns:

the signed distance from the plane to the point *P*

Compute the signed distance from the plane to the point *P*.

Used for: Math.

void LTPlane::Init(const [LTVector](#) & *n*,
float *d*
) [inline]

Parameters:

n a unit normal

P a point on the plane

Initialize a plane with a unit normal *n* and the negative of the signed distance from the origin $\mathbf{O} = (0, 0, 0)$ to the plane.

Used for: Math.

```
void LTPlane::Init( const LTVector & n,
                   const LTVector & P
                 ) [inline]
```

Parameters:

n a unit normal
P a point on the plane

Initialize a plane with a unit normal *n* and a point *P* on the plane.

Used for: Math.

```
LTVector & LTPlane::Normal( ) [inline]
```

Returns:

the unit normal *n* of the plane

Used for: Math.

```
const LTPlane LTPlane::operator-( ) [inline]
```

Returns:

a plane with the values $-n, -dist$

Negate the values of **m_Normal** and **m_Dist**. This creates a plane that is a reflection about the origin of the first (so they are parallel), and whose normal points the opposite direction.

Used for: Math.

Field Documentation

```
float LTPlane::m\_Dist
```

negative of the signed distance from the origin $O = (0, 0, 0)$ to the plane

```
LTVector LTPlane::m\_Normal
```

unit normal

The documentation for this struct was generated from the following file:

- [ltplane.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTransform Class Reference

Data Fields

[LTVector](#) [m_Pos](#)
[LTRotation](#) [m_Rot](#)
[LTVector](#) [m_Scale](#)

Detailed Description

This class use a position and quaternion to represent a coordinate frame. The [ILTTransform](#) interface can be used to work with these objects.

Field Documentation

```
LTVector LTransform::m\_Pos
```

[LTRotation](#) [LTransform::m_Rot](#)

[LTVector](#) [LTransform::m_Scale](#)

The documentation for this class was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTRect Class Reference

Public Member Functions

[LTRect](#) (int *l*=0, int *t*=0, int *r*=0, int *b*=0)
void [Init](#) (int *inLeft*, int *inTop*, int *inRight*, int *inBottom*)

Data Fields

int [left](#)
int [top](#)
int [right](#)
int [bottom](#)

Constructor & Destructor Documentation

```
LTRect::LTRect( int l = 0,  
               int t = 0,  
               int r = 0,  
               int b = 0  
               ) [inline]
```

Parameters:

l x position of left side
t y position of top side
r x position of right side
b y position of bottom side

Used for: Misc.

Member Function Documentation

```
void LTRect::Init( int inLeft,  
                  int inTop,  
                  int inRight,  
                  int inBottom  
                  ) [inline]
```

Parameters:

inLeft x position of left side
inTop y position of top side
inRight x position of right side
inBottom y position of bottom side

Initialize the values.

Used for: 2D Rendering.

Field Documentation

int [LTRect::bottom](#)

int [LRect::left](#)

int [LRect::right](#)

int [LRect::top](#)

The documentation for this class was generated from the following file:

- [ltrect.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTRendererStats Struct Reference

Data Fields

int [TextureChanges](#)
 int [TextureChangeSaves](#)
 int [WorldBaseTexMemory](#)
 int [WorldEnvMapTexMemory](#)
 int [WorldDetailTexMemory](#)
 int [WorldBumpMapTexMemory](#)
 int [WorldLightMapTexMemory](#)
 int [ModelTexMemory](#)
 int [SpriteTexMemory](#)
 int [ParticleTexMemory](#)
 int [PolyGridBaseTexMemory](#)
 int [PolyGridEnvMapTexMemory](#)
 int [PolyGridBumpMapTexMemory](#)
 int [VolumeEffectTexMemory](#)
 int [DrawPrimTexMemory](#)
 int [TotalUncompressedTexMemory](#)
 int [ModelRender_NumSkeletalRenderObjects](#)
 int [ModelRender_NumSkeletalPieces](#)
 int [ModelRender_NumRigidPieces](#)
 int [ModelRender_NumVertexAnimatedPieces](#)
 int [ModelRender_RenderStyleSets](#)
 int [ModelRender_TextureSets](#)
 int [ModelRender_LightingSets](#)
 int [ModelRender_ReallyCloseSets](#)
 int [ModelRender_ScaleSets](#)
 int [ObjectsCullTested](#)
 int [ObjectsCulled](#)
 int [WorldBlocksCullTested](#)
 int [WorldBlocksCulled](#)
 int [InsideBoxCullCount](#)
 int [FrustumCulledCount](#)
 int [OccluderCulledCount](#)
 int [ModelTriangles](#)
 int [ParticleTriangles](#)
 int [WorldTriangles](#)
 int [SpriteTriangles](#)
 int [PolyGridTriangles](#)
 int [VolumeEffectTriangles](#)
 int [ModelShadowTriangles](#)
 int [DynamicLightTriangles](#)
 int [WorldTextureMem](#)
 int [ModelTextureMem](#)
 int [ParticleTextureMem](#)
 int [SpriteTextureMem](#)
 int [PolyGridTextureMem](#)
 int [SkyPortals](#)
 int [HardwareBlocks](#)
 int [Occluders](#)
 int [VisibleLights](#)
 int [RBSections](#)

int [ShaderTris](#)
int [ShaderVerts](#)
int [ShaderDrawPrim](#)
int [ShaderBatched](#)

Field Documentation

int [LTRendererStats::DrawPrimTexMemory](#)

int [LTRendererStats::DynamicLightTriangles](#)

int [LTRendererStats::FrustumCulledCount](#)

int [LTRendererStats::HardwareBlocks](#)

int [LTRendererStats::InsideBoxCullCount](#)

int [LTRendererStats::ModelRender_LightingSets](#)

int [LTRendererStats::ModelRender_NumRigidPieces](#)

int [LTRendererStats::ModelRender_NumSkeletalPieces](#)

int [LTRendererStats::ModelRender_NumSkeletalRenderObjects](#)

int [LTRendererStats::ModelRender_NumVertexAnimatedPieces](#)

int [LTRendererStats::ModelRender_ReallyCloseSets](#)

int [LTRendererStats::ModelRender_RenderStyleSets](#)

int [LTRendererStats::ModelRender_ScaleSets](#)

int [LTRendererStats::ModelRender_TextureSets](#)

int [LTRendererStats::ModelShadowTriangles](#)

int [LTRendererStats::ModelTexMemory](#)

int [LTRendererStats::ModelTextureMem](#)

int [LTRendererStats::ModelTriangles](#)

int [LTRendererStats::ObjectsCulled](#)

int [LTRendererStats::ObjectsCullTested](#)

int [LTRendererStats::OccluderCulledCount](#)

int [LTRendererStats::Occluders](#)

int [LTRendererStats::ParticleTexMemory](#)

int [LTRendererStats::ParticleTextureMem](#)

int [LTRendererStats::ParticleTriangles](#)

int [LTRendererStats::PolyGridBaseTexMemory](#)

int [LTRendererStats::PolyGridBumpMapTexMemory](#)

int [LTRendererStats::PolyGridEnvMapTexMemory](#)

int [LTRendererStats::PolyGridTextureMem](#)

int [LTRendererStats::PolyGridTriangles](#)

int [LTRendererStats::RBSections](#)

int [LTRendererStats::ShaderBatched](#)

int [LTRendererStats::ShaderDrawPrim](#)

int [LTRendererStats::ShaderTris](#)

int [LTRendererStats::ShaderVerts](#)

int [LTRendererStats::SkyPortals](#)

int [LTRendererStats::SpriteTexMemory](#)

int [LTRendererStats::SpriteTextureMem](#)

int [LTRendererStats::SpriteTriangles](#)

int [LTRendererStats::TextureChanges](#)

int [LTRendererStats::TextureChangeSaves](#)

int [LTRendererStats::TotalUncompressedTexMemory](#)

int [LTRendererStats::VisibleLights](#)

int [LTRendererStats::VolumeEffectTexMemory](#)

int [LTRendererStats::VolumeEffectTriangles](#)

int [LTRendererStats::WorldBaseTexMemory](#)

int [LTRendererStats::WorldBlocksCulled](#)

int [LTRendererStats::WorldBlocksCullTested](#)

int [LTRendererStats::WorldBumpMapTexMemory](#)

int [LTRendererStats::WorldDetailTexMemory](#)

int [LTRendererStats::WorldEnvMapTexMemory](#)

int [LTRendererStats::WorldLightMapTexMemory](#)

int [LTRendererStats::WorldTextureMem](#)

int [LTRendererStats::WorldTriangles](#)

The documentation for this struct was generated from the following file:

- [ltrendererstats.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTRGB Struct Reference

Data Fields

[uint8 b](#)
[uint8 g](#)
[uint8 r](#)
[uint8 a](#)

Detailed Description

Color types (different byte ordering for cross platform development).

Field Documentation

[uint8 LTRGB::a](#)

[uint8 LTRGB::b](#)

[uint8 LTRGB::g](#)

[uint8 LTRGB::r](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTRotation Struct Reference

Public Member Functions

[LTRotation](#) ()
[LTRotation](#) (const [LTRotation](#) &q)
[LTRotation](#) (const [LTOrientation](#) &R)
[operator LTOrientation](#) () const
[LTRotation](#) (const [LTVector](#) &vAxis, float fAngle)
[LTRotation](#) (const [LTVector](#) &vForward, const [LTVector](#) &vUp)
[LTRotation](#) (float fPitch, float fYaw, float fRoll)
[LTRotation](#) (float x, float y, float z, float r)
void [Init](#) (float a=0, float b=0, float c=0, float d=1)
void [Identity](#) ()
float &[operator\[\]](#) ([uint32](#) i)
[LTRotation](#) & [operator=](#) (const [LTRotation](#) &q)
[LTRotation](#) & [operator=](#) (const [LTMatrix](#) &M)
void [ConvertToMatrix](#) ([LTMatrix](#) &M) const
void [ConvertFromMatrix](#) (const [LTMatrix](#) &mat)
bool [operator==](#) (const [LTRotation](#) &q) const
bool [operator!=](#) (const [LTRotation](#) &q) const
bool [IsIdentity](#) () const
[LTRotation](#) [Conjugate](#) () const
[LTRotation](#) [operator~](#) () const
[LTRotation](#) [operator*](#) (const [LTRotation](#) &q) const
void [Slerp](#) (const [LTRotation](#) &q0, const [LTRotation](#) &q1, float u)
[LTVector](#) [Right](#) () const
[LTVector](#) [Up](#) () const
[LTVector](#) [Forward](#) () const
void [Rotate](#) (const [LTVector](#) &vAxis, float fAngle)

Data Fields

float [m_Quat](#) [4]

Detailed Description

Used for: Obsolete.

Constructor & Destructor Documentation

```
LTRotation::LTRotation( ) [inline]
```

```
LTRotation::LTRotation( const LTRotation & q ) [inline]
```

```
LTRotation::LTRotation( const LTOrientation & R ) [inline]
```

```
LTRotation::LTRotation( const LTVector & vAxis,  
                        float fAngle  
                        ) [inline]
```

```
LTRotation::LTRotation( const LTVector & vForward,  
                        const LTVector & vUp  
                        ) [inline]
```

```
LTRotation::LTRotation( float fPitch,  
                        float fYaw,  
                        float fRoll  
                        ) [inline]
```

```
LTRotation::LTRotation( float x,  
                        float y,  
                        float z,  
                        float r  
                        ) [inline]
```

Member Function Documentation

```
LTRotation LTRotation::Conjugate( ) const [inline]
```

```
void LTRotation::ConvertFromMatrix( const LTMatrix & mat ) [inline]
```

```
void LTRotation::ConvertToMatrix( LTMatrix & M ) const [inline]
```

```
LTVector LTRotation::Forward( ) const [inline]
```

```
void LTRotation::Identity( ) [inline]
```

```
void LTRotation::Init( float a = 0,  
                     float b = 0,  
                     float c = 0,  
                     float d = 1  
                     ) [inline]
```

```
bool LTRotation::IsIdentity( ) const [inline]
```

```
LTRotation LTRotation::operator *( const LTRotation & q ) const [inline]
```

```
LTRotation::operator LTOrientation( ) const [inline]
```

```
bool LTRotation::operator!=( const LTRotation & q ) const [inline]
```



```
LTRotation& LTRotation::operator=( const LTMatrix & M ) [inline]
```

```
LTRotation& LTRotation::operator=( const LTRotation & q ) [inline]
```

```
bool LTRotation::operator==( const LTRotation & q ) const [inline]
```

```
float& LTRotation::operator[]( uint32 i ) [inline]
```

```
LTRotation LTRotation::operator~( ) const [inline]
```

```
LTVector LTRotation::Right( ) const [inline]
```

```
void LTRotation::Rotate( const LTVector & vAxis,  
                        float fAngle  
                        ) [inline]
```

```
void LTRotation::Slerp( const LTRotation & q0,  
                       const LTRotation & q1,  
                       float u  
                       ) [inline]
```

```
LTVector LTRotation::Up( ) const [inline]
```

Field Documentation

```
float LTRotation::m\_Quat[4]
```

The documentation for this struct was generated from the following file:

- [ltrotation.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTShaderDeviceState Class Reference

Public Types

```
enum LTMatrixType {  
    MATRIXTYPE\_WORLD0,  
    MATRIXTYPE\_WORLD1,  
    MATRIXTYPE\_WORLD2,  
    MATRIXTYPE\_WORLD3,  
    MATRIXTYPE\_VIEW,  
    MATRIXTYPE\_PROJECTION,  
    MATRIXTYPE\_VIEWPROJECTION,  
    MATRIXTYPE\_TEXTURE0,  
    MATRIXTYPE\_TEXTURE1,  
    MATRIXTYPE\_TEXTURE2,  
    MATRIXTYPE\_TEXTURE3,
```

```

MATRIXTYPE\_TEXTURE4,
MATRIXTYPE\_TEXTURE5,
MATRIXTYPE\_TEXTURE6,
MATRIXTYPE\_TEXTURE7
}
matrix types More...
enum LTLightType {
    LIGHTTYPE\_POINT,
    LIGHTTYPE\_SPOT,
    LIGHTTYPE\_DIRECTIONAL
}
light types More...

```

Public Member Functions

virtual bool [GetMatrix](#) ([LTMatrixType](#) MatrixType, bool Transpose, [LTMatrix](#) *pMatrix) const =0
 get a matrix from the device; set Transpose to true to have the matrix transposed

virtual bool [GetLight](#) (unsigned LightIndex, [LTLightDesc](#) *pLightDesc) const =0
 get a light from the device; LightIndex is in the range [0,7]

virtual bool [GetCamera](#) ([LTCameraDesc](#) *pCameraDesc) const =0
 get the camera from the device

Protected Member Functions

[LTShaderDeviceState](#) ()

Data Structures

```

struct LTCameraDesc
camera settings More...
struct LTLightDesc
light settings More...

```

Detailed Description

Shader device state (used in [IClientShell](#) shader callback functions)

Member Enumeration Documentation

enum [LTShaderDeviceState::LTLightType](#)
 light types

Enumeration values:

```

    LIGHTTYPE_POINT
    LIGHTTYPE_SPOT
    LIGHTTYPE_DIRECTIONAL

```

enum [LTShaderDeviceState::LTMatrixType](#)
 matrix types

Enumeration values:

```

    MATRIXTYPE_WORLD0
    MATRIXTYPE_WORLD1
    MATRIXTYPE_WORLD2
    MATRIXTYPE_WORLD3
    MATRIXTYPE_VIEW
    MATRIXTYPE_PROJECTION
    MATRIXTYPE_VIEWPROJECTION
    MATRIXTYPE_TEXTURE0
    MATRIXTYPE_TEXTURE1
    MATRIXTYPE_TEXTURE2
    MATRIXTYPE_TEXTURE3
    MATRIXTYPE_TEXTURE4
    MATRIXTYPE_TEXTURE5
    MATRIXTYPE_TEXTURE6
    MATRIXTYPE_TEXTURE7

```

Constructor & Destructor Documentation

[LTShaderDeviceState::LTShaderDeviceState](#)() [[inline](#), [protected](#)]

Member Function Documentation

virtual bool LTShaderDeviceState::GetCamera([LTCameraDesc](#) * *pCameraDesc*) const [pure virtual]
get the camera from the device

virtual bool LTShaderDeviceState::GetLight(unsigned *LightIndex*,
[LTLightDesc](#) * *pLightDesc*) const [pure virtual]
get a light from the device; LightIndex is in the range [0,7]

virtual bool LTShaderDeviceState::GetMatrix([LTMatrixType](#) *MatrixType*,
bool *Transpose*,
[LTMatrix](#) * *pMatrix*) const [pure virtual]
get a matrix from the device; set Transpose to true to have the matrix transposed

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

[LTShaderDeviceState::LTCameraDesc](#)

LTShaderDeviceState::LTCameraDesc Struct Reference

camera settings [More...](#)

Data Fields

[LTVector](#) [m_Position](#)
[LTRotation](#) [m_Rotation](#)
float [m_xFov](#)
float [m_yFov](#)

Detailed Description

camera settings

Field Documentation

[LTVector](#) [LTShaderDeviceState::LTCameraDesc::m_Position](#)

[LTRotation](#) [LTShaderDeviceState::LTCameraDesc::m_Rotation](#)

float [LTShaderDeviceState::LTCameraDesc::m_xFov](#)

float [LTShaderDeviceState::LTCameraDesc::m_yFov](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

[LTShaderDeviceState::LTLightDesc](#)

LTShaderDeviceState::LTLightDesc Struct Reference

light settings [More...](#)

Data Fields

[LTLightType](#) [m_LightType](#)

bool [m_Active](#)

[LTRGB](#) [m_Diffuse](#)

[LTRGB](#) [m_Specular](#)

[LTRGB](#) [m_Ambient](#)

[LTVector](#) [m_Position](#)

[LTVector](#) [m_Direction](#)

float [m_Range](#)

float [m_Falloff](#)

float [m_Atten0](#)

float [m_Atten1](#)

float [m_Atten2](#)

float [m_Theta](#)

float [m_Phi](#)

Detailed Description

light settings

Field Documentation

bool [LTShaderDeviceState::LTLightDesc::m_Active](#)

[LTRGB](#) [LTShaderDeviceState::LTLightDesc::m_Ambient](#)

float [LTShaderDeviceState::LTLightDesc::m_Atten0](#)

float [LTShaderDeviceState::LTLightDesc::m_Atten1](#)

float [LTShaderDeviceState::LTLightDesc::m_Atten2](#)

[LTRGB](#) [LTShaderDeviceState::LTLightDesc::m_Diffuse](#)

[LTVector](#) [LTShaderDeviceState::LTLightDesc::m_Direction](#)

float [LTShaderDeviceState::LTLightDesc::m_Falloff](#)

[LTLightType](#) [LTShaderDeviceState::LTLightDesc::m_LightType](#)

float [LTShaderDeviceState::LTLightDesc::m_Phi](#)

[LTVector](#) [LTShaderDeviceState::LTLightDesc::m_Position](#)

float [LTShaderDeviceState::LTLightDesc::m_Range](#)

[LTRGB LTShaderDeviceState::LTLightDesc::m_Specular](#)

float [LTShaderDeviceState::LTLightDesc::m_Theta](#)

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTTechniqueInfo Struct Reference

Data Fields

char [szName](#) [128]
int [nPasses](#)

Field Documentation

int [LTTechniqueInfo::nPasses](#)

char [LTTechniqueInfo::szName](#)[128]

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTVersionInfoExt_t Struct Reference

Data Fields

[uint32 m_dwSize](#)
[uint32 m_dwFlags](#)
[uint32 m_dwMajorVersion](#)
[uint32 m_dwMinorVersion](#)
[uint32 m_dwBuildNumber](#)
[uint32 m_dwBuildMonth](#)
[uint32 m_dwBuildDay](#)
[uint32 m_dwBuildYear](#)
char [m_sBuildName](#) [128]
[uint32 m_dwLanguage](#)
[uint32 m_dwPlatform](#)
[uint32 m_dwReserved1](#)
[uint32 m_dwReserved2](#)

Detailed Description

Enhanced version info.

Field Documentation

[uint32 LTVersionInfoExt t::m_dwBuildDay](#)

[uint32 LTVersionInfoExt t::m_dwBuildMonth](#)

[uint32 LTVersionInfoExt t::m_dwBuildNumber](#)

[uint32 LTVersionInfoExt t::m_dwBuildYear](#)

[uint32 LTVersionInfoExt t::m_dwFlags](#)

[uint32 LTVersionInfoExt t::m_dwLanguage](#)

[uint32 LTVersionInfoExt t::m_dwMajorVersion](#)

[uint32 LTVersionInfoExt t::m_dwMinorVersion](#)

[uint32 LTVersionInfoExt t::m_dwPlatform](#)

[uint32 LTVersionInfoExt t::m_dwReserved1](#)

[uint32 LTVersionInfoExt t::m_dwReserved2](#)

[uint32 LTVersionInfoExt t::m_dwSize](#)

char [LTVersionInfoExt t::m_sBuildName](#)[128]

The documentation for this struct was generated from the following file:

- [iltinfo.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

LTVertexShader Class Reference

Public Types

```
enum { VERTEXSHADER\_INVALID = 0xFFFFFFFF }  
invalid vertex shader ID More...  
enum { MAX\_CONSTANT\_REGISTERS = 96 }  
max number of user-defined constants (4 floats each) More...
```

Public Member Functions

virtual bool [IsValidShader](#) () const =0
is the shader valid?

int [GetID](#) () const
vertex shader ID

const char * [GetName](#) () const
name of shader (filename)

[LTVVertexShader](#) * [GetNext](#) ()
next in list

virtual bool [GetConstant](#) (unsigned RegisterNum, float *pf0, float *pf1, float *pf2, float *pf3)=0
get the values in a constant register

virtual bool [SetConstant](#) (unsigned RegisterNum, float f0, float f1, float f2, float f3)=0
set the values in a constant register

virtual bool [SetConstant](#) (unsigned RegisterNum, const [LTMatrix](#) &Matrix)=0
copies the values in the given matrix to the four constant registers starting at RegisterNum

virtual float * [GetConstants](#) ()=0
get the constant array, there are 4 floats per register

unsigned [GetNumConstants](#) () const
get the number of constants, there are 4 floats per register

Protected Member Functions

[LTVVertexShader](#) ()

Protected Attributes

int [m_ShaderID](#)
char [m_FileName](#) [_MAX_PATH]
[LTVVertexShader](#) * [m_pNext](#)

Detailed Description

Vertex shader definition

Member Enumeration Documentation

anonymous enum

invalid vertex shader ID

Enumeration values:

VERTEXSHADER_INVALID

anonymous enum

max number of user-defined constants (4 floats each)

Enumeration values:

MAX_CONSTANT_REGISTERS

Constructor & Destructor Documentation

[LTVVertexShader::LTVVertexShader](#)() [*inline*, *protected*]

Member Function Documentation

virtual bool [LTVVertexShader::GetConstant](#)(unsigned *RegisterNum*,
float * *pf0*,
float * *pf1*,
float * *pf2*,
float * *pf3*
) [*pure virtual*]

get the values in a constant register

virtual float* [LTVVertexShader::GetConstants](#)() [*pure virtual*]

get the constant array, there are 4 floats per register

int [LTVVertexShader::GetID](#)() const [*inline*]

vertex shader ID

```
const char* LTVertexShader::GetName( ) const [inline]
    name of shader (filename)
```

```
LTVertexShader* LTVertexShader::GetNext( ) [inline]
    next in list
```

```
unsigned LTVertexShader::GetNumConstants( ) const [inline]
    get the number of constants, there are 4 floats per register
```

```
virtual bool LTVertexShader::IsValidShader( ) const [pure virtual]
    is the shader valid?
```

```
virtual bool LTVertexShader::SetConstant( unsigned RegisterNum,
                                           const LTMatrix & Matrix
                                           ) [pure virtual]
    copies the values in the given matrix to the four constant registers starting at RegisterNum
```

```
virtual bool LTVertexShader::SetConstant( unsigned RegisterNum,
                                           float f0,
                                           float f1,
                                           float f2,
                                           float f3
                                           ) [pure virtual]
    set the values in a constant register
```

Field Documentation

```
char LTVertexShader::m\_FileName[_MAX_PATH] [protected]
```

```
LTVertexShader* LTVertexShader::m\_pNext [protected]
```

```
int LTVertexShader::m\_ShaderID [protected]
```

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

LTWarpPt Struct Reference

Data Fields

```
float source\_x
float source\_y
float dest\_x
float dest\_y
```

Field Documentation

```
float LTWarpPt::dest\_x
```

```
float LTWarpPt::dest\_y
```


float [LTWarpPt::source_x](#)

float [LTWarpPt::source_y](#)

The documentation for this struct was generated from the following file:

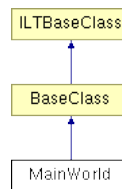
- [ltbasetypes.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

MainWorld Class Reference

Inheritance diagram for MainWorld:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Detailed Description

MainWorld.

Member Function Documentation

```
virtual uint32 MainWorld::EngineMessageFn( uint32 messageID,
                                           void * pData,
                                           float  lData
                                           ) [virtual]
```

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ModelHookData Struct Reference

Data Fields

[HLOCALOBJ](#) [m_hObject](#)

[uint32](#) [m_HookFlags](#)

Combination of MHF_ flags above.

[uint32](#) [m_ObjectFlags](#)

[LTVector](#) [m_LightAdd](#)

RGB 0-255.

[LTVector](#) [m_ObjectColor](#)

RGB 0-255.

Field Documentation

[HLOCALOBJ](#) [ModelHookData::m_hObject](#)

[uint32](#) [ModelHookData::m_HookFlags](#)

Combination of MHF_ flags above.

[LTVector](#) [ModelHookData::m_LightAdd](#)

RGB 0-255.

[LTVector](#) [ModelHookData::m_ObjectColor](#)

RGB 0-255.

[uint32](#) [ModelHookData::m_ObjectFlags](#)

The model's object flags. You can modify them in here without changing the model's flags permanently.

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ModelOBB Struct Reference

Data Fields

[LTVector](#) [m_Pos](#)

[LTVector](#) [m_Size](#)

[LTVector](#) [m_Basis](#) [3]

[uint32](#) [m_iNode](#)

float [m_Radius](#)

Field Documentation

[LTVector](#) [ModelOBB::m_Basis](#)[3]

[uint32 ModelOBB::m_iNode](#)

[LTVector ModelOBB::m_Pos](#)

float [ModelOBB::m_Radius](#)

[LTVector ModelOBB::m_Size](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ModelOBB_Depricated Struct Reference

Data Fields

[LTVector m_Pos](#)

[LTVector m_Size](#)

[LTVector m_Basis](#) [3]

[uint32 m_iNode](#)

Detailed Description

This is oriented bounding box used for models.

Field Documentation

[LTVector ModelOBB_Depricated::m_Basis](#)[3]

[uint32 ModelOBB_Depricated::m_iNode](#)

[LTVector ModelOBB_Depricated::m_Pos](#)

[LTVector ModelOBB_Depricated::m_Size](#)

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

MoveInfo Struct Reference

Data Fields

[HOBJECT m_hObject](#)
float [m_dt](#)
[LTVector m_Offset](#)

Field Documentation

float [MoveInfo::m_dt](#)

[HOBJECT MoveInfo::m_hObject](#)

[LTVector MoveInfo::m_Offset](#)

The documentation for this struct was generated from the following file:

- [iltphysics.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

NetHost Struct Reference

Data Fields

[uint32 m_Port](#)
[uint32 m_dwFlags](#)
Port if TCP/IP. If zero, it uses the default.

[uint32 m_dwMaxConnections](#)
char [m_sName](#) [4096]
bool [m_bHasPassword](#)
[uint8 m_nGameType](#)

Detailed Description

Structure used by to instruct a driver to create/host a new session.

Field Documentation

bool [NetHost::m_bHasPassword](#)

[uint32 NetHost::m_dwFlags](#)
Port if TCP/IP. If zero, it uses the default.

[uint32 NetHost::m_dwMaxConnections](#)

[uint8 NetHost::m_nGameType](#)

[uint32 NetHost::m_Port](#)

char [NetHost::m_sName](#)[4096]

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

NetService Class Reference

Data Fields

[HNETSERVICE](#) [m_handle](#)

[uint32](#) [m_dwFlags](#)

[LTGUID](#) [m_guidService](#)

Combination of NETSERVICE_ flags.

char [m_sName](#) [128]

[NetService](#) * [m_pNext](#)

Field Documentation

[uint32](#) [NetService::m_dwFlags](#)

[LTGUID](#) [NetService::m_guidService](#)

Combination of NETSERVICE_ flags.

[HNETSERVICE](#) [NetService::m_handle](#)

[NetService](#)* [NetService::m_pNext](#)

char [NetService::m_sName](#)[128]

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

NetSession Class Reference

Public Member Functions

[NetSession](#) ()

virtual [~NetSession](#) ()

void [Clear](#) ()

Data Fields

[uint32](#) [m_dwFlags](#)

[LTGUID m_guidApp](#)
[LTGUID m_guidInst](#)
[uint32 m_dwMaxConnections](#)
[uint32 m_dwCurConnections](#)
[uint32 m_Ping](#)
char [m_HostIP](#) [32]
Ping time in milliseconds.
[uint32 m_HostPort](#)
char [m_sName](#) [4096]
char [m_bHasPassword](#)
[uint8 m_nGameType](#)
[NetSession* m_pNext](#)

Constructor & Destructor Documentation

[NetSession::NetSession\(\)](#) [inline]

[virtual NetSession::~~NetSession\(\)](#) [inline, virtual]

Member Function Documentation

[void NetSession::Clear\(\)](#) [inline]

Field Documentation

char [NetSession::m_bHasPassword](#)

[uint32 NetSession::m_dwCurConnections](#)

[uint32 NetSession::m_dwFlags](#)

[uint32 NetSession::m_dwMaxConnections](#)

[LTGUID NetSession::m_guidApp](#)

[LTGUID NetSession::m_guidInst](#)

char [NetSession::m_HostIP](#)[32]

Ping time in milliseconds.

Host info. 0 length string and NO_HOST_PORT if not on TCP/IP.

[uint32 NetSession::m_HostPort](#)

[uint8 NetSession::m_nGameType](#)

[uint32 NetSession::m_Ping](#)

[NetSession* NetSession::m_pNext](#)

char [NetSession::m_sName](#)[4096]

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

NodeControlData Struct Reference

Data Fields

const [LTMatrix](#) * [m_pParentTransform](#)
const [LTMatrix](#) * [m_pFromParentTransform](#)
 [LTMatrix](#) * [m_pNodeTransform](#)
 [HOBJECT](#) [m_hModel](#)
[HMODELNODE](#) [m_hNode](#)

Detailed Description

Node Control Data This structure holds all the data that is passed into the node control callback

Field Documentation

[HOBJECT](#) [NodeControlData::m_hModel](#)

[HMODELNODE](#) [NodeControlData::m_hNode](#)

const [LTMatrix](#)* [NodeControlData::m_pFromParentTransform](#)

[LTMatrix](#)* [NodeControlData::m_pNodeTransform](#)

const [LTMatrix](#)* [NodeControlData::m_pParentTransform](#)

The documentation for this struct was generated from the following file:

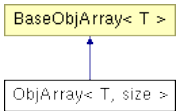
- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ObjArray< T, size > Class Template Reference

Inheritance diagram for ObjArray< T, size >:



Public Member Functions

[ObjArray](#) ()

Private Attributes

T [m_Array](#) [size]

```
template<class T, int size>
class ObjArray< T, size >::
```

Constructor & Destructor Documentation

```
template<class T, int size>
ObjArray< T, size >::ObjArray( ) [inline]
```

Field Documentation

```
template<class T, int size>
T ObjArray< T, size >::m\_Array[size] [private]
```

The documentation for this class was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ObjectCreateStruct Struct Reference

Public Member Functions

[ObjectCreateStruct](#) ()
void [Clear](#) ()

Data Fields

[uint16](#) [m_ObjectType](#)
[uint16](#) [m_ContainerCode](#)
[uint32](#) [m_Flags](#)
[uint32](#) [m_Flags2](#)
[uint8](#) [m_nRenderGroup](#)
[LTVector](#) [m_GlobalForceOverride](#)
[LTVector](#) [m_Pos](#)
[LTVector](#) [m_Scale](#)
[LTRotation](#) [m_Rotation](#)
[uint32](#) [m_UserData](#)
User data.
[HCLASS](#) [m_hClass](#)
char [m_Name](#) [MAX_CS_FILENAME_LEN+1]
This object's name.
float [m_NextUpdate](#)
This will be the object's starting NextUpdate.
[GenericPropList](#) [m_cProperties](#)
List of properties.
char [m_Filename](#) [MAX_CS_FILENAME_LEN+1]
char [m_Filenames](#) [MAX_CHILD_MODELS][MAX_CS_FILENAME_LEN+1]
char [m_SkinName](#) [MAX_CS_FILENAME_LEN+1]
char [m_SkinNames](#) [MAX_MODEL_TEXTURES][MAX_CS_FILENAME_LEN+1]
char [m_RenderStyleName](#) [MAX_CS_FILENAME_LEN+1]
char [m_RenderStyleNames](#) [MAX_MODEL_RENDERSTYLES][MAX_CS_FILENAME_LEN+1]

Detailed Description

This structure is used when creating objects. When you want to create an object, you call [ILTServer::CreateObject](#) with one of these. The structure you pass in is passed to the object's PostPropRead function, where it can override whatever it wants to.

Constructor & Destructor Documentation

[ObjectCreateStruct::ObjectCreateStruct\(\)](#) [inline]

Member Function Documentation

[void ObjectCreateStruct::Clear\(\)](#) [inline]

Field Documentation

[uint16 ObjectCreateStruct::m_ContainerCode](#)

[Container](#) code if it's a container. It's in here because you can only set it at creation time.

[GenericPropList ObjectCreateStruct::m_cProperties](#)

List of properties.

char [ObjectCreateStruct::m_Filename](#)[MAX_CS_FILENAME_LEN+1]

char [ObjectCreateStruct::m_Filenames](#)[MAX_CHILD_MODELS][MAX_CS_FILENAME_LEN+1]

[uint32 ObjectCreateStruct::m_Flags](#)

[uint32 ObjectCreateStruct::m_Flags2](#)

[LTVector ObjectCreateStruct::m_GlobalForceOverride](#)

[HCLASS ObjectCreateStruct::m_hClass](#)

NOTE: this is ONLY used during OnPreCreate processing, and is filled in by the engine.

char [ObjectCreateStruct::m_Name](#)[MAX_CS_FILENAME_LEN+1]

This object's name.

Server only info.

float [ObjectCreateStruct::m_NextUpdate](#)

This will be the object's starting NextUpdate.

[uint8 ObjectCreateStruct::m_nRenderGroup](#)

[uint16 ObjectCreateStruct::m_ObjectType](#)

Main info.

[LTVector ObjectCreateStruct::m_Pos](#)

char [ObjectCreateStruct::m_RenderStyleName](#)[MAX_CS_FILENAME_LEN+1]

This can be zero-length. If you set it for an OT_MODEL, it's the skin filename.

char [ObjectCreateStruct::m_RenderStyleNames](#)[MAX_MODEL_RENDERSTYLES][MAX_CS_FILENAME_LEN+1]

[LTRotation](#) [ObjectCreateStruct::m_Rotation](#)

[LTVector](#) [ObjectCreateStruct::m_Scale](#)

char [ObjectCreateStruct::m_SkinName](#)[MAX_CS_FILENAME_LEN+1]
This can be zero-length. If you set it for an OT_MODEL, it's the skin filename.

char [ObjectCreateStruct::m_SkinNames](#)[MAX_MODEL_TEXTURES][MAX_CS_FILENAME_LEN+1]

[uint32](#) [ObjectCreateStruct::m_UserData](#)
User data.

The documentation for this struct was generated from the following file:

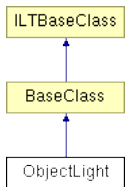
- [ltobjectcreate.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ObjectLight Class Reference

Inheritance diagram for ObjectLight:



Detailed Description

ObjectLight class (these lights only light objects, they don't light the world). These are used for landscape areas lit by **GlobalDirLight** objects (which don't light objects).

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ObjectLink Struct Reference

Data Fields

[HOBJECT](#) [m_hObject](#)
[ObjectLink](#) * [m_pNext](#)

Detailed Description

Object lists.

Field Documentation

[HOBJECT](#) [ObjectLink::m_hObject](#)

[ObjectLink*](#) [ObjectLink::m_pNext](#)

The documentation for this struct was generated from the following file:

- [iltserver.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ObjectList Struct Reference

Data Fields

[ObjectLink](#) * [m_pFirstLink](#)
int [m_nInList](#)

Field Documentation

int [ObjectList::m_nInList](#)

[ObjectLink*](#) [ObjectList::m_pFirstLink](#)

The documentation for this struct was generated from the following file:

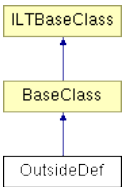
- [iltserver.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

OutsideDef Class Reference

Inheritance diagram for OutsideDef:



Detailed Description

OutsideDef class. These are used if you want the preprocessor to make a leak file for a level.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

PGColor Struct Reference

Data Fields

float [x](#)
RGB 0-255.
float [y](#)
RGB 0-255.
float [z](#)
RGB 0-255.
float [a](#)
Alpha 0-255.

Detailed Description

Structures.

Field Documentation

float [PGColor::a](#)
Alpha 0-255.

float [PGColor::x](#)
RGB 0-255.

float [PGColor::y](#)
RGB 0-255.

float [PGColor::z](#)
RGB 0-255.

The documentation for this struct was generated from the following file:

- [ltbasetypes.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

PlaySoundInfo Struct Reference

Data Fields

[uint32 m_dwFlags](#)
char [m_szSoundName](#) [_MAX_PATH+1]
[HOBJECT m_hObject](#)
[HLTSOUND m_hSound](#)
unsigned char [m_nPriority](#)

float [m_fOuterRadius](#)
 float [m_fInnerRadius](#)
 uint8 [m_nVolume](#)
 float [m_fPitchShift](#)
[LTVector](#) [m_vPosition](#)
 uint32 [m_UserData](#)
 uint8 [m_nSoundVolumeClass](#)

Detailed Description

Defines a variety of options for sounds. The options you choose can drastically change the performance of the sound rendering. This structure must be populated for each sound to be played. To initialize the **PlaySoundInfo** struct to default values, use the **PLAYSOUNDINFO_INIT** macro.

Field Documentation

[uint32 PlaySoundInfo::m_dwFlags](#)

A combination of any of the **PLAYSOUND_xxxx** flags. **PLAYSOUND_LOCAL**, **PLAYSOUND_AMBIENT** and **PLAYSOUND_3D** are mutually exclusive.

float [PlaySoundInfo::m_fInnerRadius](#)

For Ambient or 3D sounds. Clients outside the outer radius will not hear this sound. Clients within the inner radius will hear the sound at full volume. Clients between the inner and outer radius will hear an interpolated volume.

float [PlaySoundInfo::m_fOuterRadius](#)

For Ambient or 3D sounds. Clients outside the outer radius will not hear this sound. Clients within the inner radius will hear the sound at full volume. Clients between the inner and outer radius will hear an interpolated volume.

float [PlaySoundInfo::m_fPitchShift](#)

This is a multiplier to the playback frequency of the sound, which will shift its pitch. If you had a sound recorded at 22,000 Hz and set its pitch shift to 1.1f, then the output sound would be 24,200 Hz and make the sound have a higher pitch. This value is ignored unless **PLAYSOUND_CTRL_PITCH** is set.

[HOBJECT PlaySoundInfo::m_hObject](#)

The server-side object to which the sound will be attached. This parameter must be filled out if **PLAYSOUND_ATTACHED** or **PLAYSOUND_CLIENTLOCAL** is set in the .

[HLTSOUND PlaySoundInfo::m_hSound](#)

This parameter is filled out by the [ILTSoundMgr::PlaySound\(\)](#) function if the **PLAYSOUND_GETHANDLE** is set. The game can use this handle with other [ILTSoundMgr](#) functions that require a handle to the sound.

unsigned char [PlaySoundInfo::m_nPriority](#)

The priority of the sound. Zero is the lowest priority. The engine uses this priority to decide which sounds get rendered when all the voices have been taken.

[uint8 PlaySoundInfo::m_nSoundVolumeClass](#)

[Sound](#) type. This is used with functions to allow different classes of sounds to be played with different volume ranges

[uint8 PlaySoundInfo::m_nVolume](#)

The maximum volume of this sound. This value can range between 0 and 100. This value is ignored unless **PLAYSOUND_CTRL_VOLUME** is set.

char [PlaySoundInfo::m_szSoundName](#)[_MAX_PATH+1]

A string containing the path to the sound to play.

[uint32 PlaySoundInfo::m_UserData](#)

Game defined user data. The 32bit value will be passed to [IClientShell::OnPlaySound\(\)](#).

[LTVector PlaySoundInfo::m_vPosition](#)

The 3D position of the sound in world coordinates. This is only needed for Ambient or 3D sounds and if **PLAYSOUND_ATTACHED** is not used.

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)
-

API Reference

PropDef Struct Reference

Public Member Functions

[PropDef](#) (char *pName, short type, [LTVector](#) valVec, float valFloat, char *valString, unsigned long propFlags)

Data Fields

char * [m_PropName](#)
short [m_PropType](#)
[LTVector](#) [m_DefaultValueVector](#)
float [m_DefaultValueFloat](#)
char * [m_DefaultValueString](#)
unsigned long [m_PropFlags](#)
DEditInternal * [m_pDEditInternal](#)
void * [m_pInternal](#)

Constructor & Destructor Documentation

```
PropDef::PropDef( char *      pName,  
                  short      type,  
                  LTVector    valVec,  
                  float      valFloat,  
                  char *      valString,  
                  unsigned long propFlags  
                ) [inline]
```

Field Documentation

float [PropDef::m_DefaultValueFloat](#)

char* [PropDef::m_DefaultValueString](#)

[LTVector](#) [PropDef::m_DefaultValueVector](#)

Default vector/color value.

DEditInternal* [PropDef::m_pDEditInternal](#)

Don't touch!

void* [PropDef::m_pInternal](#)

Don't touch!

unsigned long [PropDef::m_PropFlags](#)

char* [PropDef::m_PropName](#)

short [PropDef::m_PropType](#)

One of the **PT_** defines above.

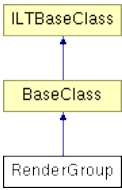
The documentation for this struct was generated from the following file:

- [Itserverobj.h](#)
-

API Reference

RenderGroup Class Reference

Inheritance diagram for RenderGroup:



Detailed Description

RenderGroup class.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

RenderPassOp Struct Reference

Data Fields

[TextureStageOps](#) [TextureStages](#) [4]
[ERenStyle](#) [BlendMode](#) [BlendMode](#)
[ERenStyle](#) [ZBufferMode](#) [ZBufferMode](#)
[ERenStyle](#) [CullMode](#) [CullMode](#)
 [uint32](#) [TextureFactor](#)
 [uint32](#) [AlphaRef](#)
 bool [DynamicLight](#)
[ERenStyle](#) [TestMode](#) [ZBufferTestMode](#)
[ERenStyle](#) [TestMode](#) [AlphaTestMode](#)
[ERenStyle](#) [FillMode](#) [FillMode](#)
 bool [bUseBumpEnvMap](#)
 [uint32](#) [BumpEnvMapStage](#)
 float [fBumpEnvMap_Scale](#)
 float [fBumpEnvMap_Offset](#)

Detailed Description

Contains the set of operations that are applied to the model piece for a given render pass. Each Render Style has one or more render passes.

See also:

[AddRenderPass\(\)](#), [SetRenderPass\(\)](#), [GetRenderPass\(\)](#) = 0;

Field Documentation

[uint32](#) [RenderPassOp::AlphaRef](#)

Alpha Ref values (for alphatesting)...

[ERenStyle](#) [TestMode](#) [RenderPassOp::AlphaTestMode](#)

The [ERenStyle_TestMode](#) value for this **RenderPassOp**.

[ERenStyle_BlendMode](#) [RenderPassOp::BlendMode](#)

The **ERenStyle_BlendMode** value for this **RenderPassOp**.

[uint32 RenderPassOp::BumpEnvMapStage](#)

Set the BumpEnvMap Stage...

bool [RenderPassOp::bUseBumpEnvMap](#)

Enable BumpEnvMap

[ERenStyle_CullMode](#) [RenderPassOp::CullMode](#)

The **ERenStyle_CullMode** value for this **RenderPassOp**.

bool [RenderPassOp::DynamicLight](#)

Enable dynamic lighting...

float [RenderPassOp::fBumpEnvMap_Offset](#)

BumpEnvMap offset.

float [RenderPassOp::fBumpEnvMap_Scale](#)

BumpEnvMap scale.

[ERenStyle_FillMode](#) [RenderPassOp::FillMode](#)

The **ERenStyle_FillMode** value for this **RenderPassOp**.

[uint32 RenderPassOp::TextureFactor](#)

ARGB value that can be used in the texture stages.

[TextureStageOps](#) [RenderPassOp::TextureStages](#)[4]

An array of [TextureStageOps](#) for this **RenderPassOp**.

[ERenStyle_ZBufferMode](#) [RenderPassOp::ZBufferMode](#)

The **ERenStyle_ZBufferMode** value for this **RenderPassOp**.

[ERenStyle_TestMode](#) [RenderPassOp::ZBufferTestMode](#)

The Z comparison function that will be used for this render pass

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

ReverbProperties Struct Reference

Data Fields

[uint32 m_dwParams](#)

Set the params bits for which fields are valid.

float [m_fVolume](#)

0.0 - 1.0

[uint32 m_dwAcoustics](#)

One of the REVERB_ACOUSTICS_XXX values.

float [m_fReflectTime](#)

0.0 - 5.0 Time for first reflection

float [m_fDecayTime](#)

0.1 - 20.0 Determines how quickly reflection diminish

float [m_fDamping](#)

Field Documentation

[uint32 ReverbProperties::m_dwAcoustics](#)

One of the REVERB_ACOUSTICS_xxx values.

[uint32 ReverbProperties::m_dwParams](#)

Set the params bits for which fields are valid.

float [ReverbProperties::m_fDamping](#)

0.0 - 2.0, == 1.0f is even damping, < 1.0f low frequencies dampen faster; > 1.0f high frequencies dampen faster

float [ReverbProperties::m_fDecayTime](#)

0.1 - 20.0 Determines how quickly reflection diminish

float [ReverbProperties::m_fReflectTime](#)

0.0 - 5.0 Time for first reflection

float [ReverbProperties::m_fVolume](#)

0.0 - 1.0

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

RMode Struct Reference

Data Fields

bool [m_bHWTnL](#)

Does this mode support hardware TnL?

char [m_InternalName](#) [128]

This is what the DLLs use to identify a card.

char [m_Description](#) [128]

This is a 'friendly' string describing the card.

[uint32 m_Width](#)

The dimensions and bit depth of this mode.

[uint32 m_Height](#)

The dimensions and bit depth of this mode.

[uint32 m_BitDepth](#)

The dimensions and bit depth of this mode.

[RMode * m_pNext](#)

The next mode in the list.

Detailed Description

Render modes are what are used to describe a video mode/video card.

Field Documentation

bool [RMode::m_bHWTnL](#)

Does this mode support hardware TnL?

[uint32 RMode::m_BitDepth](#)

The dimensions and bit depth of this mode.

char [RMode::m_Description](#) [128]

This is a 'friendly' string describing the card.

[uint32 RMode::m_Height](#)

The dimensions and bit depth of this mode.

char [RMode::m_InternalName](#)[128]

This is what the DLLs use to identify a card.

[RMode* RMode::m_pNext](#)

The next mode in the list.

[uint32 RMode::m_Width](#)

The dimensions and bit depth of this mode.

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

RSD3DOptions Struct Reference

Data Fields

bool [bUseEffectShader](#)
int [EffectShaderID](#)

Field Documentation

bool [RSD3DOptions::bUseEffectShader](#)

int [RSD3DOptions::EffectShaderID](#)

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

RSD3DRenderPass Struct Reference

Data Fields

bool [bUseVertexShader](#)
indicates whether or not to use a vertex shader

int [VertexShaderID](#)
Id of a loaded vertex shader.

bool [bUsePixelShader](#)
indicates whether or not to use a vertex shader

int [PixelShaderID](#)
Id of a loaded pixel shader.

Detailed Description

Contains render pass options for Direct3d platforms.

See also:

[SetRenderPass_D3DOptions\(\)](#), [GetRenderPass_D3DOptions\(\)](#)

Field Documentation

bool [RSD3DRenderPass::bUsePixelShader](#)
indicates whether or not to use a vertex shader

bool [RSD3DRenderPass::bUseVertexShader](#)
indicates whether or not to use a vertex shader

int [RSD3DRenderPass::PixelShaderID](#)
Id of a loaded pixel shader.

int [RSD3DRenderPass::VertexShaderID](#)
Id of a loaded vertex shader.

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SCategoryCheckCache Struct Reference

Data Fields

[int32 current_filter_version](#)
bool [category_passed](#)

Field Documentation

bool [SCategoryCheckCache::category_passed](#)

[int32 SCategoryCheckCache::current_filter_version](#)

The documentation for this struct was generated from the following file:

- [lterror.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

ServerAppHandler Class Reference

Public Member Functions

virtual [LTRESULT](#) [ShellMessageFn](#) ([ILTMessage_Read](#) &msg)
 virtual [LTRESULT](#) [ConsoleOutputFn](#) (const char *pMsg)
 virtual [LTFLOAT](#) [GetCPUUtilization](#) ()
 virtual [LTRESULT](#) [OutOfMemory](#) ()
 virtual [LTRESULT](#) [ProcessPacket](#) ([ILTMessage_Read](#) &cMsg, [uint8](#) senderAddr[4], [uint16](#) senderPort)

Member Function Documentation

virtual [LTRESULT](#) [ServerAppHandler::ConsoleOutputFn](#)(const char * *pMsg*) [inline, virtual]

virtual [LTFLOAT](#) [ServerAppHandler::GetCPUUtilization](#)() [inline, virtual]

virtual [LTRESULT](#) [ServerAppHandler::OutOfMemory](#)() [inline, virtual]

virtual [LTRESULT](#) [ServerAppHandler::ProcessPacket](#)([ILTMessage_Read](#) & *cMsg*,
 [uint8](#) *senderAddr*[4],
 [uint16](#) *senderPort*
) [inline, virtual]

virtual [LTRESULT](#) [ServerAppHandler::ShellMessageFn](#)([ILTMessage_Read](#) & *msg*) [inline, virtual]

The documentation for this class was generated from the following file:

- [server_interface.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006. All Rights Reserved.

API Reference

ServerInterface Class Reference

Public Member Functions

virtual [LTRESULT](#) [SetAppHandler](#) ([ServerAppHandler](#) *pHandler)=0
 virtual [LTRESULT](#) [RunConsoleString](#) (char *pStr)=0
 virtual [LTRESULT](#) [GetConsoleVar](#) (char *pName, [HCONSOLEVAR](#) *hVar, char *pDefaultVal)=0
 virtual [LTRESULT](#) [GetVarValueFloat](#) ([HCONSOLEVAR](#) hVar, float *val)=0
 virtual [LTRESULT](#) [GetVarValueString](#) ([HCONSOLEVAR](#) hVar, char *pStr, [uint32](#) maxLen)=0
 virtual [LTRESULT](#) [LoadConfigFile](#) (char *pFilename)=0
 virtual [LTRESULT](#) [SaveConfigFile](#) (char *pFilename)=0
 virtual [LTRESULT](#) [SendToServerShell](#) ([ILTMessage_Read](#) &msg)=0
 virtual bool [AddResources](#) (const char **pResources, [uint32](#) nResources)=0
 virtual bool [SetGameInfo](#) (void *pGameInfo, [uint32](#) nGameInfoLen)=0
 virtual bool [LoadBinaries](#) ()=0
 virtual [FileEntry](#) * [GetFileList](#) (char *pDirName)=0
 virtual void [FreeFileList](#) ([FileEntry](#) *pList)=0
 virtual bool [StartWorld](#) ([StartGameRequest](#) *pRequest)=0
 virtual int [GetNumClients](#) ()=0
 virtual bool [GetClientName](#) (int index, char *pName, int maxChars)=0
 virtual bool [SetClientName](#) (int index, char *pName, int maxChars)=0
 virtual bool [GetClientInfo](#) (int index, [ClientInfo](#) *pInfo)=0
 virtual bool [BootClient](#) ([uint32](#) dwClientID)=0
 virtual bool [GetClientPing](#) ([uint32](#) nClientID, float &ping)=0
 virtual bool [GetClientAddr](#) ([uint32](#) nClientID, [uint8](#) pAddr[4], [uint16](#) *pPort)=0
 virtual int [GetErrorCode](#) ()=0
 virtual void [GetErrorString](#) (char *pString, int maxLen)=0
 virtual bool [Update](#) (long flags)=0
 virtual bool [InitNetworking](#) (char *pDriver, [uint32](#) dwFlags)=0
 virtual bool [GetServiceList](#) ([NetService](#) *pListHead)=0
 virtual bool [FreeServiceList](#) ([NetService](#) *pListHead)=0
 virtual bool [SelectService](#) ([HNETSERVICE](#) hNetService)=0

```

virtual bool UpdateSessionName(const char *sName)=0
virtual bool HostGame(NetHost *pHostInfo)=0
virtual bool GetTcpIpAddress(char *sAddress, uint32 dwBufferSize, unsigned short &hostPort)=0
virtual LTRESULT SendTo(ILTMessage Read *pMsg, const char *sAddr, uint32 port)=0

```

Member Function Documentation

```

virtual bool ServerInterface::AddResources( const char ** pResources,
                                           uint32      nResources
                                           ) [pure virtual]

virtual bool ServerInterface::BootClient( uint32 dwClientId ) [pure virtual]

virtual void ServerInterface::FreeFileList( FileEntry * pList ) [pure virtual]

virtual bool ServerInterface::FreeServiceList( NetService * pListHead ) [pure virtual]

virtual bool ServerInterface::GetClientAddr( uint32  nClientId,
                                           uint8    pAddr[4],
                                           uint16 * pPort
                                           ) [pure virtual]

virtual bool ServerInterface::GetClientInfo( int      index,
                                           ClientInfo * pInfo
                                           ) [pure virtual]

virtual bool ServerInterface::GetClientName( int      index,
                                           char * pName,
                                           int      maxChars
                                           ) [pure virtual]

virtual bool ServerInterface::GetClientPing( uint32 nClientId,
                                           float & ping
                                           ) [pure virtual]

virtual LTRESULT ServerInterface::GetConsoleVar( char *      pName,
                                           HCONSOLEVAR * hVar,
                                           char *      pDefaultVal
                                           ) [pure virtual]

virtual int ServerInterface::GetErrorCode( ) [pure virtual]

virtual void ServerInterface::GetErrorString( char * pString,
                                           int      maxLen
                                           ) [pure virtual]

virtual FileEntry* ServerInterface::GetFileList( char * pDirName ) [pure virtual]

virtual int ServerInterface::GetNumClients( ) [pure virtual]

virtual bool ServerInterface::GetServiceList( NetService *& pListHead ) [pure virtual]

virtual bool ServerInterface::GetTcpIpAddress( char *      sAddress,
                                           uint32      dwBufferSize,

```

```
        unsigned short & hostPort
    ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::GetVarValueFloat( HCONSOLEVAR hVar,
                                                    float * val
    ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::GetVarValueString( HCONSOLEVAR hVar,
                                                    char * pStr,
                                                    uint32 maxLen
    ) [pure virtual]
```

```
virtual bool ServerInterface::HostGame( NetHost * pHostInfo ) [pure virtual]
```

```
virtual bool ServerInterface::InitNetworking( char * pDriver,
                                              uint32 dwFlags
    ) [pure virtual]
```

```
virtual bool ServerInterface::LoadBinaries( ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::LoadConfigFile( char * pFilename ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::RunConsoleString( char * pStr ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::SaveConfigFile( char * pFilename ) [pure virtual]
```

```
virtual bool ServerInterface::SelectService( HNETSERVICE hNetService ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::SendTo( ILTMessage\_Read * pMsg,
                                          const char * sAddr,
                                          uint32 port
    ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::SendToServerShell( ILTMessage\_Read & msg ) [pure virtual]
```

```
virtual LTRESULT ServerInterface::SetAppHandler( ServerAppHandler * pHandler ) [pure virtual]
```

```
virtual bool ServerInterface::SetClientName( int index,
                                             char * pName,
                                             int maxChars
    ) [pure virtual]
```

```
virtual bool ServerInterface::SetGameInfo( void * pGameInfo,
                                           uint32 nGameInfoLen
    ) [pure virtual]
```

```
virtual bool ServerInterface::StartWorld( StartGameRequest * pRequest ) [pure virtual]
```

```
virtual bool ServerInterface::Update( long flags ) [pure virtual]
```

```
virtual bool ServerInterface::UpdateSessionName( const char * sName ) [pure virtual]
```

The documentation for this class was generated from the following file:

- [server_interface.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SFunctionInfo Struct Reference

Data Fields

const char * [func_name](#)

const char * [progress](#)
[uint32 recursion_depth](#)

Field Documentation

const char* [SFunctionInfo::func_name](#)

const char* [SFunctionInfo::progress](#)

[uint32 SFunctionInfo::recursion_depth](#)

The documentation for this struct was generated from the following file:

- [lterror.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SkyDef Struct Reference

Data Fields

[LTVector m_Min](#)

[LTVector m_Max](#)

[LTVector m_ViewMin](#)

[LTVector m_ViewMax](#)

Detailed Description

Sky definition.

Field Documentation

[LTVector SkyDef::m_Max](#)

[LTVector SkyDef::m_Min](#)[LTVector SkyDef::m_ViewMax](#)[LTVector SkyDef::m_ViewMin](#)

The documentation for this struct was generated from the following file:

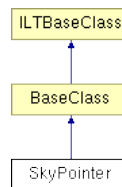
- [ltbasetypes.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

SkyPointer Class Reference

Inheritance diagram for SkyPointer:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

[LTVector](#) [m_SkyDims](#)
 float [m_InnerPercentX](#)
 float [m_InnerPercentY](#)
 float [m_InnerPercentZ](#)
[int32](#) [m_Index](#)
 char [m_ObjectName](#) [100]

Detailed Description

This works exactly the same as a [DemoSkyWorldModel](#), except you identify the object by name.

Member Function Documentation

```

virtual uint32 SkyPointer::EngineMessageFn( uint32 messageID,
                                           void * pData,
                                           float lData
                                           ) [virtual]
  
```

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "On*" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow

aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

[int32 SkyPointer::m_Index](#)

float [SkyPointer::m_InnerPercentX](#)

float [SkyPointer::m_InnerPercentY](#)

float [SkyPointer::m_InnerPercentZ](#)

char [SkyPointer::m_ObjectName](#)[100]

[LTVector SkyPointer::m_SkyDims](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SLTStorageFileInfo Struct Reference

Data Fields

[SLTTimestamp m_Create](#)
[SLTTimestamp m_Modify](#)
 [uint32 m_FileSize](#)
 [uint32 m_Attributes](#)
 char [m_FileName](#) [(256)]

Field Documentation

[uint32 SLTStorageFileInfo::m_Attributes](#)

[SLTTimestamp SLTStorageFileInfo::m_Create](#)

char [SLTStorageFileInfo::m_FileName](#)[(256)]

[uint32 SLTStorageFileInfo::m_FileSize](#)

[SLTTimestamp SLTStorageFileInfo::m_Modify](#)

The documentation for this struct was generated from the following file:

- [iltremovablestoragemgr.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SLTTimestamp Struct Reference

Data Fields

[uint8 m_Pad](#)
[uint8 m_Secs](#)
[uint8 m_Mins](#)
[uint8 m_Hours](#)
[uint8 m_Day](#)
[uint8 m_Month](#)
[uint16 m_Year](#)

Field Documentation

[uint8 SLTTimestamp::m_Day](#)

[uint8 SLTTimestamp::m_Hours](#)

[uint8 SLTTimestamp::m_Mins](#)

[uint8 SLTTimestamp::m_Month](#)

[uint8 SLTTimestamp::m_Pad](#)

[uint8 SLTTimestamp::m_Secs](#)

[uint16 SLTTimestamp::m_Year](#)

The documentation for this struct was generated from the following file:

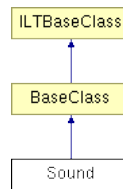
- [iltremovablestoragemgr.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Sound Class Reference

Inheritance diagram for Sound:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

char [m_Filename](#) [101]
 float [m_fOuterRadius](#)
 float [m_fInnerRadius](#)
[uint8](#) [m_nVolume](#)
 bool [m_bAmbient](#)
 unsigned char [m_nPriority](#)

Detailed Description

Sound.

Member Function Documentation

virtual [uint32](#) Sound::EngineMessageFn([uint32](#) messageID,
 void * pData,
 float lData
) [virtual]

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **lData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

bool [Sound::m_bAmbient](#)

char [Sound::m_Filename](#)[101]

float [Sound::m_fInnerRadius](#)

float [Sound::m_fOuterRadius](#)

unsigned char [Sound::m_nPriority](#)

[uint8 Sound::m_nVolume](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

Sound3DProvider Struct Reference

Data Fields

[uint32 m_dwProviderID](#)
char [m_szProvider](#)[_MAX_PATH+1]
[uint32 m_dwCaps](#)
[Sound3DProvider](#)* [m_pNextProvider](#)

Detailed Description

Contains information about a 3D sound provider.

See also:

[GetSound3DProviderLists\(\)](#), [ReleaseSound3DProviderList\(\)](#)

Field Documentation

[uint32 Sound3DProvider::m_dwCaps](#)

A combination of any of the SOUND3DPROVIDER_CAPS_XXXX flags. You can use this to identify some of the features of the 3D provider.

[uint32 Sound3DProvider::m_dwProviderID](#)

One of the SOUND3DPROVIDERID_XXX values. Many of the commonly used 3D providers have been identified with one of these values. If the provider doesn't have a unique ID, it will be set to SOUND3DPROVIDERID_UNKNOWN. If this is the case, it can be identified using the [m_szProvider](#) parameter. Compare the [m_szProvider](#) string with the known provider names listed in the Miles [Sound](#) System documentation.

[Sound3DProvider](#)* [Sound3DProvider::m_pNextProvider](#)

A pointer to the next 3D sound provider in the linked list. A NULL value indicates the end of the list. To step through all providers, keep accessing the [Sound3DProvider::m_pNextProvider](#) member variable until [Sound3DProvider::m_pNextProvider](#) is equal to NULL.

char [Sound3DProvider::m_szProvider](#)[_MAX_PATH+1]

A string descriptor identifying the 3D [Sound](#) Provider. To use a 3D sound provider, you must copy this string into the [InitSoundInfo](#) structure when initializing the sound engine.

The documentation for this struct was generated from the following file:

- [ltbasedefs.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SStaticSearchHolder Struct Reference

Data Fields

char [marker](#) [32]
[int32](#) [marker_int](#)
char [interface_name](#) [64]
char [instance_name](#) [64]
[int32](#) [version](#)

Field Documentation

char [SStaticSearchHolder::instance_name](#)[64]

char [SStaticSearchHolder::interface_name](#)[64]

char [SStaticSearchHolder::marker](#)[32]

[int32](#) [SStaticSearchHolder::marker_int](#)

[int32](#) [SStaticSearchHolder::version](#)

The documentation for this struct was generated from the following file:

- [ltmodule.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

SStaticSearchInterface Struct Reference

Data Fields

char [marker](#) [32]
[int32](#) [marker_int](#)
char [interface_name](#) [64]
char [instance_name](#) [64]
char [implementation](#) [64]
[int32](#) [version](#)

Field Documentation

char [SStaticSearchInterface::implementation](#)[64]

char [SStaticSearchInterface::instance_name](#)[64]

char [SStaticSearchInterface::interface_name](#)[64]

char [SStaticSearchInterface::marker](#)[32]

[int32](#) [SStaticSearchInterface::marker_int](#)

[int32](#) [SStaticSearchInterface::version](#)

The documentation for this struct was generated from the following file:

- [ltmodule.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

StartGameRequest Class Reference

Public Member Functions

[StartGameRequest](#) ()

Data Fields

int [m_Type](#)
 char [m_WorldName](#) [100]
 char [m_TCPAddress](#) [100]
 TCP/IP address, if any.
 char [m_RecordFilename](#) [100]
 char [m_PlaybackFilename](#) [100]

[NetSession](#) * [m_pNetSession](#)
 This must be used with STARTGAME_CLIENT.

[NetHost](#) [m_HostInfo](#)
 This must be used with STARTGAME_HOST.

[uint32](#) [m_flags](#)
 Various flags.

void * [m_pGameInfo](#)
[uint32](#) [m_GameInfoLen](#)
 void * [m_pClientData](#)
[uint32](#) [m_ClientDataLen](#)

Constructor & Destructor Documentation

[StartGameRequest::StartGameRequest](#)() [inline]

Field Documentation

[uint32](#) [StartGameRequest::m_ClientDataLen](#)

[uint32](#) [StartGameRequest::m_flags](#)

Various flags.

[uint32](#) [StartGameRequest::m_GameInfoLen](#)

[NetHost](#) [StartGameRequest::m_HostInfo](#)

This must be used with STARTGAME_HOST.

void* [StartGameRequest::m_pClientData](#)

This data gets sent up and passed into OnClientEnterWorld on the server.

void* [StartGameRequest::m_pGameInfo](#)

This data is copied over and can be accessed by the server with ServerDE::GetGameInfo() (if you're running a local or hosted game).

char [StartGameRequest::m_PlaybackFilename](#)[100]

The filename of a recorded demo. If this is filled in, the engine starts a server and fills in m_WorldName with the world that the demo record uses. You need to send a message to the server telling it to load that world.

[NetSession*](#) [StartGameRequest::m_pNetSession](#)

This must be used with STARTGAME_CLIENT.

char [StartGameRequest::m_RecordFilename](#)[100]

Filename to record into, if any (set this to 0 length if you don't want to record).

Note: When this is set, the engine starts the server but doesn't run the level; you must send a message to the server telling it to load the world.

char [StartGameRequest::m_TCPAddress](#)[100]

TCP/IP address, if any.

int [StartGameRequest::m_Type](#)

char [StartGameRequest::m_WorldName](#)[100]

The documentation for this class was generated from the following file:

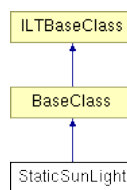
- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

StaticSunLight Class Reference

Inheritance diagram for StaticSunLight:



Public Member Functions

virtual [uint32](#) [EngineMessageFn](#) ([uint32](#) messageID, void *pData, float lData)

Data Fields

[LTVector](#) [m_InnerColor](#)
 float [m_BrightScale](#)
 float [m_ObjectBrightScale](#)
 float [m_fConvertToAmbient](#)

Detailed Description

StaticSunLight class.

Member Function Documentation

virtual [uint32](#) StaticSunLight::EngineMessageFn([uint32](#) messageID,
 void * pData,
 float lData
) [virtual]

Parameters:

messageID One of the **MID_*** define values.
pData Optional. A void pointer to data appropriate to the message type.
lData Optional. A float value appropriate to the message type.

Returns:

Returns 1.

Receives event notifications from the engine. Note that the engine actually calls one of the "**On***" notification functions, where the default implementation is to call [EngineMessageFn\(\)](#) with the appropriate **MID_***.

The meaning of **pData** and **IData** vary by message type. See the Object chapter of the LithTech Programming Guide for more information.

Before returning, this function should call the [EngineMessageFn\(\)](#) of the object's parent class to allow for inherited behavior and to allow aggregates to process the message.

EngineMessageFn will be deprecated in a future release in favor of the virtual function interface inherited from [ILTBaseClass](#).

Used For: Obsolete.

Reimplemented from [BaseClass](#).

Field Documentation

float [StaticSunLight::m_BrightScale](#)

float [StaticSunLight::m_fConvertToAmbient](#)

[LTVector StaticSunLight::m_InnerColor](#)

float [StaticSunLight::m_ObjectBrightScale](#)

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

TextureStageOps Struct Reference

Data Fields

[ERenStyle_TextureParam_TextureParam](#)
 [ERenStyle_ColorOp_ColorOp](#)
 [ERenStyle_ColorArg_ColorArg1](#)
 [ERenStyle_ColorArg_ColorArg2](#)
 [ERenStyle_AlphaOp_AlphaOp](#)
 [ERenStyle_AlphaArg_AlphaArg1](#)
 [ERenStyle_AlphaArg_AlphaArg2](#)
 [ERenStyle_UV_Source_UVSource](#)
 [ERenStyle_UV_Address_UAddress](#)
 [ERenStyle_UV_Address_VAddress](#)
 [ERenStyle_TexFilter_TexFilter](#)
 bool [UVTransform_Enable](#)
 float [UVTransform_Matrix](#) [16]
 bool [ProjectTexCoord](#)
 uint32 [TexCoordCount](#)

Detailed Description

Contains the set of operations that the render pass applies to the texture of a model piece.

See also:

[RenderPassOp](#) structure

Field Documentation

[ERenStyle_AlphaArg](#) [TextureStageOps::AlphaArg1](#)

The **ERenStyle_AlphaArg** value for this .

[ERenStyle_AlphaArg](#) [TextureStageOps::AlphaArg2](#)

The **ERenStyle_AlphaArg** value for this .

[ERenStyle_AlphaOp](#) [TextureStageOps::AlphaOp](#)

The **ERenStyle_AlphaOp** value for this .

[ERenStyle_ColorArg](#) [TextureStageOps::ColorArg1](#)

The **ERenStyle_ColorArg** value for this .

[ERenStyle_ColorArg](#) [TextureStageOps::ColorArg2](#)

The **ERenStyle_ColorArg** value for this .

[ERenStyle_ColorOp](#) [TextureStageOps::ColorOp](#)

The **ERenStyle_ColorOp** value for this .

bool [TextureStageOps::ProjectTexCoord](#)

Enable projection of uv co-ords (divide by last UV element).

uint32 [TextureStageOps::TexCoordCount](#)

The number of texture coordinates that will be output by the texture transform matrix

[ERenStyle_TexFilter](#) [TextureStageOps::TexFilter](#)

The **ERenStyle_TexFilter** value for this .

[ERenStyle_TextureParam](#) [TextureStageOps::TextureParam](#)

The **ERenStyle_TextureParam** value for this .

[ERenStyle_UV_Address](#) [TextureStageOps::UAddress](#)

The **ERenStyle_UV_Address** value for this .

[ERenStyle_UV_Source](#) [TextureStageOps::UVSource](#)

The **ERenStyle_UV_Source** value for this .

bool [TextureStageOps::UVTransform_Enable](#)

Enable matrix transform on UV Co-ords.

float [TextureStageOps::UVTransform_Matrix](#)[16]

The matrix transform.

[ERenStyle_UV_Address](#) [TextureStageOps::VAddress](#)

The **ERenStyle_UV_Address** value for this .

The documentation for this struct was generated from the following file:

- [ltrenderstyle.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

TRect2< T > Class Template Reference

Public Member Functions

[TRect2](#) ()

```

    TRect2(T l, T t, T r, T b)
    TRect2(const LTVector2< T > &vMin, const LTVector2< T > &vMax)
void Init()
void Init(T l, T t, T r, T b)
void Init(const LTVector2< T > &vMin, const LTVector2< T > &vMax)
T & Left()
T & Top()
T & Right()
T & Bottom()
const T & Left() const
const T & Top() const
const T & Right() const
const T & Bottom() const
    T GetWidth() const
    T GetHeight() const
    T GetDiagonal() const
void Sort()
bool IsSorted() const
void Expand(T Amount)
void Expand(T ExpandX, T ExpandY)
void Expand(T ExpandLeft, T ExpandTop, T ExpandRight, T ExpandBottom)
void Offset(T XOffset, T YOffset)
TRect2< T > GetIntersection(const TRect2< T > &rOther) const
TRect2< T > GetUnion(const TRect2< T > &rOther) const
    bool Overlaps(const TRect2< T > &rOther) const
    bool Contains(const LTVector2< T > pt) const
const LTVector2< T > & GetTopLeft() const
const LTVector2< T > & GetBottomRight() const
    const LTVector2< T > GetTopRight() const
    const LTVector2< T > GetBottomLeft() const

```

Data Fields

```

LTVector2< T > m\_vMin
LTVector2< T > m\_vMax

```

```

template<class T>
class TRect2< T >

```

Constructor & Destructor Documentation

```

template<class T>
TRect2< T >::TRect2( ) [inline]

```

```

template<class T>
TRect2< T >::TRect2( T    l,
                    T    t,
                    T    r,
                    T    b
                    ) [inline]

```

```

template<class T>
TRect2< T >::TRect2( const LTVector2< T > &vMin,
                    const LTVector2< T > &vMax
                    ) [inline]

```

Member Function Documentation

```

template<class T>
const T& TRect2< T >::Bottom( ) const [inline]

```

```

template<class T>
T& TRect2< T >::Bottom( ) [inline]

```

```

template<class T>
bool TRect2< T >::Contains( const LTVector2< T > pt ) const [inline]

```

```
template<class T>
void TRect2< T >::Expand( T ExpandLeft,
                        T ExpandTop,
                        T ExpandRight,
                        T ExpandBottom
                        ) [inline]
```

```
template<class T>
void TRect2< T >::Expand( T ExpandX,
                        T ExpandY
                        ) [inline]
```

```
template<class T>
void TRect2< T >::Expand( T Amount ) [inline]
```

```
template<class T>
const LTVector2<T> TRect2< T >::GetBottomLeft( ) const [inline]
```

```
template<class T>
const LTVector2<T>& TRect2< T >::GetBottomRight( ) const [inline]
```

```
template<class T>
T TRect2< T >::GetDiagonal( ) const [inline]
```

```
template<class T>
T TRect2< T >::GetHeight( ) const [inline]
```

```
template<class T>
TRect2<T> TRect2< T >::GetIntersection( const TRect2< T > & rOther ) const [inline]
```

```
template<class T>
const LTVector2<T>& TRect2< T >::GetTopLeft( ) const [inline]
```

```
template<class T>
const LTVector2<T> TRect2< T >::GetTopRight( ) const [inline]
```

```
template<class T>
TRect2<T> TRect2< T >::GetUnion( const TRect2< T > & rOther ) const [inline]
```

```
template<class T>
T TRect2< T >::GetWidth( ) const [inline]
```

```
template<class T>
void TRect2< T >::Init( const LTVector2< T > & vMin,
                     const LTVector2< T > & vMax
                     ) [inline]
```

```
template<class T>
void TRect2< T >::Init( T l,
                     T t,
                     T r,
                     T b
                     ) [inline]
```

```
template<class T>
void TRect2< T >::Init( ) [inline]
```

```
template<class T>
bool TRect2< T >::IsSorted( ) const [inline]
```

```
template<class T>
const T& TRect2< T >::Left( ) const [inline]
```

```
template<class T>
T& TRect2< T >::Left( ) [inline]
```

```
template<class T>
void TRect2< T >::Offset( T XOffset,
                        T YOffset
                        ) [inline]
```

```
template<class T>
bool TRect2< T >::Overlaps( const TRect2< T > & rOther ) const [inline]
```

```
template<class T>
const T& TRect2< T >::Right( ) const [inline]
```

```
template<class T>
T& TRect2< T >::Right( ) [inline]
```

```
template<class T>
void TRect2< T >::Sort( ) [inline]
```

```
template<class T>
const T& TRect2< T >::Top( ) const [inline]
```

```
template<class T>
T& TRect2< T >::Top( ) [inline]
```

Field Documentation

```
template<class T>
LTVector2<T> TRect2< T >::m\_vMax
```

```
template<class T>
LTVector2<T> TRect2< T >::m\_vMin
```

The documentation for this class was generated from the following file:

- [lirect.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

TRect3< T > Class Template Reference

Public Member Functions

```

TRect3 ()
TRect3 (T l, T t, T n, T r, T b, T f)
TRect3 (const TVector3< T > &vMin, const TVector3< T > &vMax)
void Init ()
void Init (T l, T t, T n, T r, T b, T f)
void Init (const TVector3< T > &vMin, const TVector3< T > &vMax)
T & Left ()
T & Top ()
T & Near ()
T & Right ()
T & Bottom ()
T & Far ()
const T & Left () const
const T & Top () const
const T & Near () const
const T & Right () const
const T & Bottom () const
const T & Far () const
    T GetWidth () const
    T GetHeight () const
    T GetDepth () const
    T GetDiagonal () const
void Sort ()
bool IsSorted () const
void Expand (T Amount)
void Expand (T ExpandX, T ExpandY, T ExpandZ)
void Expand (const TVector3< T > &Expand)
void Expand (T ExpandLeft, T ExpandTop, T ExpandNear, T ExpandRight, T ExpandBottom, T ExpandFar)
void Expand (const TVector3< T > &Min, const TVector3< T > &Max)
void Merge (const TVector3< T > &Pt)
void Merge (const TRect3< T > &rOther)
void Offset (T XOffset, T YOffset, T ZOffset)
void Offset (const TVector3< T > &Offset)
TRect3< T > GetIntersection (const TRect3< T > &rOther) const
TRect3< T > GetUnion (const TRect3< T > &rOther) const
bool Overlaps (const TRect3< T > &rOther) const
bool Contains (const TVector3< T > pt) const

```

Data Fields

[TVector3](#)< T > [m_vMin](#)
[TVector3](#)< T > [m_vMax](#)

```

template<class T>
class TRect3< T >

```

Constructor & Destructor Documentation

```

template<class T>
TRect3< T >::TRect3( ) [inline]

```

```

template<class T>
TRect3< T >::TRect3( T    l,
                    T    t,
                    T    n,
                    T    r,
                    T    b,
                    T    f
                    ) [inline]

```

```

template<class T>
TRect3< T >::TRect3( const TVector3< T > &vMin,
                    const TVector3< T > &vMax
                    ) [inline]

```

Member Function Documentation

```
template<class T>
const T& TRect3< T >::Bottom( ) const [inline]
```

```
template<class T>
T& TRect3< T >::Bottom( ) [inline]
```

```
template<class T>
bool TRect3< T >::Contains( const TVector3< T > pt ) const [inline]
```

```
template<class T>
void TRect3< T >::Expand( const TVector3< T > & Min,
                        const TVector3< T > & Max
                        ) [inline]
```

```
template<class T>
void TRect3< T >::Expand( T ExpandLeft,
                        T ExpandTop,
                        T ExpandNear,
                        T ExpandRight,
                        T ExpandBottom,
                        T ExpandFar
                        ) [inline]
```

```
template<class T>
void TRect3< T >::Expand( const TVector3< T > & Expand ) [inline]
```

```
template<class T>
void TRect3< T >::Expand( T ExpandX,
                        T ExpandY,
                        T ExpandZ
                        ) [inline]
```

```
template<class T>
void TRect3< T >::Expand( T Amount ) [inline]
```

```
template<class T>
const T& TRect3< T >::Far( ) const [inline]
```

```
template<class T>
T& TRect3< T >::Far( ) [inline]
```

```
template<class T>
T TRect3< T >::GetDepth( ) const [inline]
```

```
template<class T>
T TRect3< T >::GetDiagonal( ) const [inline]
```

```
template<class T>
T TRect3< T >::GetHeight( ) const [inline]
```

```
template<class T>
TRect3<T> TRect3< T >::GetIntersection( const TRect3< T > & rOther ) const [inline]
```

```
template<class T>
TRect3<T> TRect3< T >::GetUnion( const TRect3< T > & rOther ) const [inline]
```

```
template<class T>
T TRect3<T>::GetWidth( ) const [inline]
```

```
template<class T>
void TRect3<T>::Init( const TVector3<T> & vMin,
                    const TVector3<T> & vMax
                    ) [inline]
```

```
template<class T>
void TRect3<T>::Init( T    l,
                    T    lt,
                    T    n,
                    T    r,
                    T    b,
                    T    f
                    ) [inline]
```

```
template<class T>
void TRect3<T>::Init( ) [inline]
```

```
template<class T>
bool TRect3<T>::IsSorted( ) const [inline]
```

```
template<class T>
const T& TRect3<T>::Left( ) const [inline]
```

```
template<class T>
T& TRect3<T>::Left( ) [inline]
```

```
template<class T>
void TRect3<T>::Merge( const TRect3<T> & rOther ) [inline]
```

```
template<class T>
void TRect3<T>::Merge( const TVector3<T> & Pt ) [inline]
```

```
template<class T>
const T& TRect3<T>::Near( ) const [inline]
```

```
template<class T>
T& TRect3<T>::Near( ) [inline]
```

```
template<class T>
void TRect3<T>::Offset( const TVector3<T> & Offset ) [inline]
```

```
template<class T>
void TRect3<T>::Offset( T XOffset,
                    T YOffset,
                    T ZOffset
                    ) [inline]
```

```
template<class T>
bool TRect3<T>::Overlaps( const TRect3<T> & rOther ) const [inline]
```

```
template<class T>
const T& TRect3< T >::Right( ) const [inline]
```

```
template<class T>
T& TRect3< T >::Right( ) [inline]
```

```
template<class T>
void TRect3< T >::Sort( ) [inline]
```

```
template<class T>
const T& TRect3< T >::Top( ) const [inline]
```

```
template<class T>
T& TRect3< T >::Top( ) [inline]
```

Field Documentation

```
template<class T>
TVector3<T> TRect3< T >::m_vMax
```

```
template<class T>
TVector3<T> TRect3< T >::m_vMin
```

The documentation for this class was generated from the following file:

- [lirect.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

TVector3< T > Struct Template Reference

Public Member Functions

```

TVector3( )
TVector3(const T x, const T y, const T z)
TVector3(const LTVector3f &v)
operator LTVector3f( ) const
void Init(T a=0, T b=0, T c=0)
T Dot(const TVector3 &b) const
T MagSqr( ) const
T Mag( ) const
T LengthSquared( ) const
T Length( ) const
const TVector3 Unit( ) const
const TVector3 GetNormalized( ) const
const TVector3 & Normalize( )
bool NearlyEquals(const TVector3 &b, const T r=0) const
const TVector3 Cross(const TVector3 &b) const
const T & operator\[\](const int32 i) const
T & operator\[\](const int32 i)
bool operator==(const TVector3 &b) const
bool operator!=(const TVector3 &b) const
const TVector3 operator-( ) const
const TVector3 & operator=(const TVector3 &b)
const TVector3 & operator+=(const TVector3 &b)
const TVector3 & operator-=(const TVector3 &b)
const TVector3 & operator\*=T(const T s)
```



```

const TVector3 & operator/= (const T s)
const TVector3 operator+ (const TVector3 &b) const
const TVector3 operator- (const TVector3 &b) const
const TVector3 operator \* (const T s) const
const TVector3 operator/ (const T s) const
    bool operator> (const TVector3 &other) const
    bool operator< (const TVector3 &other) const
    bool operator>= (const TVector3 &other) const
    bool operator<= (const TVector3 &other) const
    TVector3 operator \* (const TVector3 &v) const
    TVector3 operator/ (const TVector3 &v) const
    void operator+= (T s)
    void operator-= (T s)
    T Dist (const TVector3 &b) const
    T DistSqr (const TVector3 &b) const
    T MagApprox () const
    void Norm (T nVal=1)
    void NormApprox (T nVal=1)

```

Data Fields

[T x](#)
[T y](#)
[T z](#)

Friends

const [TVector3](#) [operator *](#) (const T s, const [TVector3](#) &v)

Detailed Description

template<class T>
struct TVector3< T >

Used for: Obsolete.

Constructor & Destructor Documentation

template<class T>
[TVector3](#)< T >::TVector3() [inline]

template<class T>
[TVector3](#)< T >::TVector3(const T x,
 const T y,
 const T z
) [inline]

template<class T>
[TVector3](#)< T >::TVector3(const LTVector3f & v) [inline]

Member Function Documentation

template<class T>
 const [TVector3](#) [TVector3](#)< T >::Cross(const [TVector3](#)< T > & b) const [inline]

template<class T>
 T [TVector3](#)< T >::Dist(const [TVector3](#)< T > & b) const [inline]

template<class T>
 T [TVector3](#)< T >::DistSqr(const [TVector3](#)< T > & b) const [inline]

template<class T>
 T [TVector3](#)< T >::Dot(const [TVector3](#)< T > & b) const [inline]

```
template<class T>
const TVector3 TVector3<T>::GetNormalized( ) const [inline]
```

```
template<class T>
void TVector3<T>::Init( T a = 0,
                      T b = 0,
                      T c = 0
                    ) [inline]
```

```
template<class T>
T TVector3<T>::Length( ) const [inline]
```

```
template<class T>
T TVector3<T>::LengthSquared( ) const [inline]
```

```
template<class T>
T TVector3<T>::Mag( ) const [inline]
```

```
template<class T>
T TVector3<T>::MagApprox( ) const [inline]
```

```
template<class T>
T TVector3<T>::MagSqr( ) const [inline]
```

```
template<class T>
bool TVector3<T>::NearlyEquals( const TVector3<T> &b,
                              const T r = 0
                            ) const [inline]
```

```
template<class T>
void TVector3<T>::Norm( T nVal = 1 ) [inline]
```

```
template<class T>
const TVector3 & TVector3<T>::Normalize( ) [inline]
```

```
template<class T>
void TVector3<T>::NormApprox( T nVal = 1 ) [inline]
```

```
template<class T>
TVector3 TVector3<T>::operator *( const TVector3<T> &v ) const [inline]
```

```
template<class T>
const TVector3 TVector3<T>::operator *( const T s ) const [inline]
```

```
template<class T>
const TVector3 & TVector3<T>::operator *=( const T s ) [inline]
```

```
template<class T>
TVector3<T>::operator LTVector3f( ) const [inline]
```

```
template<class T>
bool TVector3<T>::operator!=( const TVector3<T> &b ) const [inline]
```

```
template<class T>
const TVector3 TVector3< T >::operator+( const TVector3< T > & b ) const [inline]
```

```
template<class T>
void TVector3< T >::operator+=( T s ) [inline]
```

```
template<class T>
const TVector3 & TVector3< T >::operator+=( const TVector3< T > & b ) [inline]
```

```
template<class T>
const TVector3 TVector3< T >::operator-( const TVector3< T > & b ) const [inline]
```

```
template<class T>
const TVector3 TVector3< T >::operator-( ) const [inline]
```

```
template<class T>
void TVector3< T >::operator-=( T s ) [inline]
```

```
template<class T>
const TVector3 & TVector3< T >::operator-=( const TVector3< T > & b ) [inline]
```

```
template<class T>
TVector3 TVector3< T >::operator/( const TVector3< T > & v ) const [inline]
```

```
template<class T>
const TVector3 TVector3< T >::operator/( const T s ) const [inline]
```

```
template<class T>
const TVector3 & TVector3< T >::operator/=( const T s ) [inline]
```

```
template<class T>
bool TVector3< T >::operator<( const TVector3< T > & other ) const [inline]
```

```
template<class T>
bool TVector3< T >::operator<=( const TVector3< T > & other ) const [inline]
```

```
template<class T>
const TVector3 & TVector3< T >::operator=( const TVector3< T > & b ) [inline]
```

```
template<class T>
bool TVector3< T >::operator==( const TVector3< T > & b ) const [inline]
```

```
template<class T>
bool TVector3< T >::operator>( const TVector3< T > & other ) const [inline]
```

```
template<class T>
bool TVector3< T >::operator>=( const TVector3< T > & other ) const [inline]
```

```
template<class T>
T & TVector3< T >::operator[]( const int32 i ) [inline]
```

```
template<class T>
const T& TVector3< T >::operator[] ( const int32 i ) const [inline]
```

```
template<class T>
const TVector3 TVector3< T >::Unit( ) const [inline]
```

Friends And Related Function Documentation

```
template<class T>
const TVector3 operator * ( const T s,
                           const TVector3< T > & v
                           ) [friend]
```

Field Documentation

```
template<class T>
T TVector3< T >::x
```

```
template<class T>
T TVector3< T >::y
```

```
template<class T>
T TVector3< T >::z
```

The documentation for this struct was generated from the following file:

- [ltvector.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

API Reference

VolumeEffectInfo Struct Reference

Public Types

```
enum EffectType {
    kUninitialized,
    kDynamicParticles
}
enum DynamicParticleLighting {
    kNone,
    kSinglePointNonDirectional
}
enum DynamicParticlePrimitive {
    kTrilist,
    kQuadlist
}
```

Data Fields

```
    EffectType m_EffectType
    LTVector m_Dims
VolumeEffectDPUpdateFn m_DPUpdateFn
DynamicParticlePrimitive m_DPPrimitive
    void * m_DPUserData
DynamicParticleLighting m_DPLighting
    uint32 m_DPLightConstant
    bool m_DPSaturate
```

const char * [m_DPTextureName](#)

Detailed Description

Initialization structure for a volume effect. This is passed to [ILTClient::SetupVolumeEffect\(\)](#)

Member Enumeration Documentation

enum [VolumeEffectInfo::DynamicParticleLighting](#)

Enumeration values:

kNone

kSinglePointNonDirectional

enum [VolumeEffectInfo::DynamicParticlePrimitive](#)

Enumeration values:

kTrilist

kQuadlist

enum [VolumeEffectInfo::EffectType](#)

Enumeration values:

kUninitialized

kDynamicParticles

Field Documentation

[LTVector](#) [VolumeEffectInfo::m_Dims](#)

[uint32](#) [VolumeEffectInfo::m_DPLightConstant](#)

[DynamicParticleLighting](#) [VolumeEffectInfo::m_DPLighting](#)

[DynamicParticlePrimitive](#) [VolumeEffectInfo::m_DPPrimitive](#)

[bool](#) [VolumeEffectInfo::m_DPSaturate](#)

[const char*](#) [VolumeEffectInfo::m_DPTextureName](#)

[VolumeEffectDPUUpdateFn](#) [VolumeEffectInfo::m_DPUpdateFn](#)

[void*](#) [VolumeEffectInfo::m_DPUUserData](#)

[EffectType](#) [VolumeEffectInfo::m_EffectType](#)

The documentation for this struct was generated from the following file:

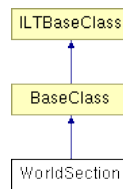
- [ltbasedefs.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

WorldSection Class Reference

Inheritance diagram for WorldSection:



Detailed Description

The WorldSection class.

The documentation for this class was generated from the following file:

- [ltengineobjects.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.